

Unit 12

SOFTWARE DEVELOPMENT

85 items · 427 marks

Syllabus sections in this unit

- 12.1 Program Development Life cycle
- 12.2 Program Design
- 12.3 Program Testing and Maintenance

Unit 12: Software Development

1. M/J 26/P21/Q3/a,b

- 3 A business operates five call centres. The software currently in use in each call centre is to be replaced; a new system will be developed by a software company.

(a) The analysis stage included a feasibility study.

State **three** other activities carried out during the analysis stage.

- 1
- 2
- 3

[3]

(b) The analysis stage has ended.

Complete the statements about the development of the new system.

Before any programming is started, the stage for the proposed new system must be completed.

For this large project, the tasks will be broken down into several ; each will be coded by a different programming team.

Once each team has completed its work, testing must be carried out.

Before the new system is delivered to the business, testing will be carried out in-house.

It is decided to install the software initially in only one of the call centres where testing will be carried out.

The new software is finally rolled out to all five call centres. In the first week of its use, staff report that the software still requires extra features which were not originally agreed. The software is withdrawn until maintenance is completed.

[6]

2. M/J 26/P21/Q8/a,b,c

- 8 Global array `MyData` contains the values shown.

Study the pseudocode algorithm and array `MyData`

```

FUNCTION MyTrace(Target : INTEGER) RETURNS INTEGER
  DECLARE LeftPointer, RightPointer, MidPointer : INTEGER
  LeftPointer ← 1
  RightPointer ← 15
  WHILE LeftPointer ≤ RightPointer
    MidPointer ← (LeftPointer + RightPointer) DIV 2
    IF MyData[MidPointer] = Target THEN
      OUTPUT "Found " & NUM_TO_STR(MidPointer)
      RETURN MidPointer
    ELSE
      IF Target > MyData[MidPointer] THEN
        LeftPointer ← MidPointer + 1
      ELSE
        RightPointer ← MidPointer - 1
      ENDIF
    ENDIF
  ENDWHILE
  OUTPUT "Not found"
  RETURN -1
ENDFUNCTION

```

	MyData
1	2
2	8
3	9
4	12
5	13
6	15
7	18
8	19
9	20
10	23
11	25
12	26
13	39
14	38
15	40

- (a) Complete the trace table for the function `MyTrace` when the following pseudocode statement is executed.

`Result ← MyTrace(15)`

LeftPointer	RightPointer	MidPointer

[4]

- (b) Give the output.

[1]

- (c) Give the value assigned to `Result`

[1]

3. M/J 26/P22/Q1/b,c,d,e,f

- 1** A program is being developed to control a simple alarm system to be used in a house. The alarm has a control panel with a keypad and a screen to display text.

The keypad consists of 36 character keys:

- 26 upper-case letter keys, allowing A to Z to be entered
- 10 digit keys, allowing 0 to 9 to be entered.

- (a)** When the alarm system is initially installed, a password must be created. When the password is initially entered, it is checked to ensure that it follows a set of validation rules. It is then permanently stored in the alarm system.

For a password to be valid it must follow these validation rules:

Rule 1: contain between six and eight characters inclusive.

Rule 2: contain at least one letter and one digit.

Rule 3: each character can only occur once.

The global variable `Pass` is used to store the password entered.

- (i)** Complete the pseudocode to declare the variable to be used to hold the password.

```
DECLARE Pass : .....
```

[1]

- (ii)** Complete the pseudocode to check that a password contains between **six** and **eight** characters inclusive (rule 1).

```
IF ..... (Pass) < 6 OR ..... (Pass) > ..... THEN
```

```
    OUTPUT "Error"
```

```
    OUTPUT "The password has to contain between 6 and 8 characters."
```

```
ENDIF
```

[2]

- (iii)** Complete the pseudocode to check that a password entered contains at least one letter and one digit character (rule 2).

```

DECLARE Index : INTEGER
DECLARE Character : CHAR
DECLARE LetterFlag, DigitFlag : .....
DigitFlag ← FALSE
LetterFlag ← .....
FOR Index ← 1 TO ..... (Pass)
    Character ← ..... (Pass, ..... , ..... )
    IF ..... (Character) = TRUE THEN
        DigitFlag ← .....
    ELSE
        LetterFlag ← .....
    ENDIF
NEXT Index
IF ..... (LetterFlag AND DigitFlag) THEN
    OUTPUT "Error"
    OUTPUT "Password needs at least one letter and one digit."
ENDIF

```

[7]

- (b) A state-transition diagram is being created as part of the design process for the module, which will control the alarm system once it has been successfully installed. Two of the states already identified are that the alarm is in:

- standby mode
- on mode (alarm system is active).

- (i) Identify **two** other states that should be part of the state-transition diagram.

1

2

[2]

- (ii) Two events that will cause a change of state are:

- entering the password when the alarm system is in standby mode
- entering the password when the alarm system is in on mode.

Identify **three** other events that would cause a change of state in the state-transition diagram for the alarm system.

1

2

3

[3]

- (c) The validation rules from 1(a) are given again:

For a password to be valid it must follow these validation rules:

Rule 1: contain between six and eight characters inclusive.

Rule 2: contain at least one letter and one digit.

Rule 3: each character can only occur once.

Complete the table with test data that could be used to test the validation rules.

Each example of test data given must:

- be invalid for the specific rule being tested
- be valid for the other **two** rules **not** being tested
- test a different aspect of the rule being tested.

Two of the tests have been completed for you.

rule being tested	example of test data	reason for invalid test data
rule 1	1A	password is too short
rule 1		
rule 2		
rule 2		
rule 3	PASSWRD2	letters are repeated
rule 3		
rule 3		

[3]

(d) Five outcomes that could be produced at the design stage in the program development lifecycle include:

- state-transition diagrams
- pseudocode
- test strategy
- test plan
- structured English.

Identify **three** other outcomes that could be produced at the design stage.

1

2

3

[3]

(e) The design and testing stages are two of the stages in the program development lifecycle used when developing the program to control the alarm system.

Identify **two** other stages in the program development lifecycle.

1

2

[2]

- (f) All the individual modules for the alarm system have now been written, tested and debugged as necessary. All these modules are combined to form the program, and further testing is now needed.

- (i) Identify this method of testing.

[1]

- (ii) One of the modules does not work as expected when added to the program.

Identify and describe a testing method that can be used to address this problem, so that the program can continue to be tested.

method

description

[3]

4. M/J 26/P23/Q3/a,b,c

- 3** A company wishes to replace the point-of-sale software that each shop uses at its checkout. A software company is developing a new system.

(a) Complete the summary about the development of the new system.

The initial stage is carried out by studying in detail the current system.

A design for the proposed new system is produced.

Following the design and coding stages, it is decided to install the completed new software initially in only two of the shops. testing is carried out.

The final stage of the project uses testing to test all programs on the company's hardware. After successful testing, the company approves the new system.

The company then installs the new software in all shops. In the first week of its use, unexpected bugs are reported. maintenance has to be carried out.

[4]

(b) For this large project, the task was divided between two programming teams.

Team A were responsible for the screen designs and all outputs; they involved the company at the earliest possible stage and obtained feedback.

Team B used a more traditional approach to the program development life cycle; the stages in the development followed an agreed sequence.

Identify the program development life cycle model adopted by each team, and describe **one** additional feature of each approach.

team A model

feature

team B model

feature

[4]

(c) State **three** activities carried out during the design stage of the program development life cycle.

1

2

3

[3]

5. M/J 26/P23/Q7/a,b

7 Study the pseudocode algorithm. Numbers will be copied into the *Data* array.

```
// STAGE 1
Num1 ← 12
Index ← 1
FOR i ← 2 TO Num1 DIV 2
    IF Num1 MOD i = 0 THEN
        Data[Index] ← i
        Index ← Index + 1
    ENDIF
NEXT i
Index ← Index - 1
OUTPUT Index, " values were found"
```

(a) (i) Complete the trace table for STAGE 1

Num1	Index	i	Data					
			1	2	3	4	5	6

[4]

(ii) State how the numbers in the *Data* array are related.

[1]

(b) The complete algorithm has a second stage.

```
// STAGE 2

Num2 ← 8
Match ← FALSE

REPEAT
    IF Num2 MOD Data[Index] = 0 THEN
        Match ← TRUE
    ELSE
        Index ← Index - 1
    ENDIF
UNTIL Match = TRUE OR Index = 0

IF Match = TRUE THEN
    OUTPUT Data[Index]
ELSE
    OUTPUT "Nothing found"
ENDIF
```

One line of the STAGE 1 code is changed:

```
Num1 ← 20
```

The programmer creates a final program with the STAGE 1 code followed immediately by the STAGE 2 code.

The STAGE 1 trace table shows the relevant final variable values after the completion of STAGE 1.

STAGE 1				Data					
Num1	Num2	Index	Match	1	2	3	4	5	6
20		4		2	4	5	10		

This is followed by running of the STAGE 2 code.

(i) Complete the trace table for STAGE 2

STAGE 2				Data					
Num1	Num2	Index	Match	1	2	3	4	5	6
20		4		2	4	5	10		

[3]

(ii) State the purpose of the STAGE 2 algorithm.

[1]

6. O/N 25/P21/Q1/b

1 A program is being developed to meet a particular customer requirement.

(b) Different test methods will be used at different stages of the program development.

Complete the table by identifying the test method that matches the test description.

The first row has been completed for you.

Test description	Test method
carried out as soon as a program module has been coded	alpha
carried out as program modules are being combined	
completed by the developers without referring to the code	
completed by the customer	

[3]

7. O/N 25/P21/Q7/a,b

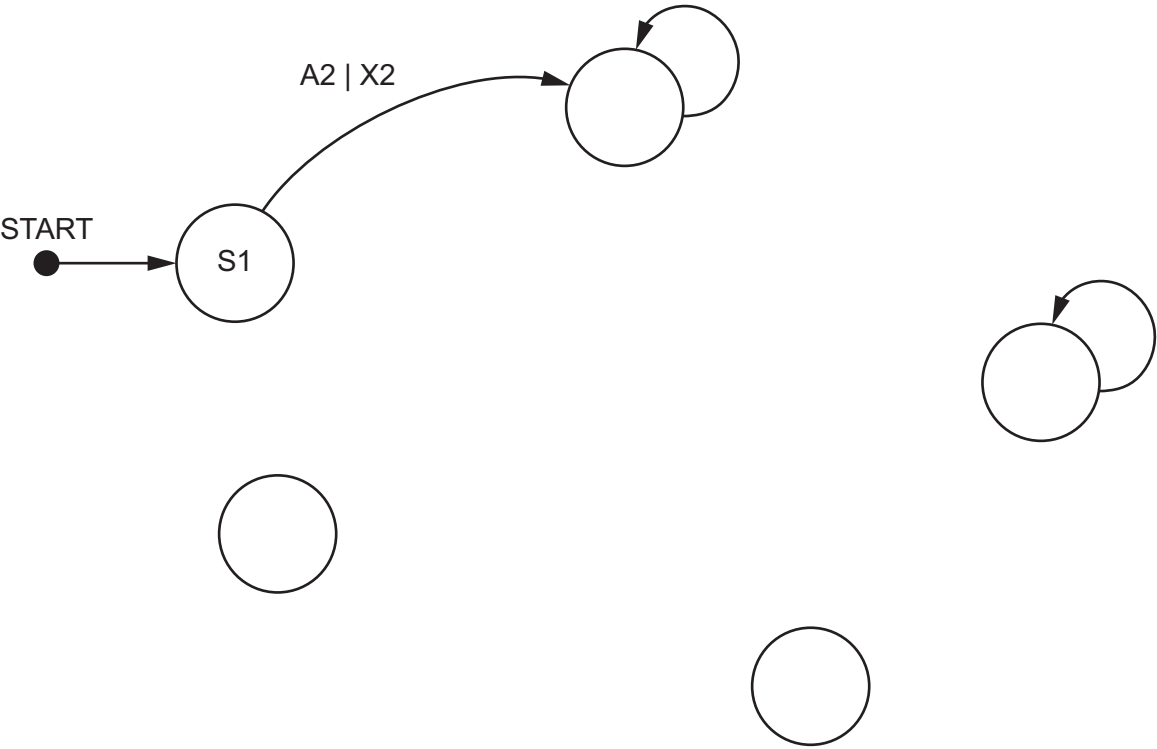
7 There are several different ways to express an algorithm during the design of a program.

(a) One part of the program contains an algorithm which is represented by a state-transition diagram.

The table shows the inputs, outputs and states for the algorithm:

Current state	Input	Output	Next state
S1	A2	X2	S3
S1	A1	X1	S2
S2	A4	X4	S5
S3	A1		S3
S3	A3	X3	S2
S3	A2	X4	S4
S4	A1	X1	S4
S4	A3		S2
S4	A4	X4	S5

Complete the state-transition diagram to represent the information given in the table.



(b) A structure chart is used to document a different part of the program.

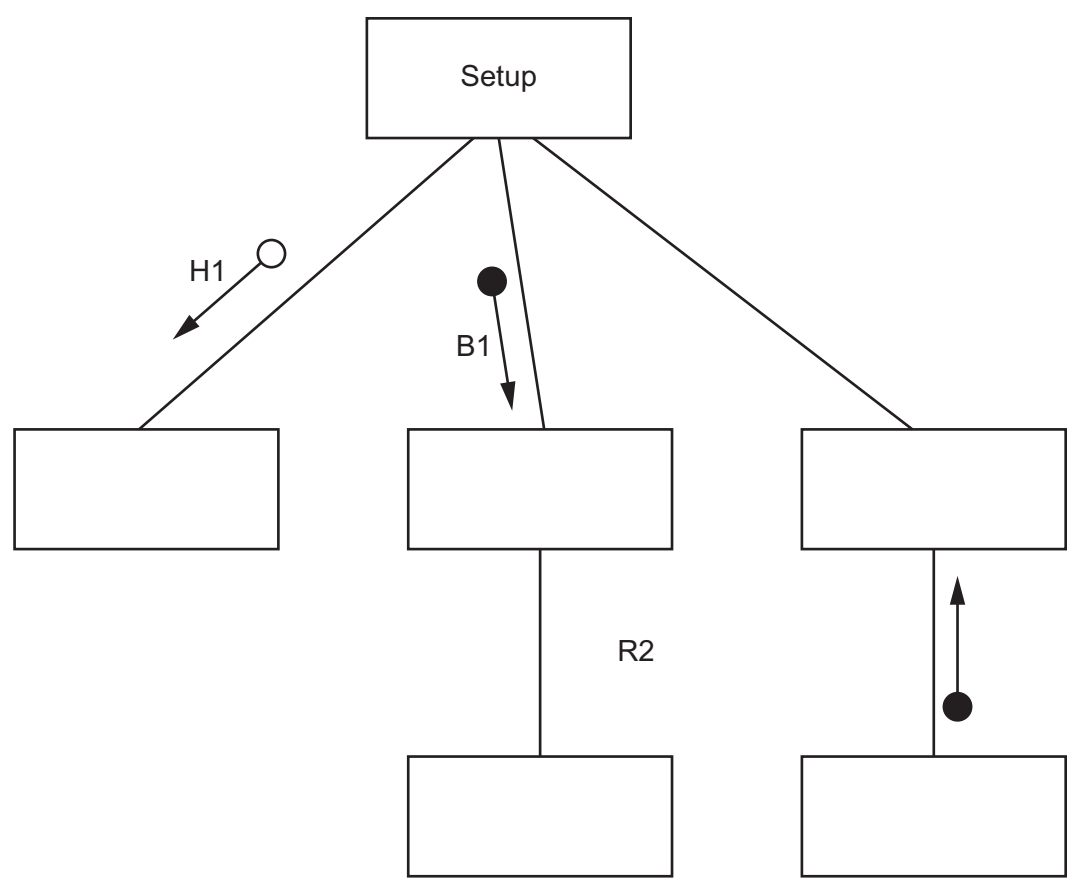
[5]

This part of the program contains six modules:

Pseudocode module header
PROCEDURE Setup()
PROCEDURE Restart(H1 : STRING, C1 : INTEGER)
FUNCTION Modify(B1 : BOOLEAN) RETURNS INTEGER
PROCEDURE Final(T1 : INTEGER)
PROCEDURE Update(BYREF R2 : STRING)
FUNCTION Confirm() RETURNS BOOLEAN

Module Setup will repeatedly call **three** of the modules.

Complete the structure chart to document the information given in the table.



8. O/N 25/P21/Q8/c

- 8 A program is being developed to manage student book loans from a college library. Students may borrow up to five books at a time from the library.

The programmer has defined a record type to define each loan.

The record data items are:

Data item	Data type	Comment
StudentID	STRING	the unique ID of the student who has borrowed the book
BookID	STRING	the unique ID of the book being borrowed
OnLoan	BOOLEAN	TRUE if the book has not been returned

The programmer has defined a global array `Loan` to store 5000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined the first program module:

Module	Description
<code>OKToBorrow()</code>	- called with a parameter of type <code>STRING</code> representing a <code>StudentID</code> - search the array for loan records for the specified student - output a suitable message to say whether the student may, or may not, borrow another book

- (c) It is decided to introduce a system of fines for books that have been borrowed for too long.

Two new requirements are defined:

- Each loan has a maximum length, represented as a number of days.
- Each book in the library will be assigned one of three categories. Each category has a different maximum loan length.

Record structure and program design changes are needed to meet these two requirements.

Outline the changes that are necessary to meet the **two** requirements.

[2]

9. O/N 25/P22/Q5/a,b

- 5** A program is developed to satisfy a specific customer requirement.

The project follows a program development life cycle model. This model divides the development process into several different stages.

- (a)** The table lists some of the development activities.

Complete the table by writing the name of the life cycle stage for each activity:

Activity	Name of life cycle stage
a structure chart is produced	
a program is modified to allow it to run on new hardware	
the programmer identifies the customer's requirements	
an Integrated Development Environment (IDE) provides context-sensitive help such as 'auto-complete'	

[4]

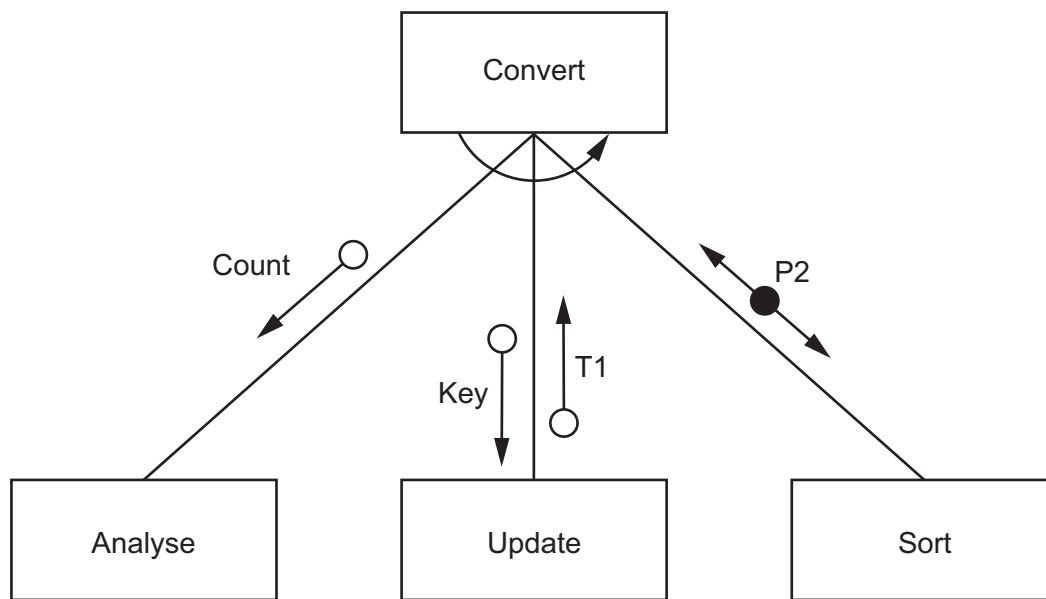
- (b)** Alpha and beta testing has been completed. The final testing stage is carried out by the customer.

Identify this final testing stage.

[1]

10. O/N 25/P22/Q7/a,b

7 The structure chart shows part of a program design:



(a) Explain the meaning of the curved arrow symbol in this structure chart.

[2]

(b) The program designer has noted that:

- Count is the number of elements in an array
- T1 is a value representing the number of elements in an array that have been modified
- Key is of type `STRING`

Write the pseudocode module headers for analyse, update and sort.

Analyse:

Update:

Sort:

[4]

11. O/N 25/P22/Q8/d

- 8 A program is being developed to manage student book loans from a college library.

The programmer has defined a record type to define each loan.

The data items are:

Data item	Data type	Comment
StudentID	STRING	the unique ID of the student who has borrowed the book
BookID	STRING	the unique ID of the book being borrowed
OnLoan	BOOLEAN	TRUE if the book has not been returned

The programmer has defined a global array `Loan` to store 7000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined a program module:

Module	Description
<code>CountLoans()</code>	- called with a parameter of type <code>STRING</code> representing a <code>StudentID</code> - counts the number of books currently on loan to the specified student - counts the number of books that the student has already returned - output both counts together with a suitable message

- (d) It is decided to extend the program so that a new module `Reminder()` will send an email to the student three days before the book is due to be returned.

Outline the changes that will need to be made to the data stored and how this data will be used to generate the reminder email.

Data

Use

[2]

12. O/N 25/P23/Q5/a

- 5 A global 1D array `Num` of integers contains four elements. A program assigns values to these elements as shown:

```
Num[1] ← 1
Num[2] ← 2
Num[3] ← 5
Num[4] ← 3
```

A procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode:

```
PROCEDURE Process(Start : INTEGER)
  DECLARE CaseVar, Index, Count : INTEGER

  Index ← Start
  Count ← 0

  WHILE Count <= 20
    CaseVar ← Num[Index]
    CASE OF CaseVar
      1 : Num[Index] ← Num[Index] + Index //clause one
        Index ← Index + 1
        Count ← Count + 1

      2 : Num[Index] ← Num[Index] + Index //clause two
        Index ← Index + 2
        Count ← Count + 2

      3 : Num[Index] ← Num[Index] * 2      //clause three
        Index ← Index + 3
        Count ← Count - 1

      4 : Count ← Count + 4                //clause four
    OTHERWISE : Count ← 20
    ENDCASE

    Index ← (Index MOD 4) + 1

  ENDWHILE
ENDPROCEDURE
```

(a) Complete the trace table by dry running the procedure when it is called in the statement:

CALL Process(1)

[illegible]

[5]

13. O/N 25/P23/Q6/a

- 6 Students are learning about a simple check digit method for data validation. In this method, a single check digit is appended to the end of an original number to give a new number.

The students are studying a method which:

- calculates the sum of all the digits in the original number
- uses integer division to calculate the remainder when the sum is divided by 10
- uses the remainder as the check digit
- appends the check digit to the original number, creating the new number.

For example:

original number	4162
sum of all digits	$4 + 1 + 6 + 2 = 13$
remainder when the sum is divided by 10 using integer division	3
new number	41623

The original number is always at least three digits in length.

When the new number is input, the check digit is used to validate the new number.

- (a) A function `Generate()` is written to take an original number as a parameter and to return the check digit.

Outline a test plan that could be used to fully test function `Generate()`

Assume that the parameter is valid.

[2]

14. O/N 25/P23/Q7/a,b

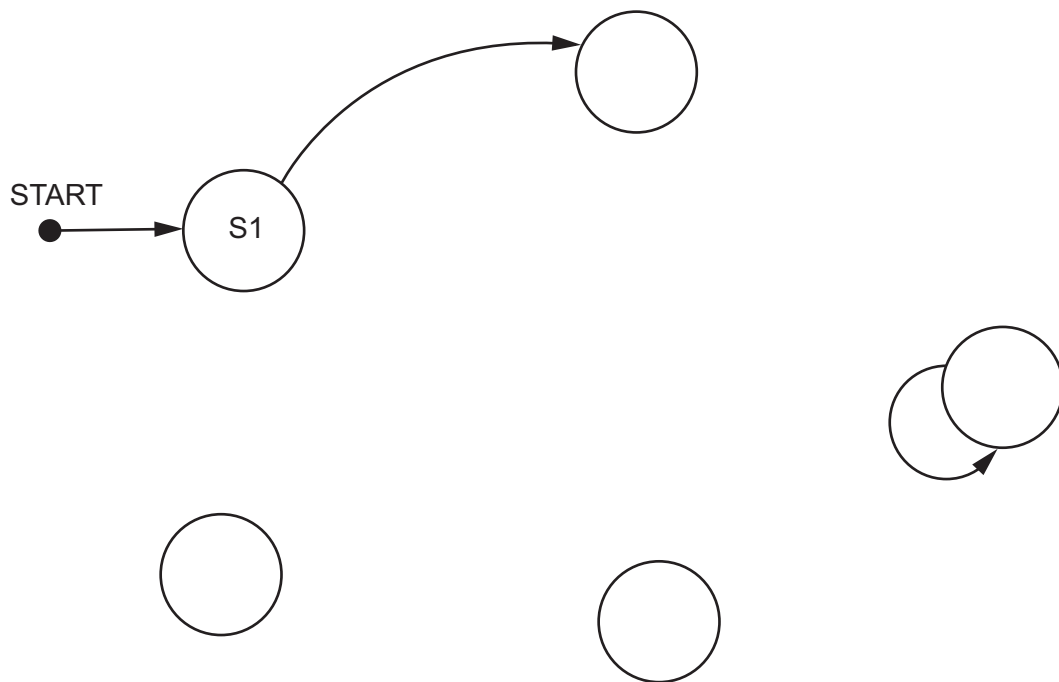
- 7 There are several different ways to express an algorithm during the design of a program.

- (a) One part of the program contains an algorithm which is represented by a state-transition diagram.

The table shows the inputs, outputs and states for the algorithm:

Current state	Input	Output	Next state
S1	A1		S2
S2	A2	X4	S3
S3	A1	X1	S3
S3	A2	X1	S3
S3	A3	X3	S4
S3	A4		S2
S4	A3	X4	S5
S4	A4	X4	S5
S4	A1		S2

Complete the state-transition diagram to represent the information given in the table:



[5]

- (b) A structure chart is used to document a different part of the program, made up of five modules.

Program notes:

- module Setup calls either module Restart, or module Confirm
- module Confirm takes a string as a parameter and returns an integer
- module Modify has no parameters and returns a Boolean
- module Update takes a string as a parameter
- module Restart repeatedly calls module Update followed by module Modify
- module Restart takes a string as a parameter that is passed by reference.

Draw a structure chart to represent the relationship between the **five** modules, including all parameters and return values.

[4]

15. M/J 25/P21/Q2/a,b,c

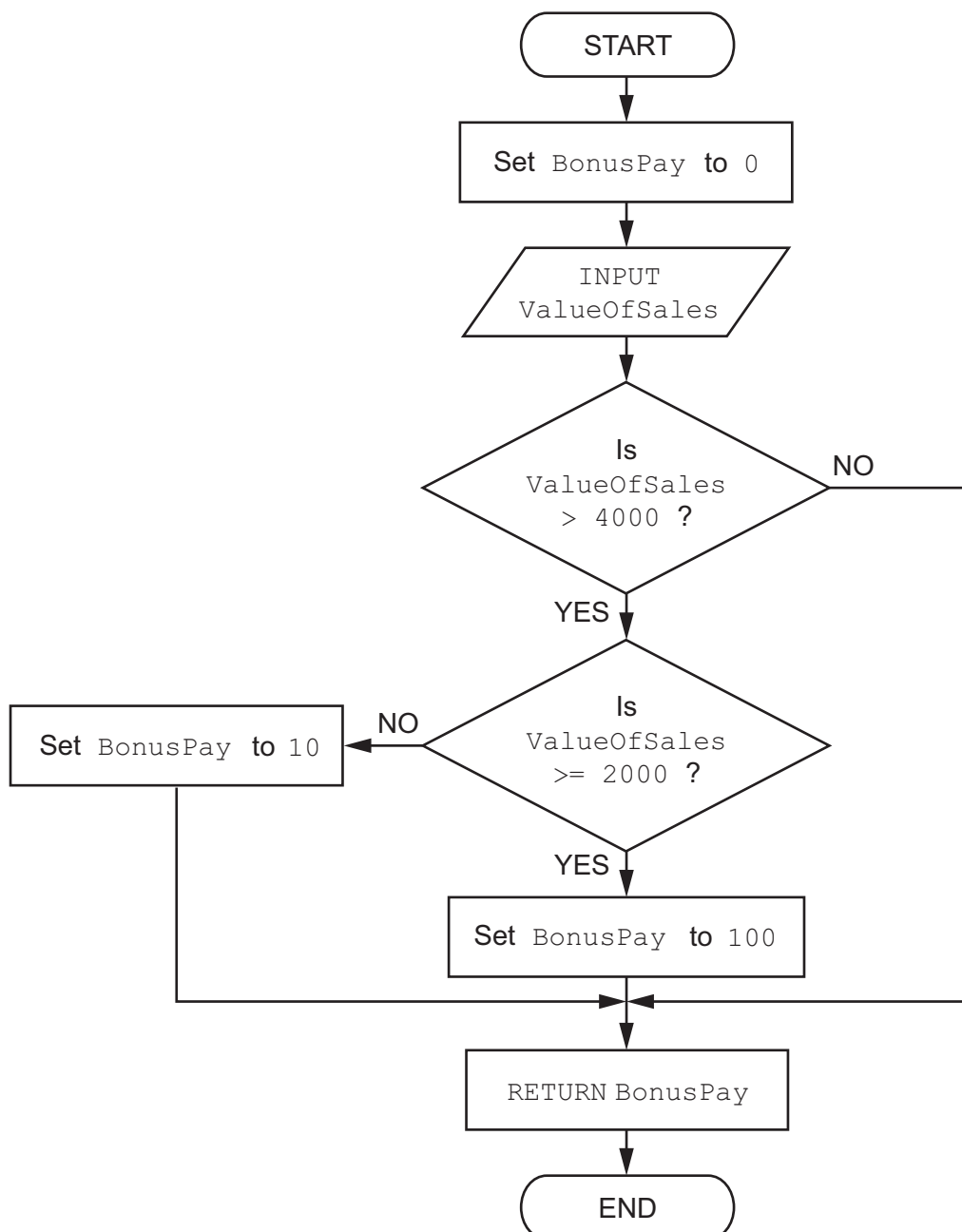
2 A program is being developed to calculate the pay of employees working for a company.

A function `CalculateBonus()` calculates bonus pay based on the value of sales.

Bonus pay is calculated as shown in the table.

Value of sales (in dollars)	Bonus pay (in dollars)
below 2000	0
between 2000 and 4000 inclusive	10
above 4000	100

A flowchart for the function `CalculateBonus()` has been designed.



(a) The flowchart contains logic errors.

One logic error is that `BonusPay` will **not** be set to 10 although the `ValueOfSales` input is between 2000 and 4000 inclusive.

(i) Explain why this error occurs.

[1]

(ii) Explain how this error could be corrected.

[1]

(b) There are different ways to reduce the risk of errors when developing the new program, such as the use of constants.

(i) State a value that could be replaced by a constant in the function `CalculateBonus()`

[1]

(ii) Explain how the use of constants helps to reduce the risk of programming errors.

[2]

(iii) One other way that can reduce the risk of errors when writing the program is the use of library routines.

Explain how library routines can reduce the risk of programming errors.

[1]

(c) All the logic errors have been corrected, and the program has been coded.

Identify and describe **two** other types of error that the program could contain.

Type of error

Description

Type of error

Description

[4]

16. M/J 25/P21/Q4/b

4 Study the algorithm:

```
DECLARE Chars : ARRAY[1:4] OF CHAR
DECLARE I, J : INTEGER
DECLARE Key : CHAR
I ← 2
Chars[1] ← 'D'
Chars[2] ← 'T'
Chars[3] ← 'H'
Chars[4] ← 'R'
WHILE I <= 4                                //Outer loop
    Key ← Chars[I]
    J ← I - 1
    WHILE J >= 0 AND Chars[J] > Key          //Inner loop
        Chars[J + 1] ← Chars[J]
        J ← J - 1
    ENDWHILE
    Chars[J + 1] ← Key
    I ← I + 1
ENDWHILE
```

(b) Complete the trace table by dry running the algorithm.

17. M/J 25/P21/Q6/a

- 6** A program monitors the speed of vehicles as they move around a large building site.

Each vehicle contains a sensor which reads an integer value that represents the speed of the vehicle. The value is expected to be in the range 0 to 60 inclusive.

The sensors cannot read values less than 0.

A program module has been written to validate the values read by the sensors.

- (a)** A test plan is needed to fully test the module.

Complete the table. The first line has been completed for you.

Assume the sensors generate only integer values.

Type of test data	Test data value	Expected outcome
normal	36	data item is accepted

[4]

18. M/J 25/P21/Q7/b(ii)

- 7** A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
points	an integer value that is increased by one for every cup of coffee the customer orders

When a customer visits the shop and orders coffee, the scheme operates as follows:

- The total number of points is increased by the number of coffees ordered.
- If just one cup of coffee is ordered and the number of points goes above 10, then:
 - the cup of coffee they have just ordered is given to them free of charge
 - the number of points is reduced by 11.
- If the order is for multiple coffees and the number of points goes above 10, then:
 - they get one coffee free of charge for every 11 points
 - the number of points is reduced by 11 for each free coffee.

For example, the:

- customer currently has 9 points
- customer orders 16 cups of coffee
- total number of points now becomes 25
- customer gets 2 free coffees and now has 3 points left.

The programmer has defined a program module that is called every time a customer places an order:

Module	Description
<code>CustomerOrder()</code>	- called with two integer parameters: - the number of coffees ordered - the current number of points - output a suitable message giving the number of free coffees - return a value for the new points total.

- (b) A text file `Loyalty.txt` will be used to store the data items for the loyalty scheme. The data items for each customer will be stored on a separate line of the text file where each data item is separated by a comma:

`<CustomerID>,<Points>`

The contents of the text file `Loyalty.txt` will always be stored in ascending order by customer ID.

When the data items are read from or written to the text file `Loyalty.txt`, they may need to be converted to the appropriate data type.

Each customer has a unique customer ID starting at "100001" with this value increasing by one each time a new customer joins the loyalty scheme.

When a customer joins the loyalty scheme, they are assigned the next customer ID and value of points is set to 0.

For example, if the loyalty scheme has 204 customers and a new customer joins the loyalty scheme, the following line is added to the text file `Loyalty.txt`:

`"100205,0"`

You can assume that the number of customers in the loyalty scheme will never be more than 9000.

The programmer has defined a program module as follows:

Module	Description
<code>AddNewCustomers()</code>	- called with an integer parameter representing the number of new customers to be added to the loyalty scheme - adds a new line, containing the required information, to the text file <code>Loyalty.txt</code> for each customer added to the loyalty scheme - outputs each new customer ID added to the loyalty scheme

- (ii) The requirements for `AddNewCustomers()` are changed. There may be no existing customers in the loyalty scheme.

Explain the changes that will need to be made to the module `AddNewCustomers()`

[4]

19. M/J 25/P22/Q1/a,b

1 A program is being developed to control the production line in a factory.

(a) A number of different program life cycles are available for the development of a program.

(i) Explain the need for different program development life cycles.

[1]

(ii) Coding is a stage in a program development life cycle.

State **one** consideration that would influence the choice of programming language.

[1]

(b) The program has been in use for a number of months and adaptive maintenance is required.

(i) Give **three** reasons why adaptive maintenance may be required.

1

2

3

[3]

(ii) As well as adaptive maintenance, other types of program maintenance may be needed.

Identify **one** other type of program maintenance.

[1]

20. M/J 25/P22/Q2/b

2 A program to calculate the pay of employees working for a company is being designed.

- (b)** One of the program modules used to calculate employees' weekly bonus pay has been completed. The amount of bonus pay they receive is based on the number of hours worked and the value of sales made as shown in the table.

Bonus pay and value of sales are both in dollars.

Hours worked	Value of sales	Bonus pay
between 1 and 40 inclusive	2000 or less	0
between 1 and 40 inclusive	above 2000	50
above 40	2000 or less	10
above 40	above 2000	100

- (i)** The module is tested using white-box testing.

State **two** tests, using valid data, that can be used to test different paths through the program.

Test one:

Hours worked

Value of sales

Bonus pay

Test two:

Hours worked

Value of sales

Bonus pay

[2]

- (ii)** The program to calculate pay uses a number of modules which are each called from different places in the program.

The program is to be tested using stub testing before all the program modules have been completed.

Describe stub testing.

[2]

- (iii)** The program has been completed and compiles successfully.

The program is tested using black-box testing.

Identify and describe **one** type of error that black-box testing could detect.

Type of error

Description

[2]

21. M/J 25/P22/Q5/a

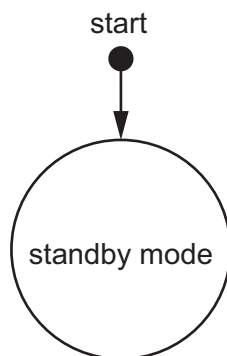
- 5 An automated digital camera system is used to take a sequence of pictures of animals in a garden. During the design of the system, a state-transition diagram is produced.

The table details the states in the automated digital camera system along with the events which cause the states to change.

The system starts in standby mode. The sequence of pictures is taken when in active mode.

Current state	Event	Next state
standby mode	turn on	detect mode
detect mode	turn off	standby mode
detect mode	movement detected	active mode
active mode	20 seconds elapsed	sequence complete
active mode	turn off	standby mode
sequence complete	time saved	detect mode

- (a) Complete the state-transition diagram for the automated digital camera system.



[4]

22. M/J 25/P22/Q6/a

also Unit 10 (3 marks); this unit 4 marks

- 6 Two arrays `Data` and `Pointer` are accessed by the procedure `Place()`

`Data` and `Pointer` are both global arrays of type `INTEGER`

The contents of these two arrays are shown:

Data		Pointer	
1	1018	1	7
2	1007	2	3
3	1010	3	1
4	1056	4	6
5	1092	5	-1
6	1062	6	5
7	1034	7	4
8	0	8	9
9	0	9	10
10	0	10	-1

Study the pseudocode:

```

PROCEDURE Place(Value : INTEGER, Start : INTEGER, Unused : INTEGER)
  DECLARE New, Current, Last : INTEGER
  CONSTANT NullPointer = -1
  New ← Unused
  Last ← NullPointer
  Current ← Start
  WHILE Current <> NullPointer AND Data[Current] < Value
    Last ← Current
    Current ← Pointer[Current]
  ENDWHILE
  Pointer[New] ← Pointer[Last]
  Pointer[Last] ← New
  Data[New] ← Value
ENDPROCEDURE

```

- (a) (i) Complete the trace table below by dry running the procedure `Place()` when it is called by the statement:

`CALL Place(1043, 2, 8)`

Value	Start	Unused	New	Last	Current

- (ii) Complete the diagram showing the contents of the global arrays `Data` and `Pointer` after the procedure `Place()` has run to completion when called as shown in part (a)(i).

	Data		Pointer
1	1018	1	
2	1007	2	
3	1010	3	
4	1056	4	
5	1092	5	
6	1062	6	
7	1034	7	
8		8	
9		9	
10		10	

[3]

23. M/J 25/P23/Q4/b

- 4 A programmer is developing a new stock control program for a shop owner to produce sales reports.

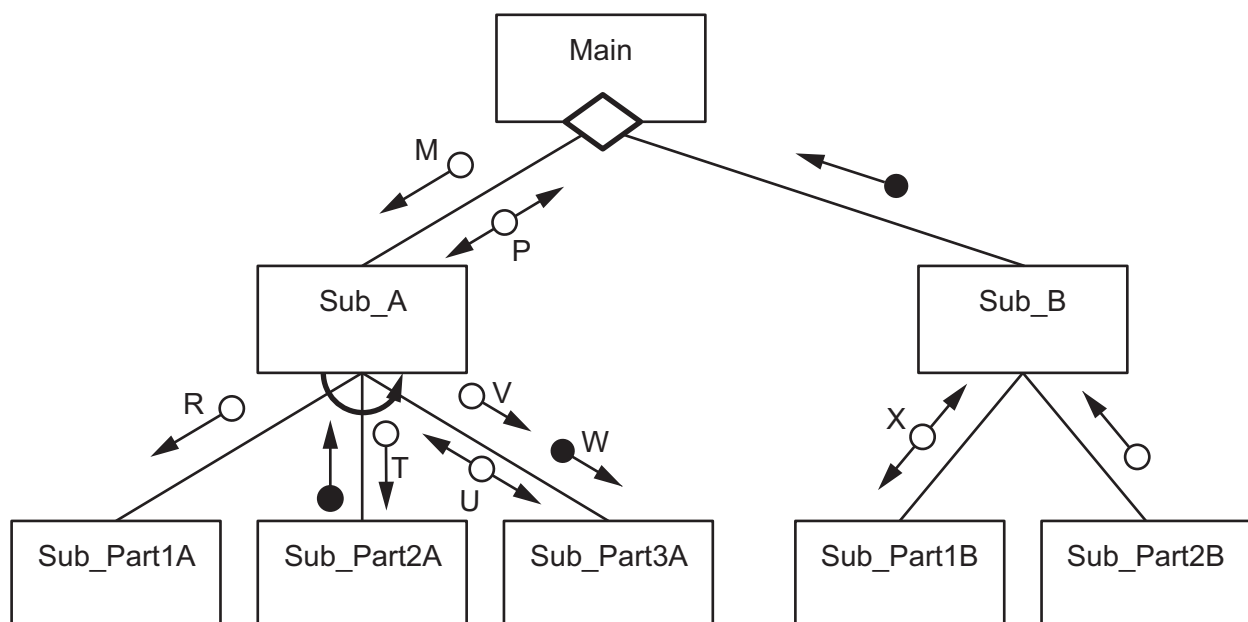
(b) Once the program has been completed, it is tested using the walkthrough method.

Describe the walkthrough method and explain how it can be used to identify errors.

[3]

24. M/J 25/P23/Q6/b,c

- 6 Study the structure chart:



Some of the parameter data types are:

- R and V are of type `INTEGER`
- T is of type `REAL`
- U is of type `STRING`
- W is of type `BOOLEAN`.

Some of the modules in the structure chart are functions, others are procedures.

(b) Write the pseudocode to define the module headers:

Sub_Part1A
Sub_Part2A
Sub_Part3A

[5]

(c) The structure chart uses the two symbols shown.

Complete the table to explain what each symbol means and how the relevant modules in the structure chart are affected.

Symbol	Explanation
	
	

[3]

25. O/N 24/P21/Q1/a

- 1** A program will calculate the tax payable based on the cost of an item.

Calculations will occur at many places in the program and these involve the use of one of three tax rates.

Tax rate values represent a percentage. For example, a tax rate value of 5.23 represents 5.23%. In this case, the tax payable on an item costing \$100 would be \$5.23.

Tax rate values are used at several places within the program. One example is given in pseudocode as follows:

```
HighRate ← FALSE
CASE OF ItemCost
    <= 50 : TaxRate ← 3.75      // tax rate of 3.75%
    <= 200 : TaxRate ← 5.23    // tax rate of 5.23%
    > 200 : TaxRate ← 6.25     // tax rate of 6.25%
        HighRate ← TRUE
ENDCASE
TaxPayable ← ItemCost * TaxRate // tax payable
```

- (a)** The pseudocode contains a logical error.

Identify the error **and** suggest a correction.

Error

Correction

[2]

26. O/N 24/P21/Q3/c

- 3** An algorithm will output the **last** three lines from a text file `Result.txt`

The lines need to be output in the same order as they appear in the file.

Assume:

- Three variables `LineX`, `LineY` and `LineZ` will store the three lines. These are of type string and all three variables have been initialised to an empty string.
- The file exists and contains **at least** three lines.

- (a)** The algorithm to output the lines is expressed in eight steps.

Complete the steps.

1. Open the file
2. Loop until
3. and store in `ThisLine`
4. Assign `LineY` to `LineX`
5. Assign `LineZ` to `LineY`
6. Assign `ThisLine` to `LineZ`
7. After the loop,
8. Output `LineX`, `LineY`, `LineZ`

[4]

- (c)** The requirement changes, and the algorithm will now output three lines from the file, starting from a **given** line number.

The modified algorithm will be implemented as a function which will:

- be called with an integer parameter representing the given line number
- output three lines, starting at the given line
- return `TRUE` if the 3 lines are output, or `FALSE` if it was not possible to output the 3 lines.

Describe the changes that need to be made to **steps 2 to 8** of the algorithm given in part **(a)**.

[4]

27. O/N 24/P21/Q4/a

- 4 A program includes the following assignment statement:

```
Result ← STR_TO_NUM(x) / STR_TO_NUM(y)
```

When the program evaluates the expression in the statement, it performs a calculation.

Variable `Result` is of type real and variables `x` and `y` are of type string.

Two checks are required before the calculation is performed:

1. The two strings represent valid numeric values.
2. The numeric value of string `y` is not zero.

- (a) Identify the type of error that could occur if these checks are **not** carried out **and** state a cause of this error.

Type

Cause

[2]

28. O/N 24/P21/Q5/a,b

- 5 A software developer follows a program development life cycle. The life cycle divides the development process into various stages.

- (a) The following table lists some development activities.

Complete the table by writing the name of the life cycle stage for each activity.

Activity	Name of life cycle stage
The walkthrough method is used.	
An algorithm is implemented in a programming language.	
The client is interviewed about problems with the current system.	
The program is modified to run on new hardware.	
Records and file structures are defined.	

[5]

- (b) The program contains a validation function.

(i) The function will:

- take an integer value as a parameter
- return `TRUE` if the value is within the range 24 to 37, inclusive
- otherwise return `FALSE`.

Complete the table to define a test plan to thoroughly test the operation of the function.

Type of test data	Test data value	Expected result
Normal	30	TRUE

[4]

(ii) The function is to be tested on its own. When it is shown to work correctly the function will be combined with other modules and testing will continue.

Identify the type of testing that this represents.

[1]

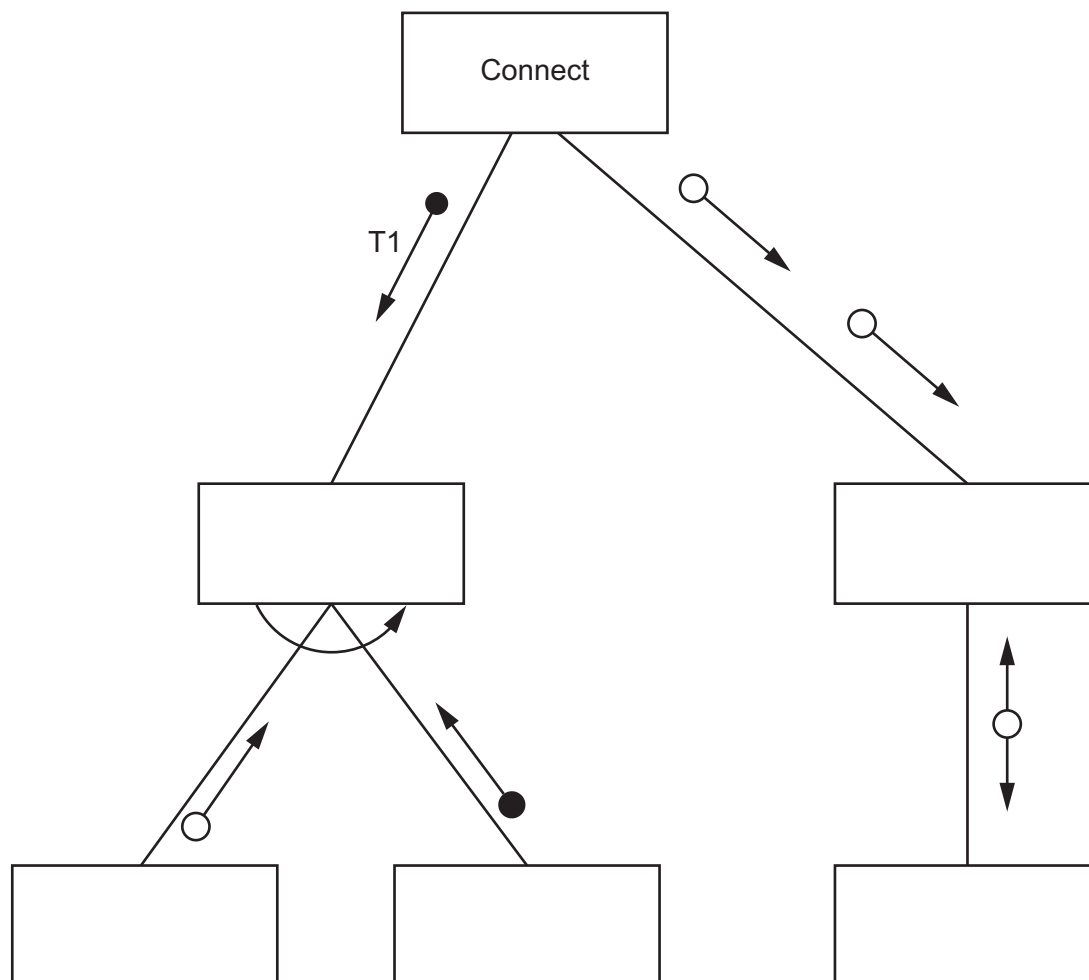
29. O/N 24/P21/Q7/a,b

7 A program contains six modules with headers as follows:

Pseudocode module header
PROCEDURE Connect ()
FUNCTION Activate (H1 : STRING, Code : INTEGER) RETURNS BOOLEAN
FUNCTION Sync (T1 : BOOLEAN, S2 : REAL) RETURNS INTEGER
PROCEDURE Initialise (BYREF ID : INTEGER, BYVAL CC : INTEGER)
FUNCTION Reset (RA : STRING) RETURNS INTEGER
FUNCTION Enable (SA : INTEGER) RETURNS BOOLEAN

Module Connect () will call either Activate () or Sync (). This is decided at run-time.

(a) Complete the structure chart for these modules.



[5]

(b) Explain the meaning of the curved arrow symbol used in the diagram in part (a).

[2]

30. O/N 24/P22/Q1/a

- 1 A program has been developed and released for general use. After a few months of use an error is detected where under certain circumstances it outputs an unexpected value.

(a) The error in the program needs to be corrected.

Identify the stage of the program development life cycle that this correction is made in.

[1]

31. O/N 24/P22/Q5/a,b

- 5 A program contains a global 1D array `Data` with 100 elements of type `INTEGER`.

The program contains a function `Process()` expressed in pseudocode as follows:

```
FUNCTION Process(Number : INTEGER, Label : STRING) RETURNS STRING
  DECLARE Index, Count : INTEGER
  DECLARE ReturnValue : STRING

  Count ← INT(100 / Number)
  Index ← Data[Number]

  CASE OF (Index MOD 2)
    0 : ReturnValue ← TO_UPPER(RIGHT(Label, Count))
    1 : ReturnValue ← "*****"
  ENDCASE

  RETURN RetVal
ENDFUNCTION
```

- (a) Run-time errors can be generated in different ways. For example, a run-time error will be generated if a function is called with invalid parameters.

The pseudocode contains three statements that could generate a run-time error.

Write the **three** statements **and** explain how each could generate a run-time error.

Statement 1

Explanation

Statement 2

Explanation

Statement 3

Explanation

[3]

- (b) One type of run-time error can cause a program to stop responding ('freezing').

Identify a particular type of programming construct that can generate this type of error **and** explain why it occurs.

Construct

Explanation

[2]

32. O/N 24/P22/Q6/b

- 6 A shop sells sandwiches and snacks. The owner chooses a 'daily special' sandwich which is displayed on a board outside the shop. Each 'daily special' has two different fillings and is made with one type of bread.

The owner wants a program to randomly choose the 'daily special' sandwich.

The program designer decides to store the possible sandwich fillings in a 1D array of type string.

The array is declared in pseudocode as follows:

```
DECLARE Filling : ARRAY [1:35] OF STRING
```

Each element contains the name of one filling.

An example of the first five elements is as follows:

Index	Element value
1	"Cheese"
2	"Onion"
3	"Salmon"
4	"Anchovies"
5	"Peanut Butter"

A second 1D array stores the possible bread used:

```
DECLARE Bread : ARRAY [1:10] OF STRING
```

Each element contains the name of one type of bread.

An example of the first three elements is as follows:

Index	Element value
1	"White"
2	"Brown"
3	"Pitta"

Both arrays may contain unused elements. The value of these will be an empty string and they may occur anywhere in each array.

A procedure `Special()` will output a message giving the 'daily special' sandwich made from **two** randomly selected different fillings and **one** randomly selected bread.

Unused array elements must **not** be used when creating the 'daily special' sandwich.

Using the above examples, the output could be:

```
"The daily special is Cheese and Onion on Brown bread."
```

- (b) The owner decides that some combinations of fillings do not go well together. For example, anchovies and peanut butter.

Describe how the design could be changed to prevent certain combinations being selected.

33. O/N 24/P22/Q7/c,d

- 7** A coffee shop runs a computerised loyalty card system.

Customers are issued with a loyalty card with their name together with a unique customer ID.

Loyalty points are added to their card each time they spend money at the shop.

The following information is stored for each customer: ID, name, home address, email address, mobile phone number, date of birth, number of points, date of last visit and amount of money spent at last visit.

A new module will generate a personalised email message to each loyalty card customer who has not visited the coffee shop in the last three months. The message will include a unique voucher code which can be used to authorise a discount if the customer goes to the shop within the next two weeks.

- (c)** It is decided to adopt a formal program development life cycle model for the development of the new module.

The analysis of the new module is complete and the project moves on to the design stage. During this stage all the necessary algorithms and module designs will be defined.

State **three other** items that will be defined for the new module during the design stage.

1

2

3

[3]

- (d)** Part of the coffee shop program contains three program modules as follows:

- Module `Init()` has no parameters and returns a Boolean.
- Module `Reset()` takes a string as a parameter and returns an Integer.
- Module `Check()` repeatedly calls `Init()` followed by `Reset()`.

Draw a structure chart to represent the relationship between the **three** modules, including all parameters and return values.

[3]

34. O/N 24/P23/Q5/a,b

- 5 A software developer follows a program development life cycle. The life cycle divides the development process into various stages.

(a) The following table lists some development activities.

Complete the table by writing the name of the life cycle stage for each activity.

Activity	Name of life cycle stage
A compiler is used.	
A program that has been released for general use is modified.	
The dry run method is used.	
The program structure is specified.	

[4]

- (b) A software developer has written modules `Test_A()` and `Test_B()`. These have been written but contain errors. These modules are called from several places in the main program and testing of the main program (integration testing) has to stop.

Identify a method that can be used to continue testing the main program **before** the errors in these modules have been corrected **and** describe how this would work.

Method

Description

[3]

35. O/N 24/P23/Q7/a

- 7 A coffee shop owner wants to introduce a computerised loyalty card system.

A programmer discusses the details of the system with the shop owner.

- (a) Identify the stage of the program development life cycle that this discussion is part of **and** give a document that will be produced during this stage.

Stage

Document

[2]

36. M/J 24/P21/Q5/a,b

- 5 A global 1D array of strings contains three elements which are assigned values as shown:

```
Data[1] ← "aaaaaa"
Data[2] ← "bbbbbb"
Data[3] ← "cccccc"
```

Procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode as follows:

```

PROCEDURE Process (Format : STRING)
    DECLARE Count, Index, L : INTEGER
    DECLARE Result : STRING
    DECLARE C : CHAR

    Result ← "*****"

    FOR Count ← 1 TO LENGTH (Format) STEP 2
        C ← MID (Format, Count, 1)
        L ← STR_TO_NUM (MID (Format, Count + 1, 1))

        Index ← (Count + 1) DIV 2

        CASE OF C
            'X' : Result ← TO_UPPER (Data[Index])
            'Y' : Result ← TO_LOWER (Data[Index])
            'Z' : Result ← "***" & Data[Index]
        ENDCASE

        Data[Index] ← LEFT (Result, L)
    NEXT Count

ENDPROCEDURE

```

(a) Complete the trace table by dry running the procedure when it is called as follows:

CALL Process ("X3Y2W4")

Count	C	L	Index	Result	Data[1]	Data[2]	Data[3]

- (b) The procedure is to be modified. If variable `C` is assigned a value other than 'X', 'Y' or 'Z', then procedure `Error()` is called and passed the value of variable `C` as a parameter.

This modification can be implemented by adding a **single line** of pseudocode.

- (i) Write the single line of pseudocode.

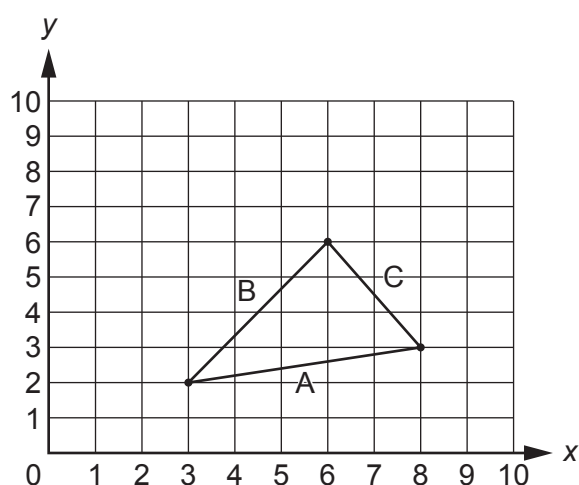
[1]

- (ii) State where this new line should be placed.

[1]

37. M/J 24/P21/Q6/b

- 6 Three points on a grid form a triangle with sides of length A, B and C as shown in the example:



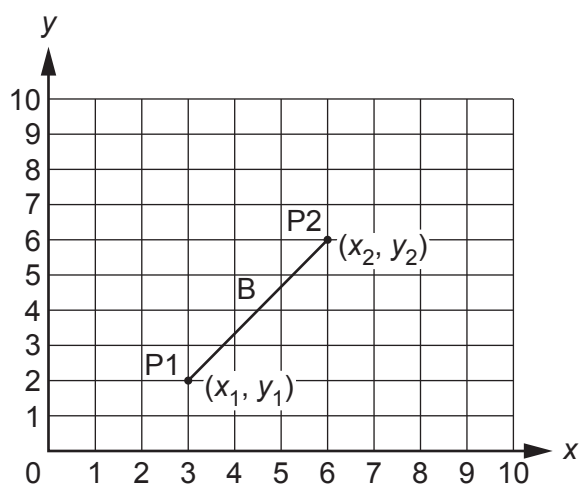
A triangle is said to be right-angled if the following test is true (where A is the length of the longest side):

$$A^2 = B^2 + C^2$$

A^2 means A multiplied by A, for example 3^2 means 3×3 which evaluates to 9

You can calculate A^2 , B^2 and C^2 by using the coordinates of the endpoints of each line.

For example, B^2 is calculated as follows:



The endpoints, P1 and P2, have the coordinates (3, 2) and (6, 6).

The value B^2 is given by the formula:

$$B^2 = (x_1 - x_2)^2 + (y_1 - y_2)^2$$

In this example:

$$B^2 = (3 - 6)^2 + (2 - 6)^2$$

$$B^2 = (-3)^2 + (-4)^2$$

$$B^2 = 9 + 16$$

$$B^2 = 25$$

(a) A function `IsRA()` will:

- take three sets of integers as parameters representing the coordinates of the three endpoints that form a triangle
- return `TRUE` if the endpoints form a right-angled triangle, otherwise return `FALSE`.

In pseudocode, the operator '^' represents an exponent, which is the number of times a value is multiplied by itself. For example, the expression Value^2 may be written in pseudocode as `Value ^ 2`

Complete the pseudocode for the function `IsRA()`.

```
FUNCTION IsRA(x1, y1, x2, y2, x3, y3 : INTEGER) RETURNS BOOLEAN
ENDFUNCTION
```

[6]

(b) The test used to check if a triangle is right-angled can be written in two ways:

$$A^2 = B^2 + C^2$$

or

$$A = \sqrt{B^2 + C^2}$$

The symbol $\sqrt{}$ represents the square root operation. For example, $\sqrt{81} = 9$

A new function `SQRT()` is written to perform the square root operation. The function takes an integer number as a parameter and returns a positive real value representing the square root of the number.

During testing it is found that the `SQRT()` function returns a value that is only accurate to 4 decimal places.

For example, `SQRT(25)` returns 5.0000125 rather than the correct value of 5.0

The function `IsRA()` from part (a) is modified to use the new `SQRT()` function to test if a triangle is right-angled.

Describe a problem that might occur when using the modified `IsRA()` function **and** suggest a solution that still allows the `SQRT()` function to be used.

Problem

Solution

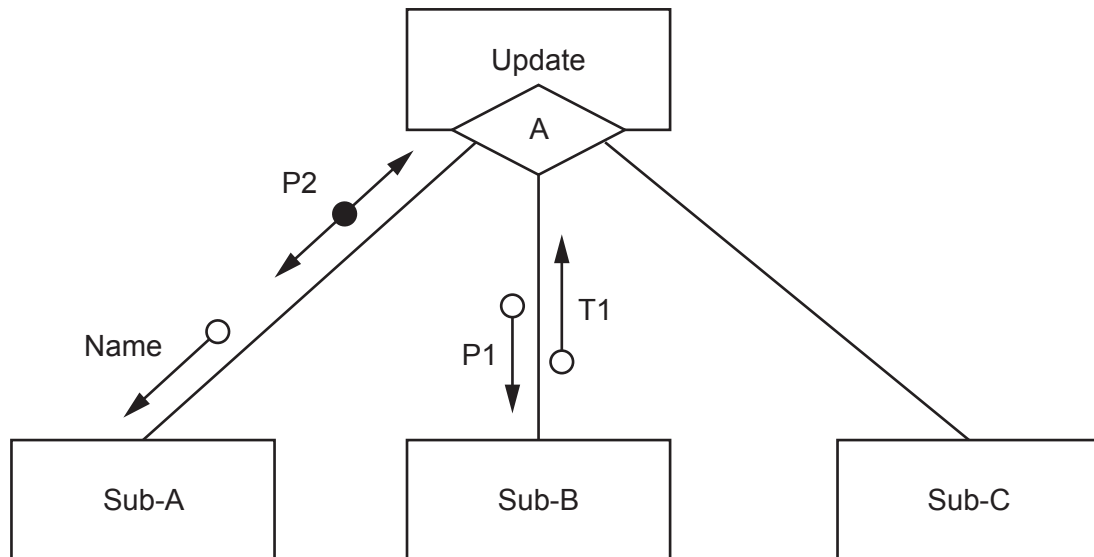
[2]

38. M/J 24/P21/Q7/b

- 7 A fitness club has a computerised membership system. The fitness club offers a number of different exercise classes.

The following information is stored for each club member: name, home address, email address, mobile phone number, date of birth and the exercise(s) they are interested in.

(b) The structure chart illustrates part of the membership program:



Data item notes:

- Name contains the name of a club member
- P1 and T1 are of type real.

(i) Explain the meaning of the diamond symbol (labelled with the letter A) in the chart.

[2]

(ii) Write the pseudocode module headers for Sub-A and Sub-B.

Sub-A

Sub-B

[4]

39. M/J 24/P22/Q5/a(i),b

5 A program is being designed in pseudocode.

The program contains a global 1D array `Data` of type string containing 200 elements.

The first element has the index value 1.

A procedure `Process()` is written to initialise the values in the array:

```
PROCEDURE Process(Label : STRING)
  DECLARE Index : INTEGER
  Index ← 0
  INPUT Data[Index]
  WHILE Index < 200
    Index ← Index + 1
    CASE OF (Index MOD 2)
      0 : Data[Index] ← TO_UPPER(Label)
      1 : Data[Index] ← TO_LOWER(Label)
      OTHERWISE : OUTPUT "Alarm 1201"
    ENDCASE
  NEXT Index
  OUTPUT "Completed " & Index & " times"
ENDPROCEDURE
```

(a) (i) The pseudocode contains **two** syntax errors and **one** other error.

Identify the errors.

Syntax error 1

Syntax error 2

Other error

[3]

(b) After correcting all syntax errors, the pseudocode is translated into program code which compiles without generating any errors.

When the program is executed it unexpectedly stops responding.

Identify the type of error that has occurred.

[1]

40. M/J 24/P22/Q7/b

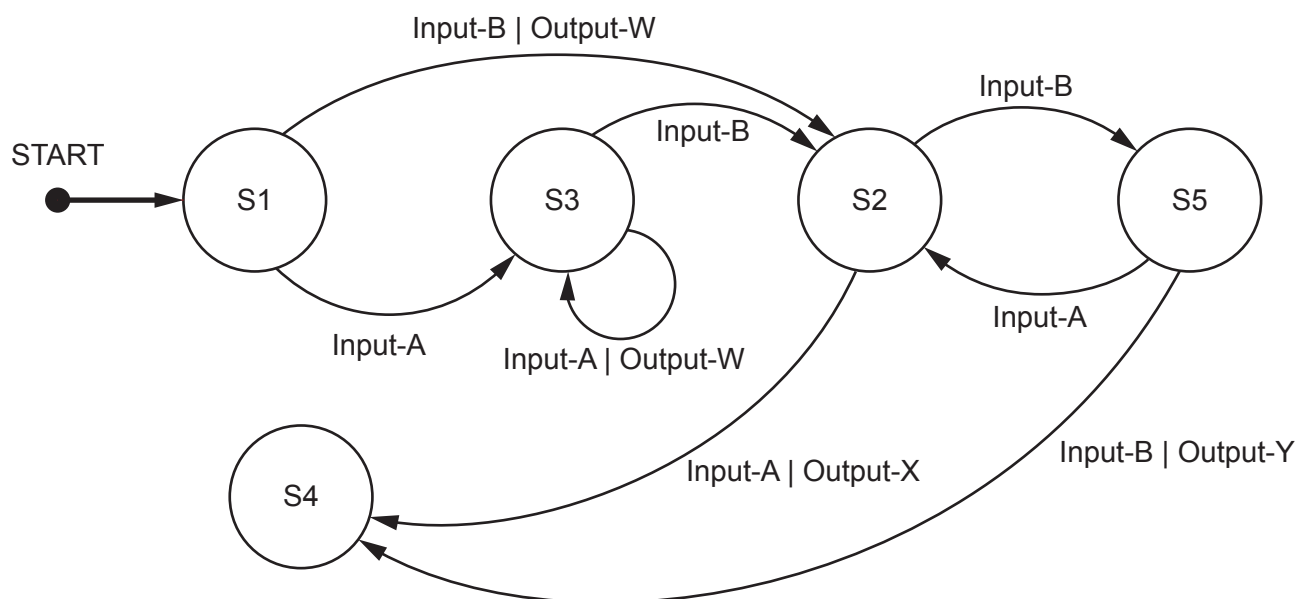
7 A fitness club has a computerised membership system.

The system stores information for each club member: name, home address, email address, mobile phone number, date of birth and exercise preferences.

Many classes are full, and the club creates a waiting list for each class. The club adds details of members who want to join a class that is full to the waiting list for that class.

When the system identifies that a space is available in one of the classes, a new module will send a text message to each member who is on the waiting list.

(b) A different part of the program is represented by the following state-transition diagram.



(i) Complete the table to show the inputs, outputs and next states.

Assume that the current state for each row is given by the 'Next state' on the previous row. For example, the first Input-A is made when in state S1.

If there is no output for a given transition, then the output cell should contain 'none'.

The first two rows have been completed.

Input	Output	Next state
		S1
Input-A	none	S3
	Output-W	
	none	
Input-B		
Input-A		
		S4

[5]

(ii) Identify the input sequence that will cause the minimum number of state changes in the transition from S1 to S4.

[1]

41. M/J 24/P23/Q6/b(ii)

- 6 A program displays a progress bar to inform the user of the progress of tasks that take a significant time to complete, such as those involving file transfer operations.

Task progress is divided into 11 steps. Each step represents the amount of progress as a percentage. An image is associated with each step and each image is stored in a different file.

Different progress bar images may be selected. For a given image, files all have the same filename root, with a different suffix.

The table illustrates the process for using the image with filename root `BargraphA`

Step	Percentage progress	Image filename	Image
1	< 10	BargraphA-1.bmp	
2	>= 10 and < 20	BargraphA-2.bmp	
3	>= 20 and < 30	BargraphA-3.bmp	
   			
9	>= 80 and < 90	BargraphA-9.bmp	
10	>= 90 and < 100	BargraphA-10.bmp	
11	100	BargraphA-11.bmp	

A procedure `Progress()` will:

- be called with two parameters:
 - an integer representing the percentage progress (0 to 100 inclusive)
 - a string representing the image filename root
- generate the full image filename
- call a procedure `Display()` using the full image filename as the parameter.



(b) The definition of procedure `Progress()` is provided here for reference:

A procedure `Progress()` will:

- be called with two parameters:
 - an integer representing the percentage progress (0 to 100 inclusive)
 - a string representing the image filename root
- generate the full image filename
- call a procedure `Display()` using the full image filename as the parameter.

`Progress()` will be rewritten and a new module `Progress2()` produced with these requirements:

- an additional parameter of type integer will specify the total number of steps
- the image filename will be returned (procedure `Display()` will **not** be called from within `Progress2()`).

(ii) State **one** benefit of increasing the number of steps.

[1]



42. M/J 24/P23/Q7/a(i),b

- 7 Seven program modules form part of a program. A description of the relationship between the modules is summarised below. Any return values are stated in the description.

Module name	Description
Mod-A	calls Mod-B followed by Mod-C
Mod-B	- called with parameters Par1 and Par2 - calls either Mod-D or Mod-E, determined when the program runs - returns a Boolean value
Mod-C	- called with parameters Par1 and Par3 - Par3 is passed by reference - repeatedly calls Mod-F followed by Mod-G
Mod-D	called with parameter Par2
Mod-E	- called with parameter Par3 - returns an integer value
Mod-F	called with parameter Par3
Mod-G	- called with parameter Par3 - Par3 is passed by reference

Parameters in the table are as follows:

- Par1 and Par3 are of type string.
- Par2 is of type integer.

- (a) (i) Identify the modules that would be implemented as functions.

[1]

- (b) Draw a structure chart to show the relationship between the seven modules and the parameters passed between them.

[6]

**43. O/N 23/P21/Q5/a,b**

- 5 An algorithm is designed to find the smallest numeric value from an input sequence and count how many numeric values have been input.

An example of an input sequence is:

23, AB56, 17, 23ZW, 4, 10, END

Numeric input values are all integers and non-numeric input is ignored, except for the string "END" which is used to terminate the sequence.

The algorithm is expressed in pseudocode as shown:

```
DECLARE NextInput : STRING
DECLARE Min, Count, Num : INTEGER

Min ← 999
Count ← 0

REPEAT
    INPUT NextInput
    IF IS_NUM(NextInput) = TRUE THEN
        Num ← STR_TO_NUM(NextInput)
        IF Num > Min THEN
            Min ← Num
        ENDIF
        Count ← Count & 1
    ENDIF
UNTIL NextInput ← "END"

OUTPUT "The minimum value is ", Min, " and the count was ", Count
```

(a) The pseudocode contains three errors due to the incorrect use of **operators**.

Identify each error **and** state the correction required.

1

2

3

[3]

(b) The operator errors are corrected and the algorithm is tested as follows:

The input sequence:

18, 4, ONE, 27, 189, ERIC, 3, 65, END

produces the output:

The minimum value is 3 and the count was 6

The algorithm is tested with a different test data sequence. The sequence contains a mix of integer and non-numeric values. It is terminated correctly but the algorithm produces unexpected results.

(i) Explain the problem with the algorithm.

[2]

(ii) Give a sequence of **four** test data values that could be input to demonstrate the problem.

Value 1

Value 2

Value 3

Value 4

[2]

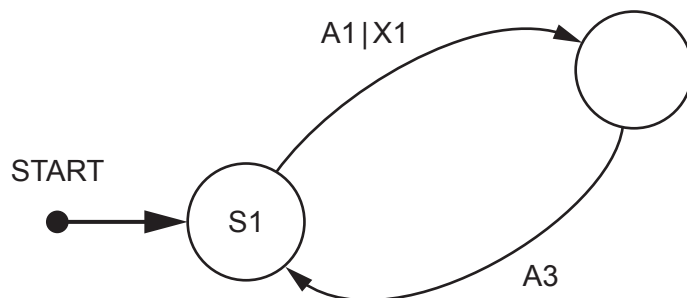
44. O/N 23/P21/Q7

7 An algorithm is represented by a state-transition diagram.

The table shows the inputs, outputs and states for the algorithm:

Current state	Input	Output	Next state
S1	A1	X1	S2
S2	A3	none	S1
S2	A2	X4	S5
S5	A1	X1	S5
S5	A4	X2	S2
S5	A3	none	S3
S1	A9	X9	S3
S3	A9	X9	S4

Complete the state-transition diagram to represent the information given in the table.



[5]

45. O/N 23/P21/Q8/c

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

<STX><DestinationID><SourceID><Data><ETX>

Field number	Field name	Description
n/a	STX	a single character marking the start of the message (ASCII value 02)
1	DestinationID	three numeric characters that identify the destination computer
2	SourceID	three numeric characters that identify the source computer
3	Data	a variable length string containing the data being sent (Minimum length is 1 character)
n/a	ETX	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Kevin" being sent from computer "101" to computer "232":

```
<STX>"232101Hello Kevin"<ETX>
```

Each computer will run a copy of the same program. Each program will contain a global variable, `MyID` of type string, that contains the unique ID of the computer in which the program is running.

The programmer has defined the first two program modules as follows:

Module	Description
<code>Transmit()</code> (already written)	• takes two parameters: ◦ a string containing a message ◦ an integer containing a port number • transmits the message using the given port
<code>SendFile()</code>	• takes three parameters: ◦ a string containing a text file name ◦ a string containing a Destination ID ◦ an integer containing a Port number • transmits the file one line at a time • transmits a final message with data string "****"

(c) One of the text files to be sent contains several blank lines (lines that do not contain any text).

(i) Explain why this is a problem.

[2]

(ii) Explain how the message format could be changed to allow a blank line to be sent.

[2]

46. O/N 23/P22/Q4/b

4 A procedure `Count ()` will:

1. input a value (all values will be positive integers)
2. count the number of odd values and count the number of even values
3. repeat from step 1 until the value input is 99
4. output the two count values, with a suitable message.

The value 99 must not be counted.

(b) The procedure `Count ()` is to be tested.

Typical test data would consist of odd and even values, for example:

23, 5, 64, 100, 2002, 1, 8, 900, 99

The purpose of this test would be to test a typical mix of even and odd values and check the totals.

Give **three** test data sequences that may be used to test **different** aspects of the procedure.

Do **not** include invalid data.

Sequence 1:

Test data

Purpose of test.

Sequence 2:

Test data

Purpose of test.

Sequence 3:

Test data

Purpose of test.

[3]

47. O/N 23/P22/Q5

5 A global 1D array of integers contains four elements, which are assigned values as shown:

```
Mix[1] ← 1
Mix[2] ← 3
Mix[3] ← 4
Mix[4] ← 2
```

A procedure `Process ()` manipulates the values in the array.

The procedure is written in pseudocode:

```
PROCEDURE Process(Start : INTEGER)
  DECLARE Value, Index, Count : INTEGER

  Index ← Start
  Count ← 0

  REPEAT
    Value ← Mix[Index]
    Mix[Index] ← Mix[Index] - 1
    Index ← Value
    Count ← Count + 1
  UNTIL Count = 5

  Mix[4] ← Count * Index

ENDPROCEDURE
```

Complete the trace table on the opposite page by dry running the procedure when it is called as follows:

```
CALL Process(3)
```

[illegible]

[6]

48. O/N 23/P22/Q7/a,b

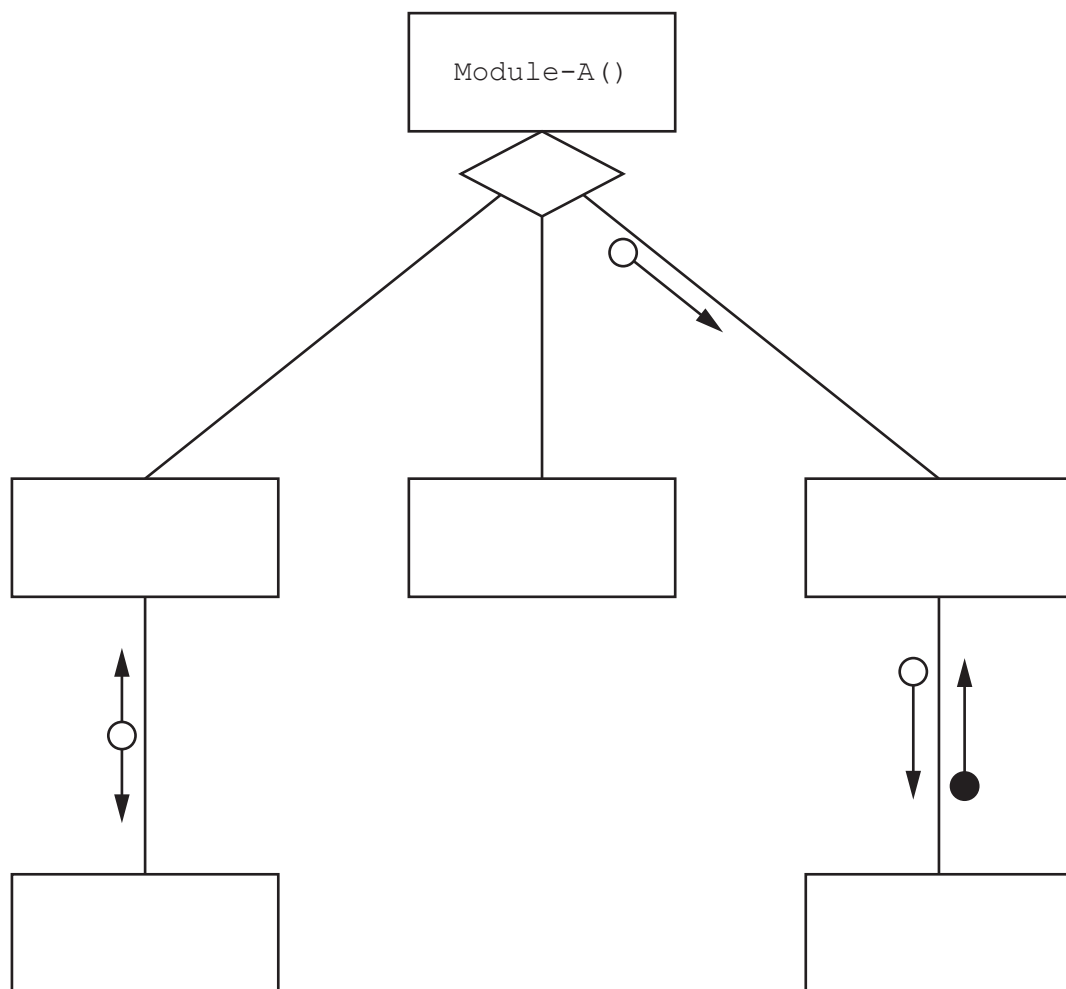
7 A program contains six modules:

Pseudocode module header
PROCEDURE Module-A()
PROCEDURE Module-X(T1 : INTEGER, S2 : REAL)
PROCEDURE Reset(BYREF Code : INTEGER)
FUNCTION Restore(OldCode : INTEGER) RETURNS BOOLEAN
FUNCTION Module-Y(RA : INTEGER, RB : BOOLEAN) RETURNS BOOLEAN
FUNCTION Module-Z(SA : INTEGER) RETURNS INTEGER

Module-X() **calls** Reset()

Module-Y() **calls** Restore()

(a) Complete the structure chart for these modules.



[4]

(b) Explain the meaning of the diamond symbol as used in the diagram in part (a).

[2]

49. O/N 23/P23/Q1/d

- 1 A program is being developed in pseudocode before being converted into a programming language.
- (d) The hardware that runs the program is changed and the program needs to be modified so that it works with the new hardware.

Identify the type of maintenance that this represents **and** give **one other** reason why this type of maintenance may be needed.

Type

Reason

[2]

50. O/N 23/P23/Q5

- 5 A global 1D array of integers contains four elements, which are assigned values as shown:

```
Mix[1] ← 4
Mix[2] ← 2
Mix[3] ← 3
Mix[4] ← 5
```

A procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode as follows:

```
PROCEDURE Process(Start : INTEGER)
    DECLARE Value, Index, Total : INTEGER

    Index ← Start
    Total ← 0

    WHILE Total < 20
        Value ← Mix[Index]
        Total ← Total + Value

        IF Index < 4 THEN
            Mix[Index] ← Mix[Index] + Mix[Index+1]
        ELSE
            Mix[Index] ← Mix[Index] + Mix[1]
        ENDIF
        Index ← (Value MOD 4) + 1

    ENDWHILE

    Mix[1] ← Total * Index

ENDPROCEDURE
```

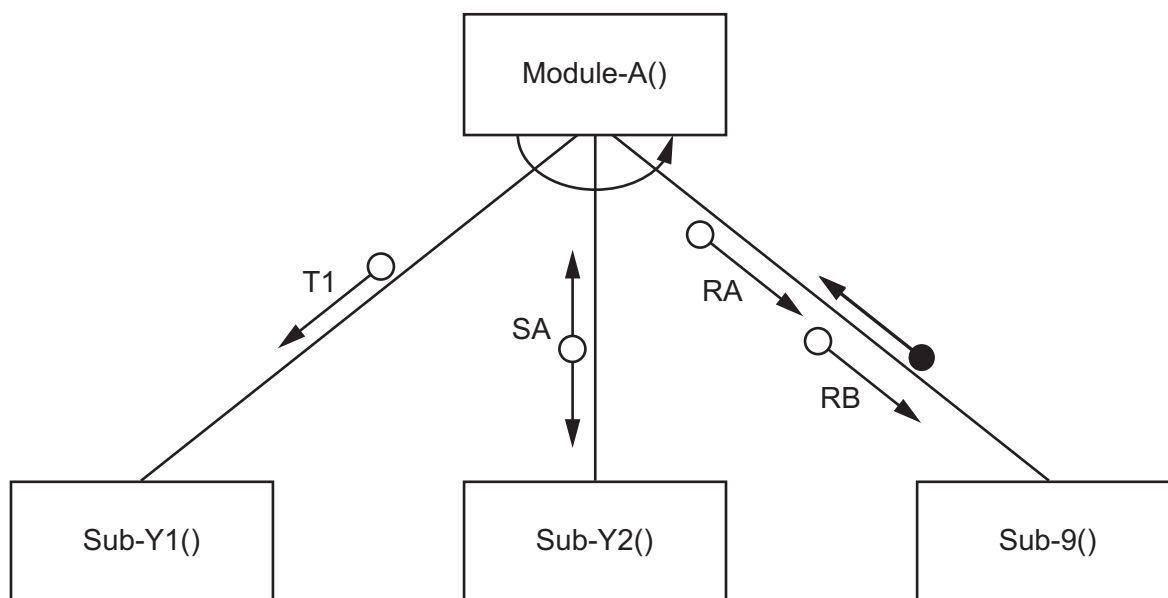
Complete the trace table on the opposite page by dry running the procedure when it is called as follows:

```
CALL Process(2)
```

Index	Value	Total	Mix[1]	Mix[2]	Mix[3]	Mix[4]

51. O/N 23/P23/Q7/a

7 A structure chart shows the modular structure of a program:



(a) Explain the meaning of the curved arrow symbol which begins and ends at Module-A().

[2]

52. O/N 23/P23/Q8/b,d

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

<STX><DestinationID><SourceID><Data><ETX>

Field name	Description
STX	a single character marking the start of the message (ASCII value 02)
DestinationID	three numeric characters identifying the destination computer
SourceID	three numeric characters identifying the source computer
Data	a variable length string containing the data being sent (Minimum length is 1 character)
ETX	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Jack" being sent from computer "202" to computer "454":

```
<STX>"454202Hello Jack"<ETX>
```

Each computer will run a copy of the same program. Each program will contain a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.

The first two program modules are defined as follows:

Module	Description
<code>GetData()</code> (already written)	• returns the data field from a message that has been received • If no message is available, the module waits until one has been received.
<code>ReceiveFile()</code>	• takes a file name as a parameter of type string • creates a text file with the given file name (no checking required) • writes the data field returned by <code>GetData()</code> to the file • repeats until the data field is "*****", which is not written to the file • outputs a final message giving the total number of characters written to the file, for example: 132456 characters were written to newfile.txt

- (b) The use of the string "*****" as explained in the module description for `ReceiveFile()` may cause a problem.

Explain the problem and suggest a solution.

Problem

Solution

[3]

(c) Two new modules are defined, which will allow two users to exchange messages.

Module	Description
<code>Transmit()</code> (already written)	- takes two parameters: - a string representing a message - an integer representing a port number - transmits the message using the given port
<code>Chat()</code>	- takes two parameters: - a string representing a Destination ID - an integer representing a port number - extracts data from a received message using <code>GetData()</code> and outputs it - forms a message using data input by the user and sends it using <code>Transmit()</code> - repeats until either the output string or the sent string is "Bye"

Reminders:

- Each program contains a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.
- Messages are sent between computers as a string of characters organised into fields as shown:

`<STX><DestinationID><SourceID><Data><ETX>`

Write pseudocode for module `Chat()`.

Modules `GetData()` and `Transmit()` must be used.

[7]

(d) Module `GetData()` returns the data field from a message that has been received. If no message is available, the module waits until one has been received.

Explain the limitation of this on module `Chat()` from part (c).

Describe a modification to `GetData()` to address this limitation.

Limitation

Modification

[3]

53. M/J 23/P21/Q5/a,b

- 5** Several companies are developing websites to market a new type of games console. The company that is first to create a website that can demonstrate the interactive features of the games console will have an advantage over the others. The requirements for the website are likely to change as more information about the features of the console are made available.

One company has decided to develop their website using a program development life cycle based on the waterfall model.

- (a) (i)** Give **two** reasons why this may **not** be the most appropriate model to use in this case.

Reason 1

Reason 2

[2]

- (ii)** Identify a **more appropriate** program development life cycle model for this scenario.

[1]

- (b)** The website has been running in test mode for several weeks.

Identify **and** describe a final stage of testing that should take place before the website is made available to all customers.

Stage

Description

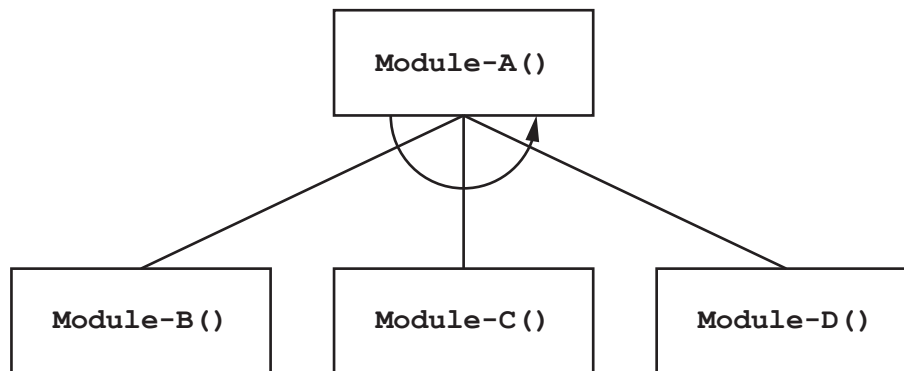
[3]

54. M/J 23/P21/Q7/b

- 7** A school has a computerised library system that allows students to borrow books for a fixed length of time. The system uses text files to store details of students, books and loans.

A new module is to be written which will generate emails to each student who has an overdue book.

- (b)** The program designer produces a structure chart for the new module. Part of the structure chart is shown:



- (i)** Explain the relationship between the four modules shown.

[2]

- (ii)** Two new modules are added: `Module-X()` and `Module-Y()`.

- `Module-X()` has no parameters.
- `Module-Y()` will take a string and a real number as parameters and return a Boolean value.
- `Module-D()` will call either `Module-X()` or `Module-Y()`.

Draw **only** the part of the structure chart that represents the relationship between `Module-X()`, `Module-Y()` and `Module-D()`.

[3]

55. M/J 23/P21/Q8/c

- 8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique for each item in the range "0001" to "5999" inclusive
SupplierCode	5 alphabetic characters	to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
<code>SuppExists()</code> (already written)	- called with a parameter of type string representing a supplier code - returns <code>TRUE</code> if the supplier code is already in use, otherwise returns <code>FALSE</code>
<code>IsNewSupp()</code>	- called with a parameter of type string representing a new supplier code - returns <code>TRUE</code> if the string only contains alphabetic characters (either upper or lower case) and the supplier code is not already in use, otherwise returns <code>FALSE</code>

- (b) A new module has been defined:

Module	Description
<code>CheckNewItem()</code>	- called with a parameter of type string representing a line of item information - checks to see whether an item with the same <code>ItemNum</code> already exists in the file - returns <code>TRUE</code> if the <code>ItemNum</code> is not already in the file, otherwise returns <code>FALSE</code>

Write **efficient** pseudocode for module `CheckNewItem()`.

[7]

- (c) The program modules `SuppExists()`, `IsNewSupp()` and `CheckNewItem()` are part of a group of modules that are combined to create a complete stock control program.

Each module in the program is tested individually during development and is debugged as necessary. It is then added to the program and further testing performed.

(i) Identify this method of testing.

[1]

(ii) One of the modules does not work properly when it is added to the program.

Describe a testing method that can be used to address this problem so that testing can continue and other modules can be added.

[2]

56. M/J 23/P22/Q5/a,b,d

- 5** A programmer has produced the following pseudocode to output the square root of the numbers from 1 to 10.

Line numbers are for reference only.

```
10 DECLARE Num : REAL
11 Num ← 1.0
...
40 REPEAT
41     CALL DisplaySqrt(Num)
42     Num ← Num + 1.0
43 UNTIL Num > 10
...
50 PROCEDURE DisplaySqrt(BYREF ThisNum : REAL)
51     OUTPUT ThisNum
52     ThisNum ← Sqrt(ThisNum)           // Sqrt returns the square root
53     OUTPUT " has a square root of ", ThisNum
54 ENDPROCEDURE
```

The pseudocode is correctly converted into program code.

Function `Sqrt()` is a library function and contains no errors.

The program code compiles without errors, but the program gives unexpected results. These are caused by a design error.

- (a)** Explain why the program gives unexpected results.

[3]

- (b)** Explain why the compiler does **not** identify this error.

[1]

- (d)** The pseudocode is converted into program code as part of a larger program.

During compilation, a complex statement generates an error.

The programmer does not want to delete the complex statement but wants to change the statement so that it is ignored by the compiler.

State how this may be achieved.

[1]

57. M/J 23/P22/Q7/b

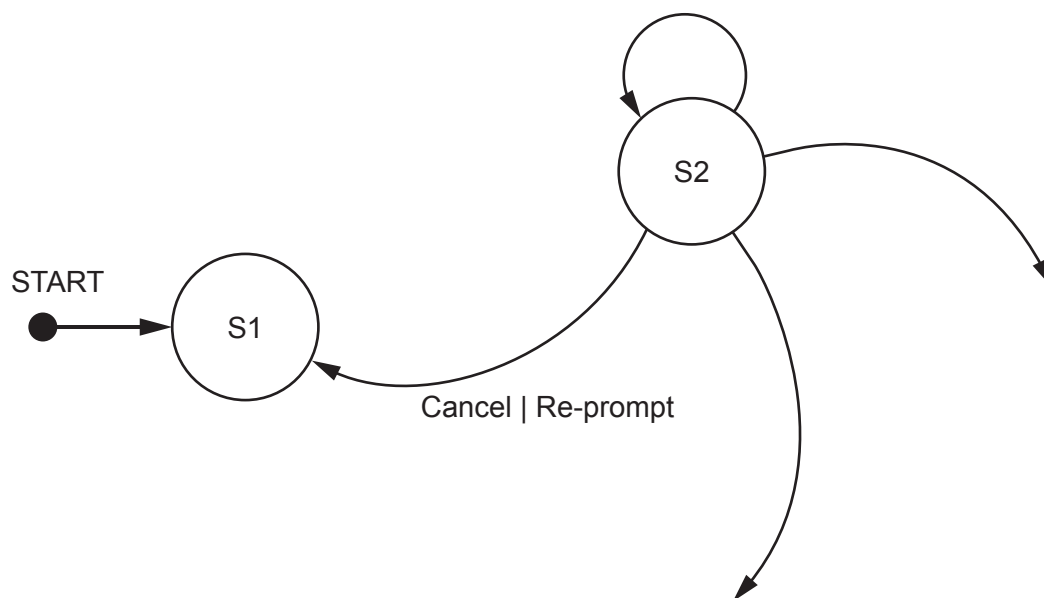
- 7 A computer system for a shop stores information about each customer. The items of information include name and address (both postal and email) together with payment details and order history. The system also stores the product categories they are interested in and how they would like to be contacted.
- (b) The program includes a module to validate a Personal Identification Number (PIN). This is used when customers pay for goods using a bank card.

A state-transition diagram has been produced for this module.

The table show the inputs, outputs and states for this part of the program:

Current state	Input	Output	Next state
S1	Input PIN		S2
S2	Re-input PIN	Display error	S2
S2	Cancel	Re-prompt	S1
S2	Valid PIN	Enable payment	S4
S2	Too many tries	Block Account	S3

Complete the state-transition diagram to represent the information given in the table.



[4]

58. M/J 23/P22/Q8/c

- 8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Valid item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique number for each item in the range "0001" to "5999" inclusive
SupplierCode	3 alphabetic characters	to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
<code>OnlyAlpha()</code> (already written)	- called with a parameter of type string - returns <code>TRUE</code> if the string contains only alphabetic characters, otherwise returns <code>FALSE</code>
<code>CheckInfo()</code>	- called with a parameter of type string representing a line of item information - checks to see whether the item information in the string is valid - returns <code>TRUE</code> if the item information is valid, otherwise returns <code>FALSE</code>

- (c) The program contains modules `SuppExists()` and `CheckSupplier()`. These have been written but contain errors. These modules are called from several places in the main program and testing of the main program (integration testing) has had to stop.

Identify a method that can be used to continue testing the main program before the errors in these modules have been corrected **and** describe how this would work.

Method

Description

59. M/J 23/P23/Q1/b

- 1 The following pseudocode represents part of the algorithm for a program.

Line numbers are for reference only.

```
10  DECLARE Sheet4 : ARRAY[1:2, 1:50] OF INTEGER
...
100  FOR PCount ← 0 TO 49
101      Sheet4[1, PCount] ← 0
102      Sheet4[2, PCount] ← 47
103  NEXT PCount
```

- (b) The pseudocode contains two errors. One error is that variable `PCount` has not been declared.

Identify the **other** error **and** state the line number where it occurs.

Error

Line number

[2]

60. M/J 23/P23/Q5/a,b

- 5 A program is designed, coded and compiled without errors. The compiled code is sent for testing.

- (a) The program will be tested using the walkthrough method.

Additional information will be needed before this method can be used.

Identify this additional information **and** explain why it is needed.

Additional information

Explanation

[3]

- (b) Testing is completed and the program is made available to users.

Some time later, changes are made to the program to improve the speed of response.

State the type of maintenance that has been applied to the program.

[1]

61. M/J 23/P23/Q7/b

- 7 A school has a library system which allows students to borrow books for a length of time. Information relating to students and books is stored in text files. Student information includes name, home address, email address, date of birth, tutor and subject choices. Book information includes author, title, subject category, library location and the date that the book was borrowed.

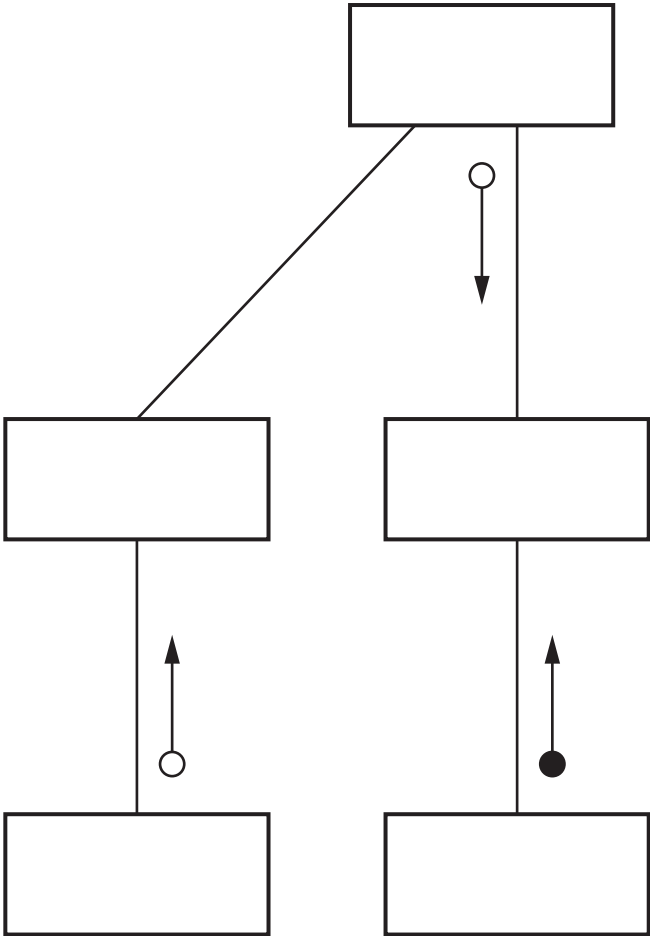
A program helps the staff to manage the borrowing of books.

(b) Part of the library program contains program modules with headers as follows:

Pseudocode module header
PROCEDURE Module-X()
PROCEDURE Module-Y(BYREF RA : INTEGER, SA : REAL)
PROCEDURE Overlay()
FUNCTION Replace(RA : INTEGER, RB : BOOLEAN) RETURNS BOOLEAN
FUNCTION Reset(TA : STRING) RETURNS INTEGER

Module-X() and Module-Y() are both called from module Overlay().

Complete the structure chart.



62. M/J 23/P23/Q8/c

- 8 A computer shop assembles desktop computers, using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique number for each item in the range “0001” to “5999” inclusive
SupplierCode	3 alphabetic characters	code to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does not contain all possible values in the range.

The programmer has defined the first program module as follows:

Module	Description
<code>ChangeSupp()</code>	- called with two parameters <code>Code1</code> and <code>Code2</code> of type string that represent valid supplier codes - creates a new file <code>NewStock.txt</code> from the contents of the file <code>Stock.txt</code> where any reference to <code>Code1</code> is replaced by <code>Code2</code> - returns a count of the number of items that have had their supplier code changed

(b) A new module is required:

Module	Description
<code>Report_1()</code>	- takes a parameter of type string that represents a <code>SupplierCode</code> - searches the <code>Stock.txt</code> file for each line of item information that contains the given <code>SupplierCode</code> - produces a formatted report of items for the given <code>SupplierCode</code>, for example, for supplier DRG, the output could be: <pre>Report for Supplier: DRG Item Description 1234 USB Printer Cable 3m 1273 32GB USB Flash Drive 1350 Mouse Mat 320 x 240mm Number of items listed: 3</pre>

Write pseudocode for module `Report_1()`.

- (c) The format of the output from module `Report_1()` from **part (b)** is changed. The number of items listed is moved to the top of the report as shown in the example:

```
Report for Supplier: DRG
Number of items listed: 3

Item          Description

1234          USB Printer Cable 3m
1273          32GB USB Flash Drive
1350          Mouse Mat 320 x 240mm
```

- (i) Explain why this new layout would increase the complexity of the algorithm.

[2]

- (ii) The algorithm will be modified to produce the report in the new format. The modified algorithm will be implemented so that the file `Stock.txt` is only read once.

Describe the modified algorithm.

[3]

63. O/N 22/P21/Q6/b

- 6 The following pseudocode algorithm attempts to check whether a string is a valid email address.

```
FUNCTION IsValid(InString : STRING) RETURNS BOOLEAN
  DECLARE Index, Dots, Ats, Others : INTEGER
  DECLARE NextChar : CHAR
  DECLARE Valid : BOOLEAN

  Index ← 1
  Dots ← 0
  Ats ← 0
  Others ← 0
  Valid ← TRUE

  REPEAT
    NextChar ← MID(InString, Index, 1)
    CASE OF NextChar
      '.' : Dots ← Dots + 1
      '@' : Ats ← Ats + 1
           IF Ats > 1 THEN
             Valid ← FALSE
           ENDIF
      OTHERWISE : Others ← Others + 1
    ENDCASE

    IF Dots > 1 AND Ats = 0 THEN
      Valid ← FALSE
    ELSE
      Index ← Index + 1
    ENDIF
```

```

UNTIL Index > LENGTH(InString) OR Valid = FALSE

IF NOT (Dots >= 1 AND Ats = 1 AND Others > 8) THEN
    Valid ← FALSE
ENDIF

RETURN Valid

ENDFUNCTION

```

(b) (i) Complete the trace table by dry running the function when it is called as follows:

Result ← IsValid("Liz.123@big@net")

Index	NextChar	Dots	Ats	Others	Valid

[5]

(ii) State the value returned when IsValid() is called using the expression shown in part (b)(i).

[1]

64. O/N 22/P21/Q7/b

7 A simple arithmetic expression is stored as a string in the format:

<Value1><Operator><Value2>

An operator character is one of the following: '+' '-' '*' '/'

Example arithmetic expression strings:

"803+1904"

"34/7"

(b) A string representing an arithmetic expression could be in the correct format but be impossible to evaluate.

Give an example of a correctly formatted string and explain why evaluation would be impossible.

Example string

Explanation

[2]

65. O/N 22/P21/Q8/d

- 8 A teacher is designing a program to perform simple syntax checks on programs written by students. Student programs are submitted as text files, which are known as project files.

A project file may contain blank lines.

The teacher has defined the first program module as follows:

Module	Description
<code>CheckFile()</code>	• takes the name of an existing project file as a parameter of type string • returns <code>TRUE</code> if the file is valid (it contains at least 10 non-blank lines), otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `CheckFile()`.

[7]

Further modules are defined as follows:

Module	Description
<code>CheckLine()</code>	• takes a line from a project file as a parameter of type string • returns zero if the line is blank or contains no syntax error, otherwise returns an error number as an integer
<code>CountErrors()</code>	• takes two parameters: ◦ the name of a project file as a string ◦ the maximum number of errors as an integer • uses <code>CheckFile()</code> to test the project file. Outputs an error message and ends if the project file is not valid • calls <code>CheckLine()</code> for each line in the project file • counts the number of errors • outputs the number of errors or a warning message if the maximum number of errors is exceeded

- (d) Module `CheckLine()` includes a check for syntax errors.

Two examples of syntax error that **cannot** be detected from examining a **single** line are those involving selection and iteration.

Give **two other** examples.

1

2

[2]

66. O/N 22/P22/Q1/b

- 1 (a) A programmer is developing an algorithm to solve a problem. Part of the algorithm would be appropriate to implement as a subroutine (a procedure or a function).

(i) State **two** reasons why the programmer may decide to use a subroutine.

1

2

[2]

(ii) A procedure header is shown in pseudocode:

```
PROCEDURE MyProc (Count : INTEGER, Message : STRING)
```

Give the correct term for the identifiers `Count` and `Message` **and** explain their use.

Term

Use

[2]

- (b) The algorithm in **part (a)** is part of a program that will be sold to the public.
All the software errors that were identified during in-house testing have been corrected.

Identify **and** describe the additional test stage that may be carried out before the program is sold to the public.

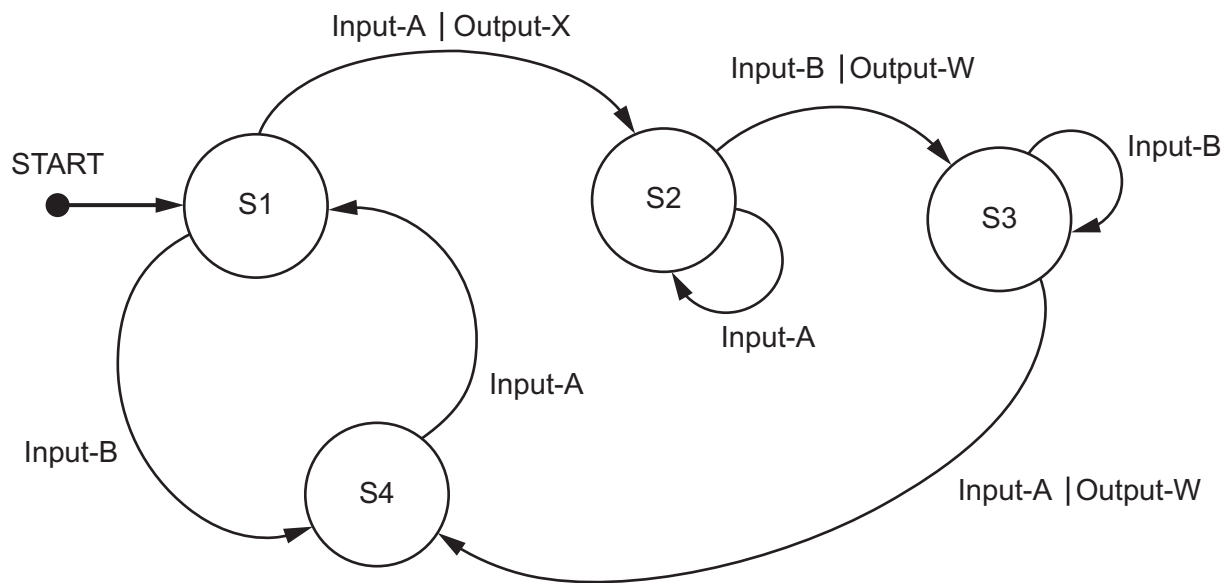
Test stage

Description

[4]

67. O/N 22/P22/Q2/b

(b) Examine the following state-transition diagram.



Complete the table to show the inputs, outputs and next states.

Input	Output	Next state
		S1
Input-A		
		S2
	Output-W	
	Output-W	

[4]

68. O/N 22/P22/Q6/b

- 6 (a) The factorial of an integer number is the product of all the integers from that number down to 1.

In general, the factorial of n is $n \times (n-1) \times \dots \times 2 \times 1$

For example, the factorial of 5 is $5 \times 4 \times 3 \times 2 \times 1 = 120$

In this question, n will be referred to as the `BaseNumber`.

A function `FindBaseNumber()` will:

- be called with a positive, non-zero integer value as a parameter

- return `BaseNumber` if the parameter value is the factorial of the `BaseNumber`
- return `-1` if the parameter value **is not** a factorial.

For example:

Parameter value	Value returned
120	5
12	-1
6	3
1	1

`FindBaseNumber(12)` will return `-1` because 12 is not a factorial.

You may use the rest of this page for rough working.

Write pseudocode for the function `FindBaseNumber()`.

[7]

- (b) A program is written to allow a user to input a sequence of values to be checked using the function `FindBaseNumber()`.

The user will input one value at a time. The variable used to store the user input has to be of type string because the user will input 'End' to end the program.

Valid input will be converted to an integer and passed to `FindBaseNumber()` and the return value will be output.

Complete the table by giving **four** invalid strings that may be used to test distinct aspects of the required validation. Give the reason for your choice in each case.

Input	Reason for choice

69. O/N 22/P23/Q1/b,c,d

1 A program is required for a shopping website.

(b) The designer considers the use of a development life cycle to split the development of the website into several stages.

(i) State **one** benefit of a development life cycle when developing the website.

[1]

(ii) Analysis is one stage of a development life cycle.

State **one** document that may be produced from the analysis stage of the website project.

[1]

(c) The program will be developed using the Rapid Application Development (RAD) life cycle.

(i) State **one** principle of this life cycle.

[1]

(ii) Give **two** benefits and **one** drawback of its use compared to the waterfall life cycle.

Benefit 1

Benefit 2

Drawback

[3]

(d) Adaptive maintenance needs to be carried out on the website program.

Give **two** reasons why adaptive maintenance may be required.

1

2

[2]

70. O/N 22/P23/Q5/d

- 5 (a) A text string contains three data items concatenated as shown:

<StockID><Description><Cost>

Item lengths are:

Item	Length
StockID	5
Description	32
Cost	the remainder of the string

A procedure `Unpack()` takes four parameters of type string. One parameter is the original text string. The other three parameters are used to represent the three data items shown in the table and are assigned values within the procedure. These values will be used by the calling program after the procedure ends.

- (d) The program that includes `Unpack()` is tested using the walkthrough method.

Describe this method **and** explain how it can be used to identify an error.

[3]

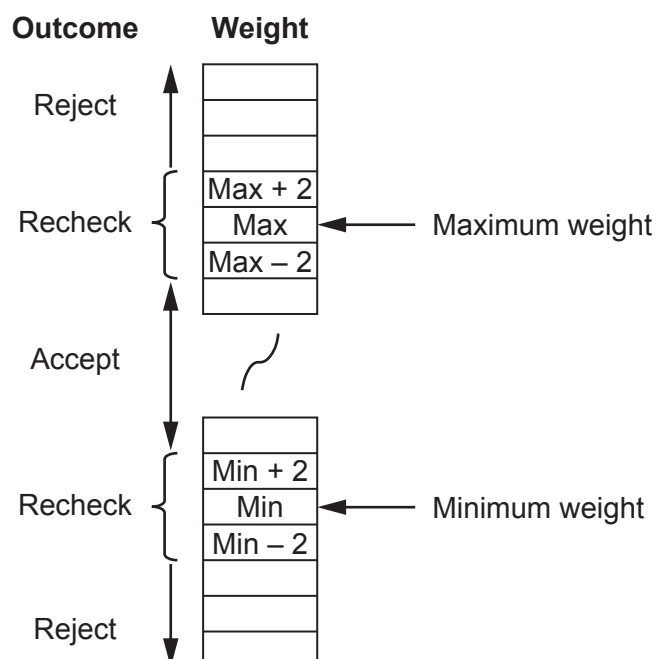
71. O/N 22/P23/Q6/a

- 6 Components are weighed during manufacture. Weights are measured to the nearest whole gram.

Components that weigh at least 3 grams more than the maximum weight, or at least 3 grams less than the minimum weight, are rejected.

A component is rechecked if it weighs within 2 grams of either the maximum or minimum weight.

The final outcome of weighing each component is shown below:



A function `Status()` will be called with three parameters. These are integers representing the weight of an individual component together with the minimum and maximum weights.

The value returned from the function will be as follows:

Outcome	Return value
Accept	'A'
Reject	'R'
Recheck	'C'

- (a) Complete the following test plan for **five** tests that could be performed on function `Status()`. The tests should address all possible outcomes.

Test number	Component weight	Min	Max	Expected return value
1				'A'
2				
3				
4				
5				

[5]

72. O/N 21/P21/Q1/a

- 1 Sylvia is testing a program that has been written by her colleague. Her colleague tells her that the program does not contain any syntax errors.

- (a) (i) State what her colleague means by “does not contain any syntax errors”.

[1]

- (ii) Identify **and** describe **one** other type of error that the program may contain.

Type of error

Description

[2]

73. O/N 21/P21/Q4/a,b

4 A program controls the heating system in a sports hall.

Part of the program involves reading a value from a sensor. The sensor produces a numeric value that represents the temperature. The value is an integer, which should be in the range 0 to 40 inclusive.

A program function has been written to validate the values from the sensor.

(a) A test plan is needed to test the function.

Complete the table. The first line has been completed for you.

You can assume that the sensor will generate only integer data values.

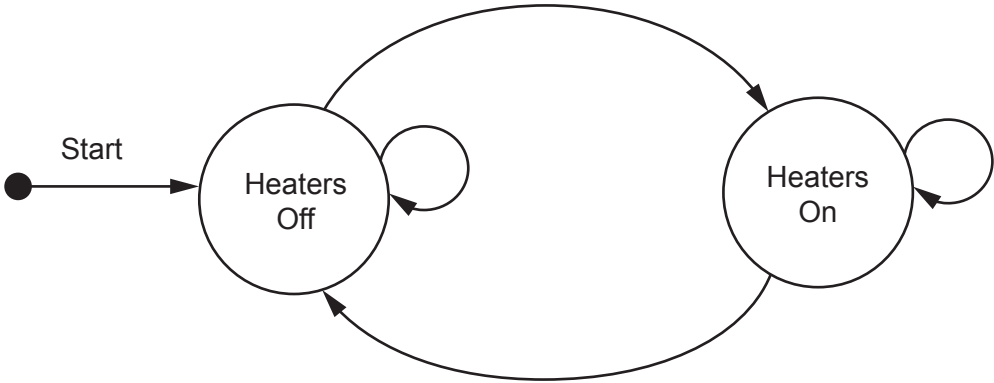
Test	Test data value	Explanation	Expected outcome
1	23	Normal data	Data is accepted
2			
3			
4			
5			

[4]

(b) A program module controls the heaters. This module operates as follows:

- If the temperature is below 10, switch the heaters on.
- If the temperature is above 20, switch the heaters off.

Complete the following state-transition diagram for the heating system:



[3]

74. O/N 21/P21/Q5/b

- 5 The following data items will be recorded each time a student successfully logs on to the school network:

Data item	Example data
Student ID	"CJL404"
Host ID	"Lib01"
Time and date	"08:30, June 01, 2021"

The Student ID is six characters long. The other two data items are of variable length.

A single string will be formed by concatenating the three data items. A separator character will need to be inserted between items two and three.

For example:

```
"CJL404Lib01<separator>08:30, June 01, 2021"
```

Each string represents one log entry.

A programmer decides to store the concatenated strings in a 1D array `LogArray` that contains 2000 elements. Unused array elements will contain an empty string.

- (b) The choice of data structure was made during one stage of the program development life cycle.

Identify this stage.

[1]

75. O/N 21/P22/Q2/b

- (b) A software company is working on a project to develop a website for a school.

The school principal has some ideas about the appearance of the website but is unclear about all the details of the solution. The principal would like to see an initial version of the website.

- (i) Identify a life cycle method that would be appropriate in this case.

Give a reason for your choice.

Life cycle method

Reason

[2]

- (ii) The website project has progressed to the design stage.

State **three** activities that will take place during the design stage of the program development life cycle.

1

2

3

[3]

76. O/N 21/P22/Q6/c(i)

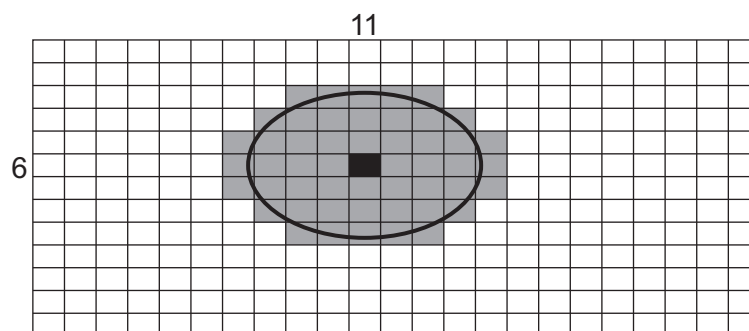
- 6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements will be set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch on the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point of the touch.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram shown is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has decided to use global values `CentreRow` and `CentreCol` as coordinate values for the centre point.

The programmer has started to define program modules as follows:

Module	Description
<code>FirstRowSet()</code>	• Searches for the first row that has an array element set to 1 • Returns the index of that row (1 is the first row) • Returns -1 if there are no elements set to 1
<code>LastRowSet()</code>	• Searches for the last row that has an array element set to 1 • Returns the index of that row • Returns -1 if there are no elements set to 1
<code>FirstColSet()</code>	• Searches for the first column that has an array element set to 1 • Returns the index of that column (1 is the first column) • Returns -1 if there are no elements set to 1
<code>LastColSet()</code>	• Searches for the last column that has an array element set to 1 • Returns the index of that column • Returns -1 if there are no elements set to 1

(c) The programmer decides to produce a **single** search module `FindSet()`, which will be able to perform each of the individual searches performed by the first four modules in the table.

- (i) Outline the changes needed to convert one of the existing modules into this single module.

[2]

77. O/N 21/P23/Q1/a

- 1 Sylvia is testing a program that has been written by her colleague. Her colleague tells her that the program does not contain any syntax errors.

(a) (i) State what her colleague means by “does not contain any syntax errors”.

[1]

(ii) Identify **and** describe **one** other type of error that the program may contain.

Type of error

Description

[2]

78. O/N 21/P23/Q4/a,b

4 A program controls the heating system in a sports hall.

Part of the program involves reading a value from a sensor. The sensor produces a numeric value that represents the temperature. The value is an integer, which should be in the range 0 to 40 inclusive.

A program function has been written to validate the values from the sensor.

(a) A test plan is needed to test the function.

Complete the table. The first line has been completed for you.

You can assume that the sensor will generate only integer data values.

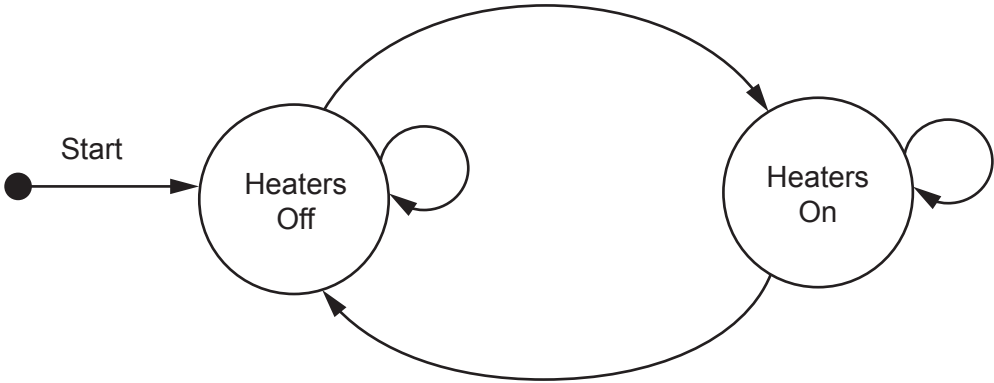
Test	Test data value	Explanation	Expected outcome
1	23	Normal data	Data is accepted
2			
3			
4			
5			

[4]

(b) A program module controls the heaters. This module operates as follows:

- If the temperature is below 10, switch the heaters on.
- If the temperature is above 20, switch the heaters off.

Complete the following state-transition diagram for the heating system:



[3]

79. O/N 21/P23/Q5/b

- 5 The following data items will be recorded each time a student successfully logs on to the school network:

Data item	Example data
Student ID	"CJL404"
Host ID	"Lib01"
Time and date	"08:30, June 01, 2021"

The Student ID is six characters long. The other two data items are of variable length.

A single string will be formed by concatenating the three data items. A separator character will need to be inserted between items two and three.

For example:

"CJL404Lib01<separator>08:30, June 01, 2021"

Each string represents one log entry.

A programmer decides to store the concatenated strings in a 1D array `LogArray` that contains 2000 elements. Unused array elements will contain an empty string.

- (b) The choice of data structure was made during one stage of the program development life cycle.

Identify this stage.

[1]

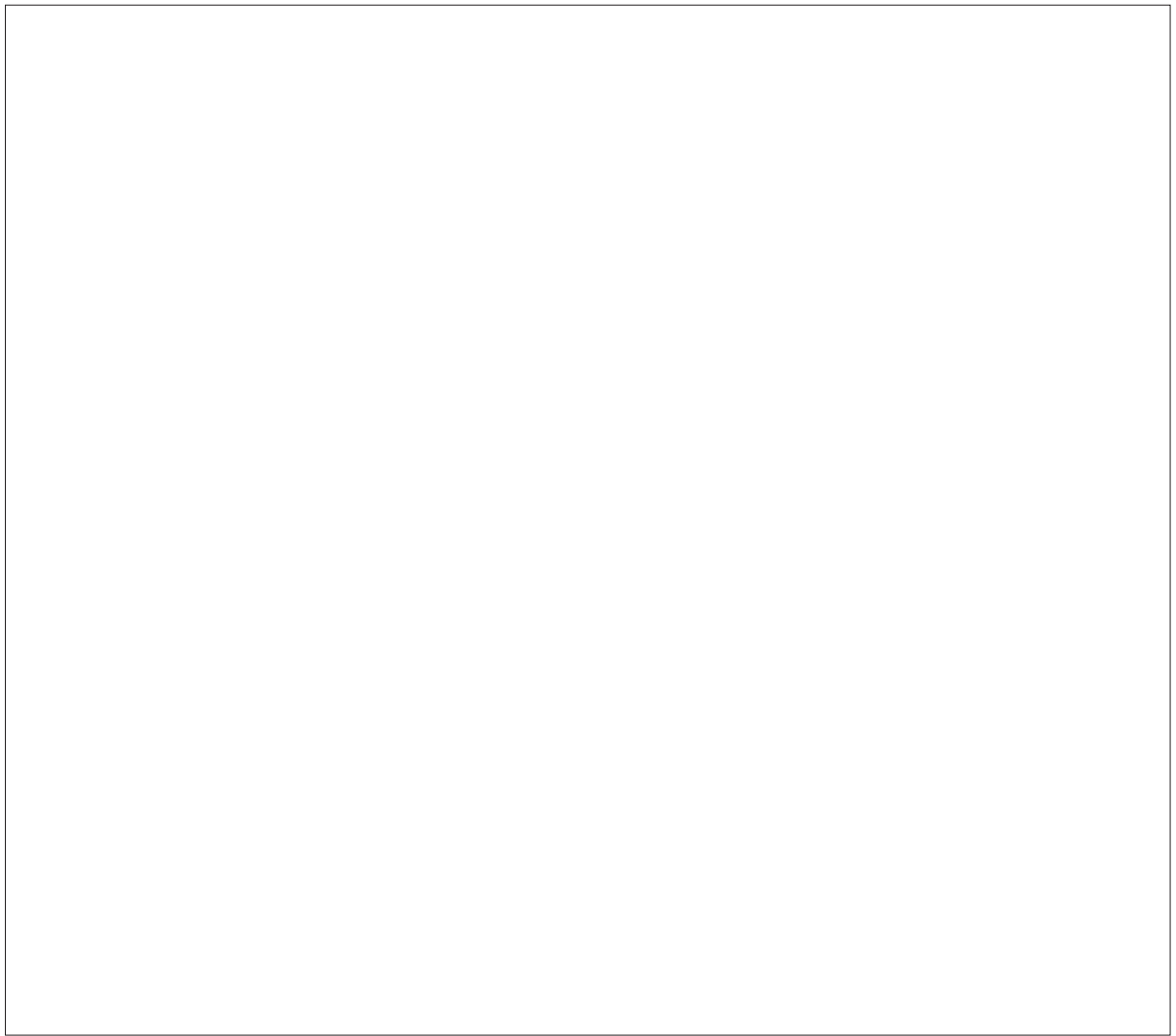
80. M/J 21/P21/Q2/a

- 2 (a) Four program modules form part of a program for a library.

A description of the relationship between the modules is summarised as follows:

Module name	Description
<code>UpdateLoan()</code>	• Calls either <code>LoanExtend()</code> or <code>LoanReturn()</code>
<code>LoanExtend()</code>	• Called with parameters <code>LoanID</code> and <code>BookID</code> • Calls <code>CheckReserve()</code> to see whether the book has been reserved for another library user • Returns <code>TRUE</code> if the loan has been extended, otherwise returns <code>FALSE</code>
<code>CheckReserve()</code>	• Called with <code>BookID</code> • Returns <code>TRUE</code> if the book has been reserved, otherwise returns <code>FALSE</code>
<code>LoanReturn()</code>	• Called with parameters <code>LoanID</code> and <code>BookID</code> • Returns a <code>REAL</code> (which is the value of the fine to be paid in the case of an overdue loan)

Draw a structure chart to show the relationship between the four modules and the parameters passed between them.



[5]

81. M/J 21/P21/Q4/a,c

- 4 Study the following pseudocode. Line numbers are for reference only.

```

10 FUNCTION Convert(Name : STRING) RETURNS STRING
11
12   DECLARE Flag: BOOLEAN
13   DECLARE Index : INTEGER
14   DECLARE ThisChar : CHAR
15   DECLARE NewName : STRING
16
17   CONSTANT SPACECHAR = ' '
18
19   Flag ← TRUE
20   Index ← 1
21   NewName ← ""           // formatted name string
22
23   WHILE Index <= LENGTH(Name)
24     ThisChar ← MID(Name, Index, 1)
25     IF Flag = TRUE THEN
26       NewName ← NewName & UCASE(ThisChar)
27       IF ThisChar <> SPACECHAR THEN
28         Flag ← FALSE
29       ENDIF
30     ELSE
31       NewName ← NewName & ThisChar
32     ENDIF
33     IF ThisChar = SPACECHAR THEN
34       Flag ← TRUE
35     ENDIF
36     Index ← Index + 1
37   ENDWHILE
38
39   RETURN NewName
40
41 ENDFUNCTION

```

(a) Complete the trace table below by dry running the function when it is called as follows:

Result ← Convert("VinVa∇∇Cup")

Note: The symbol '∇' has been used to represent a space character.
Use this symbol for any space characters in the trace table.

[1]

- (ii) Change 2 may be implemented by moving one line of the pseudocode.

Write the number of the line to be moved and state its new position.

Line number

New position

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LCASE(ThisChar : CHAR) RETURNS CHAR
returns the character value representing the lower case equivalent of ThisChar
Characters that are not upper case alphabetic are returned unchanged

Example: LCASE('W') returns 'w'

UCASE(ThisChar : CHAR) RETURNS CHAR
returns the character value representing the upper case equivalent of ThisChar
Characters that are not lower case alphabetic are returned unchanged

Example: UCASE('a') returns 'A'

TO_UPPER(ThisString : STRING) RETURNS STRING
returns a string formed by converting all characters of ThisString to upper case

Example: TO_UPPER("Error 803") returns "ERROR 803"

TO_LOWER(ThisString : STRING) RETURNS STRING
returns a string formed by converting all characters of ThisString to lower case

Example: TO_LOWER("JIM 803") returns "jim 803"

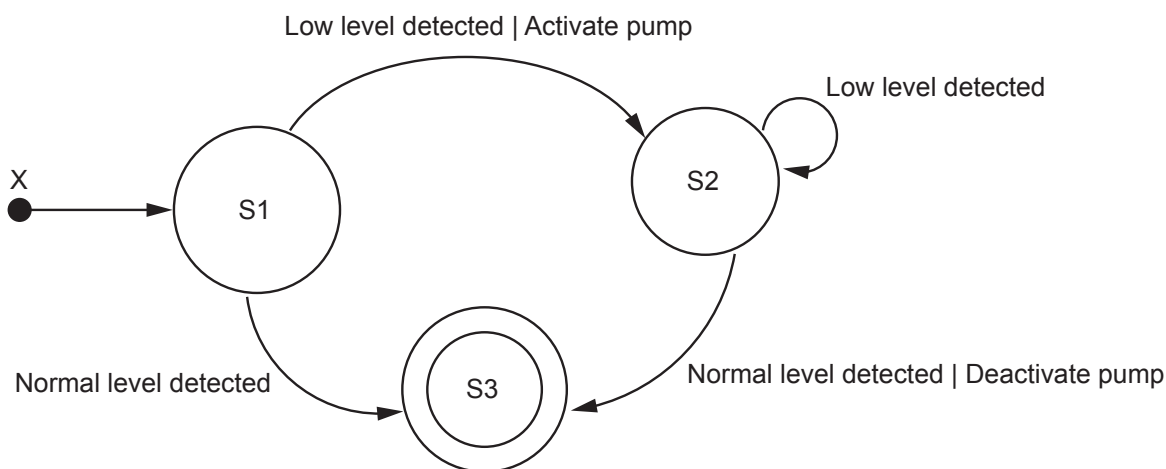
NUM_TO_STR(x : <data type>) RETURNS STRING
returns a string representation of a numeric value

Note: <data type> may be REAL or INTEGER

Example: NUM_TO_STR(87.5) returns "87.5"

82. M/J 21/P22/Q2/a

2 (a) Examine the following state-transition diagram.



(i) Complete the table with reference to the diagram.

Answer

The number of transitions that result in a different state	
The number of transitions with associated outputs	
The label that should replace 'X'	
The final or halting state	

[4]

(ii) The current state is S1. The following inputs occur.

1. Low level detected
2. Low level detected
3. Low level detected
4. Low level detected

Give the number of outputs and the current state.

Number of outputs

Current state

[2]

83. M/J 21/P22/Q7/a,b

7 A procedure, `FormatName()`:

- is called with a string containing words and spaces as its parameter. In this context, a word is any sequence of characters that does not contain a space character.
- creates a new formatted string from this string with the following requirements:
 1. Any leading spaces removed (spaces before the first word).
 2. Any trailing spaces removed (spaces after the last word).
 3. Any multiple spaces between words converted to a single space.
 4. All characters converted to lower case.

The `FormatName()` procedure has been written in a programming language and is to be tested using the black-box method.

- (a) Give a test string that could be used to show that all **four** formatting requirements have been applied correctly.

Use the symbol '∇' to represent a space character.

[3]

- (b) The `FormatName()` procedure should assign a value to the global variable `FString`.

There is a fault in the program, which means that the assignment does not always take place.

Explain **two** ways of exposing the fault.

[2]

84. M/J 21/P23/Q2/a

- 2 (a) Four program modules form part of a program for a library.

A description of the relationship between the modules is summarised as follows:

Module name	Description
<code>UpdateLoan()</code>	• Calls either <code>LoanExtend()</code> or <code>LoanReturn()</code>
<code>LoanExtend()</code>	• Called with parameters <code>LoanID</code> and <code>BookID</code> • Calls <code>CheckReserve()</code> to see whether the book has been reserved for another library user • Returns <code>TRUE</code> if the loan has been extended, otherwise returns <code>FALSE</code>
<code>CheckReserve()</code>	• Called with <code>BookID</code> • Returns <code>TRUE</code> if the book has been reserved, otherwise returns <code>FALSE</code>
<code>LoanReturn()</code>	• Called with parameters <code>LoanID</code> and <code>BookID</code> • Returns a <code>REAL</code> (which is the value of the fine to be paid in the case of an overdue loan)

Draw a structure chart to show the relationship between the four modules and the parameters passed between them.



[5]

85. M/J 21/P23/Q4/a,c

- 4 Study the following pseudocode. Line numbers are for reference only.

```

10 FUNCTION Convert(Name : STRING) RETURNS STRING
11
12   DECLARE Flag: BOOLEAN
13   DECLARE Index : INTEGER
14   DECLARE ThisChar : CHAR
15   DECLARE NewName : STRING
16
17   CONSTANT SPACECHAR = ' '
18
19   Flag ← TRUE
20   Index ← 1
21   NewName ← ""           // formatted name string
22
23   WHILE Index <= LENGTH(Name)
24     ThisChar ← MID(Name, Index, 1)
25     IF Flag = TRUE THEN
26       NewName ← NewName & UCASE(ThisChar)
27       IF ThisChar <> SPACECHAR THEN
28         Flag ← FALSE
29       ENDIF
30     ELSE
31       NewName ← NewName & ThisChar
32     ENDIF
33     IF ThisChar = SPACECHAR THEN
34       Flag ← TRUE
35     ENDIF
36     Index ← Index + 1
37   ENDWHILE
38
39   RETURN NewName
40
41 ENDFUNCTION

```

(a) Complete the trace table below by dry running the function when it is called as follows:

Result ← Convert("VinVa∇∇Cup")

Note: The symbol '∇' has been used to represent a space character.
Use this symbol for any space characters in the trace table.

[1]

- (ii) Change 2 may be implemented by moving one line of the pseudocode.

Write the number of the line to be moved and state its new position.

Line number

New position

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LCASE(ThisChar : CHAR) RETURNS CHAR
returns the character value representing the lower case equivalent of ThisChar
Characters that are not upper case alphabetic are returned unchanged

Example: LCASE('W') returns 'w'

UCASE(ThisChar : CHAR) RETURNS CHAR
returns the character value representing the upper case equivalent of ThisChar
Characters that are not lower case alphabetic are returned unchanged

Example: UCASE('a') returns 'A'

TO_UPPER(ThisString : STRING) RETURNS STRING
returns a string formed by converting all characters of ThisString to upper case

Example: TO_UPPER("Error 803") returns "ERROR 803"

`TO_LOWER(ThisString : STRING)` RETURNS STRING
returns a string formed by converting all characters of `ThisString` to lower case

Example: `TO_LOWER("JIM 803")` returns "jim 803"

`NUM_TO_STR(x : <data type>)` RETURNS STRING
returns a string representation of a numeric value

Note: <data type> may be REAL or INTEGER

Example: `NUM_TO_STR(87.5)` returns "87.5"

Answers Section

1. M/J 26/P21/Q3/a,b

3(a)	MP1 Investigate the current system /any example // investigate any other similar systems MP2 Discussion/interviews with managers MP3 Discussion/interviews with staff who operate the system MP4 Carry out a survey of employees/customers // carry out a focus group of employees/customers MP5 Observe the current system in use MP6 Produce the <u>requirements specification</u> MP7 Decide on the (program development) life cycle to be used Max 3	3
3(b)	Before any programming is started, the DESIGN stage for the proposed new system must be completed. For this large project, the tasks will be broken down into several MODULES // SUB-TASKS each will be coded by a different programming team. Once each team has completed its work, INTEGRATION // UNIT testing must be carried out. Before the new system is delivered to the business, ALPHA testing is carried out in-house. It is decided to install the software initially in only one of the call centres where BETA testing is carried out. The new software is finally rolled out to all five call centres. In the first week of its use, staff report that the software still requires extra features which were not originally agreed. The software is withdrawn until ADAPTIVE maintenance is completed. 1 mark per gap	6

2. M/J 26/P21/Q8/a,b,c

8(a)	<table><tr><th>LeftPointer</th><th>RightPointer</th><th>MidPointer</th></tr><tr><td>1</td><td>15</td><td>8</td></tr><tr><td></td><td>7</td><td>4</td></tr><tr><td>5</td><td></td><td>6</td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>	LeftPointer	RightPointer	MidPointer	1	15	8		7	4	5		6																4
	LeftPointer	RightPointer	MidPointer																										
	1	15	8																										
		7	4																										
	5		6																										
8(b)	Found 6	1																											
8(c)	6	1																											

3. M/J 26/P22/Q1/b,c,d,e,f

1(b)(i)	One mark per state Example answers: 1 Sensor activated (mode) // Intruder/movement detected (mode) // Alert Mode 2 Alarm sounding // Light flashing (mode) // Alarm activated 3 Error mode // Power loss detected (mode) // Battery too low mode 4 Tamper detected (mode) // Lockdown (mode) 5 Disarm Mode // Maintenance Mode // Password <u>Change</u> Mode // Service Mode Max 2	2	
1(b)(ii)	One mark per event Example answers: 1 Movement / pressure (detected) 2 Window opened / broken (detected) 3 Door open (detected) 4 Entering wrong password (repeatedly) 5 Entering password to enter / exit maintenance mode / silence alarm 6 Tampering / damaging alarm (system) 7 Time delay // Time passed after alarm triggered / before alarm activated 8 Battery charge too low 9 Pressing panic button	3	Do not accept repeating examples in stem. Reject Turning system on. Reject reference to creating password.

1(c)

Example Solution

rule being tested	example of test data	reason for invalid test data
rule 1	1A	password is too short
rule 1	PHRASE345678	Password too long
rule 2	ABCDEF	Does not contain a digit
rule 2	1234567	Does not contain a letter
rule 3	PASSWRD2	letters are repeated
rule 3	ABCDEF11	Digits / Numbers are repeated
rule 3	AAA111	Both digits and letters are repeated Accept Characters repeated

Mark as follows:

Rule 1 test:

1 mark for **one** example of test data for rule 1 **and** appropriate reason

Test data	A string of more than 8 characters that contains both letters and digits. The letters must be upper case Each character only occurs once
Reason	Password is too long

3

Do not accept once if lower-case letters or characters that are not letters or digits used.

Reject if test data breaks Rule 2 or/and Rule 3

1(c)	Rule 2 test (1 mark): 1 mark for two examples of test data that only breaks rule 2 and appropriate reasons. <table><tr><td>Test data</td><td>A string between 6 and 8 characters long that contains no digits</td></tr><tr><td>Reason</td><td>Does not contain a digit</td></tr><tr><td>Test data</td><td>A string between 6 and 8 characters long that contains no letters</td></tr><tr><td>Reason</td><td>Does not contain a letter</td></tr></table> Rule 3: 1 mark for two examples of test data for rule 3 and appropriate reasons <table><tr><td>Test data</td><td>A string between 6 and 8 characters long where only digits are repeated</td></tr><tr><td>Reason</td><td>Digits are repeated</td></tr><tr><td>Test data</td><td>A string between 6 and 8 characters long where both characters and digits are repeated</td></tr><tr><td>Reason</td><td>Both digits and letters are repeated</td></tr></table>	Test data	A string between 6 and 8 characters long that contains no digits	Reason	Does not contain a digit	Test data	A string between 6 and 8 characters long that contains no letters	Reason	Does not contain a letter	Test data	A string between 6 and 8 characters long where only digits are repeated	Reason	Digits are repeated	Test data	A string between 6 and 8 characters long where both characters and digits are repeated	Reason	Both digits and letters are repeated		Reject if letters repeated Reject if digits repeated Reject characters repeated Reject if letters repeated Reject if no letters
Test data	A string between 6 and 8 characters long that contains no digits																		
Reason	Does not contain a digit																		
Test data	A string between 6 and 8 characters long that contains no letters																		
Reason	Does not contain a letter																		
Test data	A string between 6 and 8 characters long where only digits are repeated																		
Reason	Digits are repeated																		
Test data	A string between 6 and 8 characters long where both characters and digits are repeated																		
Reason	Both digits and letters are repeated																		
1(d)	One mark per example Example answers: 1 Structure chart // Problem / Program broken into modules // Modular Diagram 2 Flowchart 3 Data Flow Diagram 4 Identifier / Variable table // Data structure / Record structure (designs) 5 File structure // Database Design 6 E-R Diagram 7 HCI Design // Report Design 8 Class Diagram // UML Diagram 9 Programming language to use Max 3	3	Reject 'Decomposition' by itself Reject Module Overview by itself																
1(e)	One mark per stage 1 Analysis 2 Coding // Implementation 3 Maintenance Max 2 marks	2	NE Development / Programming																
1(f)(i)	Integration	1																	

1(f)(ii)	One mark for the name, one for the description Method: Stub (testing) Description: 1 A simple / dummy module is written to replace the module (that is not working) 2 The simple / dummy module will return an expected value / will output a message (to show the module has been called)	3	Do not mark description if method incorrect
1(a)(i)	STRING	1	
1(a)(ii)	IF LENGTH(Pass) < 6 OR LENGTH(Pass) > 8 / >= 9 THEN Mark as follows: 1 Mark: Use of LENGTH function twice 1 Mark: Number 8 used in second condition	2	
1(a)(iii)	Mark as follows: 1 DECLARE LetterFlag, DigitFlag: <u>BOOLEAN</u> 1 mark for <u>BOOLEAN</u> 2 LetterFlag ← <u>FALSE</u> 1 mark for <u>FALSE</u> 3 FOR Index ← 1 TO <u>LENGTH</u>(Pass) 1 mark for <u>LENGTH</u> 4 Character ← <u>MID</u>(Pass, <u>Index</u>, <u>1</u>) 1 mark for <u>MID</u>, <u>Index</u> and 1 5 IF <u>IS_NUM</u>(Character) = TRUE THEN 1 mark for <u>IS_NUM</u> 6 DigitFlag ← <u>TRUE</u> ELSE LetterFlag ← <u>TRUE</u> 1 mark for both variables being set to TRUE 7 IF <u>NOT</u>(LetterFlag AND DigitFlag) THEN 1 mark for <u>NOT</u>	7	

4. M/J 26/P23/Q3/a,b,c

3(a)	The initial ANALYSIS stage is carried out by studying in detail the current system. A design for the proposed new system is produced. Following the design and coding stages, it is decided to install the completed new software initially in only two of the shops. BETA testing is carried out. The final stage of the project uses ACCEPTANCE testing to test all programs on the company's hardware. After successful testing, the company approves the new system. The company then installs the new software in all shops. In the first week of its use, unexpected bugs are reported. CORRECTIVE maintenance has to be carried out.	4	
3(b)	Team A MP1 Rapid Application Development /RAD // Iterative MP2 Large project can be broken down into smaller steps MP3 All stakeholders/users are involved (at all stages) MP4 There is constant discussion between the client (and Team A) MP5 Prioritises the client needs and end-user experience MP6 A <u>prototype</u> is available at an early stage MP7 More readily allows for requirement changes to be made MP8 Minimal planning is required MP9 Programs/modules can be developed in parallel Max 2 Team B MP10 Waterfall (method/model) MP11 Next stage does not start until the current stage is successfully completed MP12 More predictable for budget MP13 More predictable for timeline/completion MP14 All requirements are identified before coding MP15 Late changes cannot be accommodated Max 2	4	Reject 'early version'.
3(c)	MP1 Production of flowcharts // structure <u>charts</u> // STDs // Structured English // E-R diagrams MP2 Consider data storage (hardware / database / files) MP3 Design inputs screens // Design screen outputs // reports to be produced MP4 Select (Programming) language to be used MP5 Produce <u>pseudocode algorithms</u> / data structures / procedures/functions/modules are produced / Identifier table MP6 Produce a testing strategy statement // (detailed) test plan MP7 Consider – is there relevant code available from program libraries? Max 3	3	Reject 'program structure'.

5. M/J 26/P23/Q7/a,b

7(a)(i)	<table><tr><th rowspan="2">Num1</th><th rowspan="2">Index</th><th rowspan="2">i</th><th colspan="6">Data</th></tr><tr><th>1</th><th>2</th><th>3</th><th>4</th><th>5</th><th>6</th></tr><tr><td>12</td><td>1</td><td>2</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td>2</td><td>3</td><td></td><td>3</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td>3</td><td>4</td><td></td><td></td><td>4</td><td></td><td></td><td></td><td></td></tr><tr><td></td><td>4</td><td>5</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td>5</td><td>6</td><td></td><td></td><td></td><td>6</td><td></td><td></td><td></td></tr><tr><td></td><td>4</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>									Num1	Index	i	Data						1	2	3	4	5	6	12	1	2	2								2	3		3							3	4			4						4	5									5	6				6					4																													4	
	Num1	Index	i	Data																																																																																																						
				1	2	3	4	5	6																																																																																																	
	12	1	2	2																																																																																																						
		2	3		3																																																																																																					
		3	4			4																																																																																																				
		4	5																																																																																																							
		5	6				6																																																																																																			
		4																																																																																																								
Mark by column(s)																																																																																																										

7(a)(ii)	Factors / the divisors of Num1 / 12 // all the numbers which divide exactly into Num1	1	
----------	---	---	--

7(b)(i)	<table><tr><th>Num 1</th><th>Num2</th><th>Index</th><th>Match</th><th>1</th><th>2</th><th>3</th><th>4</th><th>5</th><th>6</th></tr><tr><td>20</td><td>8</td><td>4</td><td>FALSE</td><td>2</td><td>4</td><td>5</td><td>10</td><td></td><td></td></tr><tr><td></td><td></td><td>3</td><td>FALSE</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td>2</td><td>TRUE</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>										Num 1	Num2	Index	Match	1	2	3	4	5	6	20	8	4	FALSE	2	4	5	10					3	FALSE									2	TRUE																																																									3	
	Num 1	Num2	Index	Match	1	2	3	4	5	6																																																																																												
	20	8	4	FALSE	2	4	5	10																																																																																														
			3	FALSE																																																																																																		
			2	TRUE																																																																																																		
Max 2 for any additional incorrect entries																																																																																																						

7(b)(ii)	Highest common factor // Greatest common divisor of Num1 and Num2 // by good explanation i.e. 4 is the highest number that divides exactly into both 8 and 20	1	
----------	--	---	--

6. O/N 25/P21/Q1/b

1(b)

Test description	Test method
carried out as soon as a program module has been coded	alpha
carried out as program modules are being combined	integration
completed by the developers without referring to the code	black-box
completed by the customer	acceptance / beta

One mark per row (2 to 4)

3

7. O/N 25/P21/Q7/a,b

7(a)	One mark for each: 1 Label A1 X1 added on line from S1 to state labelled S2 2 States S3, S4 and S5 labelled 3 Line from S2 to S5 with event label A4 X4 4 Lines from S3 with correct event labels including 'loop' label 5 Lines from S4 with correct event labels including 'loop'	5
------	--	----------

7(b)	<pre> graph TD Setup[Setup] -- H1 --> Restart[Restart] Setup -- C1 --> Restart Setup -- B1 --> Modify[Modify] Setup -- T1 --> Final[Final] Restart --> Setup Modify --> Modify Modify -- R2 --> Update[Update] Update --> Modify Final --> Confirm[Confirm] Confirm --> Final </pre> One mark per point: 1 All boxes correctly labelled 2 Iteration arrow 3 Parameter to Restart 4 Return value from Modify and parameter to Final 5 BYREF parameter to Update	5
------	--	---

8. O/N 25/P21/Q8/c

8(c)	Award marks for reference to following design features: Change to existing Record: 1 Store date due back as a new data item / in the loan record 2 Store date of loan and the loan length category as new data items / in the loan record Max 1 New Record 3 New Records with Category (and Loan length field) Change to Program: 4 (Use the loan length category of a book to) calculate the date due date 5 Compare due date with today's date (to check if overdue / to calculate fine) 6 Map loan length category to length of loan / date due back Max 1 Max 2 marks	2
------	--	---

9. O/N 25/P22/Q5/a,b

5(a)	Activity	Name of life cycle stage	4
	a structure chart is produced	Design	
	a program is modified to allow it to run on new hardware	Maintenance	
	the programmer identifies the customer's requirements	Analysis	
	an Integrated Development Environment (IDE) provides context-sensitive help such as 'auto-complete'	Coding	
One mark per row			
5(b)	Acceptance testing		1

10. O/N 25/P22/Q7/a,b

7(a)	Example explanation: The arrow means that <code>Convert</code> repeatedly calls <code>Analyse</code> then <code>Update</code> and then <code>Sort</code> (in that order)¹⁰ MP1 Reference to 'repetition'... MP2 Naming all <u>four</u> modules correctly and the sequence is stated or implied	2
7(b)	MP1 <u>PROCEDURE Analyse (Count : INTEGER)</u> <u>FUNCTION Update (Key : STRING) RETURNS INTEGER</u> MP2 MP3 MP4 <u>PROCEDURE Sort (BYREF P2 : BOOLEAN)</u> Mark as follows: One mark per underlined part	4

11. O/N 25/P22/Q8/d

8(d)	Additional date(s) solution MP1 Store the return date of each loan // the start date <u>and</u> the loan period MP2 Algorithm will do a calculation involving the MP1 data item(s) OR The student email address solution MP3 Store the student <u>email address</u> MP4 In order to send the email // the student is alerted to return the book Max 2	2
------	---	----------

12. O/N 25/P23/Q5/a

5(a)

5

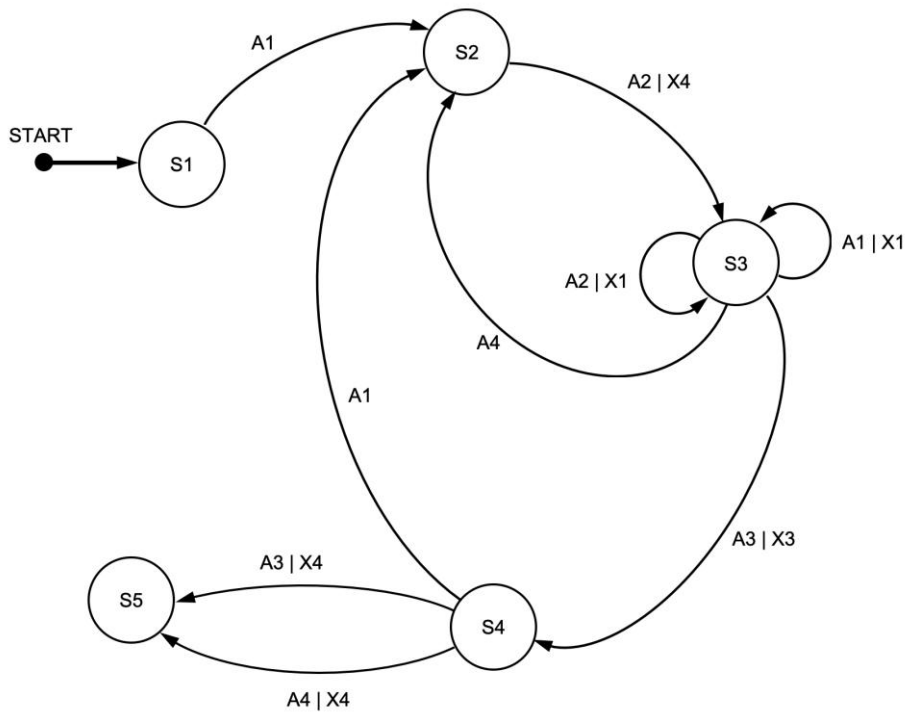
Index	CaseVar	Count	Num [1]	Num [2]	Num [3]	Num [4]
1		0	1	2	5	3
	1					
2		1	2			Zone 1
3						
	5					
		20				
4						
	3					
7		19				6
4						
	6					
		20				
1						
	2					
3		22	3			
4						
Zone 2	Zone 3	Zone 4	Zone 5			

Zones 1 to 4 : One mark for each set of values as shown
Zone 5: Num[4] set to 6 when Index is 7
and
Num[1] set to 3 when index is 3 for **second** time

13. O/N 25/P23/Q6/a

6(a)	One mark per point:	2
	1 Create a list of numbers that should generate all of the possible check digit values	
	2 and check that each / the / a generated value is as expected	

14. O/N 25/P23/Q7/a,b

7(a)	 One mark for each: 1 S2 to S5 labelled 2 Label A1 added to event from S1 to S2 3 Event lines from S2 to S3 and S3 to S2 correctly labelled 4 Events lines from S3 to S3 and event line from S3 to S4 all labelled correctly 5 Event lines from S4 to S5 correctly and event line from S4 to S2 labelled correctly Max 4 for any additional events/lines	5
------	---	----------

7(b)	<pre> graph TD Setup{Setup} --> Restart[Restart] Setup --> Confirm[Confirm] Restart --> Update[Update] Restart --> Modify[Modify] Update --> Restart Modify --> Restart Confirm --> Setup </pre> One mark for each: 1 All boxes correctly labelled and connected in correct hierarchy 2 BYREF parameter to Restart, parameter to Confirm and return 3 Diamond and loop symbol 4 Parameter to Update and return from Modify	4
------	--	---

15. M/J 25/P21/Q2/a,b,c

2(a)(i)	MP1 The two decision boxes are in the wrong order // The first decision symbol is wrong MP2 The first decision should check for values greater than or equal to 2000 Max 1	1
2(a)(ii)	Explaining how to correct the flowchart. MP1 Swap around the two decision boxes MP2 The first decision box should check for values greater than or equal to 2000 and the second decision box should check for values above 4000 Max 1	1
2(b)(i)	2000 / 4000 / 10 / 100	1
2(b)(ii)	MP1 The value cannot be changed // avoids being different in two places MP2 Makes the program easier to understand MP3 Avoids repeatedly writing the same value throughout the program MP4 A change to the value requires a change in only one statement Max 2	2

2(b)(iii)	MP1 (The code is) tried and tested so error free MP2 Provides functionality / code that the programmer may find difficult to code themselves Max 1	1
2(c)	MP1 Syntax (error) MP2 Rules of programming language // the language grammar was not followed MP3 Run-time (error) MP4 The program performs an illegal operation Mark as follows: One mark for naming each type of error and one mark for the description	4

16. M/J 25/P21/Q4/b

4(b)

6

I	Key	J	Chars [J]	Chars			
				[1]	[2]	[3]	[4]
2				'D'	'T'	'H'	'R'
	'T'	1	'D'				
3							
	'H'	2	'T'			'T'	
		1	'D'				
					'H'		
4							
	'R'	3	'T'				'T'
		2	'H'			('R')	
						'R'	
5							

MP1

MP2

MP3

MP4

MP5

MP6

Chars array final values

'D'	'H'	'R'	'T'
-----	-----	-----	-----

Mark as follows:

MP1, MP2, MP3, MP4, MP5

1 mark per enclosure

17. M/J 25/P21/Q6/a

6(a)

Type of test data	Test data value	Expected outcome
normal	36	data item is accepted
boundary/ extreme/ normal	0 / 1	data item is accepted
boundary/ extreme/ normal	59 / 60	data item is accepted
abnormal	>= 61	data item is rejected
normal	15	data item is accepted

4

Mark as follows:
1 mark for each row with (Type **and** Value **and** Outcome)

18. M/J 25/P21/Q7/b(ii)

7(b)(ii)	MP1 Check if the text file <code>loyalty.txt</code> exists / is empty MP2 If file does not exist it must be created (using write mode) MP3 If the file has to be created / is empty then set the first <code>CustomerID</code> to 100001 MP4 Write "100001,0" to <code>loyalty.txt</code> (using write mode) MP5 For all but the first customer (to be added to the empty file) use the existing code / module Max 4	4
----------	--	----------

19. M/J 25/P22/Q1/a,b

1(a)(i)	One mark for Choice of program development cycle depends on the approach/method required to produce the program Or One mark for a factor that would influence the choice of program development cycle, e.g. • Rapid development of program/prototypes required • Need for prototypes (at an early stage) • Complexity of problem • Skills / experience of development team / programmer(s) • Budget / Time / Resources available • Size of development team • Developer / programmer can return to earlier stages / any stage • Avoidance of repeating previous stages in development of program • How much involvement clients will have • Allows the (program) requirements to be changed during program development • (Program) requirements agreed at start of development of program Max 1	1
1(a)(ii)	Which programming language would best be suited to control the production line // Which programming language would best suit the problem being solved	1
1(b)(i)	Examples include: • Change to (production line) requirements • New technology / hardware available (to control production line) • Changes made to library modules used • Change in relevant legislation Max 3	3
1(b)(ii)	Perfective // Corrective	1

20. M/J 25/P22/Q2/b

2(b)(i)	One mark for each of set test values Hours worked between 1 and 40 inclusive Sales value ≤ 2000 Bonus Pay: 0 Hours worked above 40 Sales value ≤ 2000 Bonus Pay: 10 Hours worked between 1 and 40 inclusive Sales value > 2000 Bonus Pay: 50 Hours worked above 40 Sales value above 2000 Bonus Pay: 100 max 2	2
2(b)(ii)	One mark per point - Simple modules are written to replace each of the unfinished modules. - Each simple module will return an expected value / will output a message to show it has been called.	2
2(b)(iii)	One mark for naming type of error and one mark for corresponding description Type of error: Logic (error) Description: Where the program does not behave as expected / Does not give expected result / An error in the logic of the algorithm Or Type of error: Run-time (error) Description: The program performs an illegal operation Max 2	2

21. M/J 25/P22/Q5/a

5(a)

```
graph TD; Start((Start)) --> Standby((standby mode)); Standby -- "(turn) on" --> Detect((detect mode)); Detect -- "(turn) off" --> Standby; Detect -- "movement detected" --> Active((active mode)); Active -- "(turn) off" --> Standby; Active -- "20 second elapsed" --> SeqComp((sequence complete)); SeqComp -- "time saved" --> Detect;
```

Mark as follows:

1 Mark: The three new states drawn and labelled.

1 Mark: Two of the events drawn with labels connecting correct states and correct line direction

1 Mark: Four of the events drawn with labels connecting correct states and correct line direction.

1 Mark: All and only six events drawn with labels connecting correct states and correct line direction.

4

22. M/J 25/P22/Q6/a

6(a)(i)

Value	Start	Unused	New	Last	Current
1043	2	8	8	-1	2
				2	3
				3	1
				1	7
				7	4

Region 1

Region 2

Region 3

Region 4

Award 1 mark per region

4

6(a)(ii)

	Data		Pointer
1	1018	1	7
2	1007	2	3
3	1010	3	1
4	1056	4	6
5	1092	5	-1
6	1062	6	5
7	1034	7	8
8	1043	8	4
9	0	9	10
10	0	10	-1

Mark as follows:

1 mark for global array `Data` row 8 containing the value 1043

1 mark for global array `Pointer` row 7 containing the value 8 and row 8 containing the value 4

1 mark for all other rows in both arrays

3

23. M/J 25/P23/Q4/b

4(b)	MP1 Going through the program a line / statement at a time // doing a dry run // single-stepping (at run-time with an IDE) // Peer testing/review is carried out MP2 Creating a trace table MP3 Draw up test data // draw up list of inputs with expected output // boundary, normal, abnormal data is used MP4 Errors will be indicated when a variable / output gives an unexpected value MP5 Faults in the logic of the program can be detected // Errors may be indicated by an unexpected path through the program Max 3	3
------	---	----------

24. M/J 25/P23/Q6/b,c

6(b)	Sub_Part1A: <u>PROCEDURE Sub Part1A(R: INTEGER)</u> 1 Mark Sub_Part2A: <u>FUNCTION Sub Part2A(T : REAL) RETURNS BOOLEAN</u> 2 Marks Sub_Part3A: <u>PROCEDURE Sub Part3A(V: INTEGER, W: BOOLEAN, BYREF U : STRING)</u> 2 Marks	5						
6(c)	<table><tr><th>Symbol</th><th>Explanation</th></tr><tr><td></td><td>The module Main calls either Sub_A or Sub_B (which one is called is determined at run time)</td></tr><tr><td></td><td>The module Sub_A will repeatedly call Sub_Part1A followed by Sub_Part2A then by Sub_Part3A</td></tr></table> MP1 Shape1: reference to <u>selection</u> MP2 Shape2: reference to <u>repetition</u> // iteration MP3 Descption (×2) including correct use of all module names in both parts	Symbol	Explanation		The module Main calls either Sub_A or Sub_B (which one is called is determined at run time)		The module Sub_A will repeatedly call Sub_Part1A followed by Sub_Part2A then by Sub_Part3A	3
Symbol	Explanation							
	The module Main calls either Sub_A or Sub_B (which one is called is determined at run time)							
	The module Sub_A will repeatedly call Sub_Part1A followed by Sub_Part2A then by Sub_Part3A							

25. O/N 24/P21/Q1/a

1(a)	One mark per point: Error: The calculation of TaxPayable is incorrect Correction: TaxPayable ← (ItemCost * TaxRate) / 100	2
------	---	----------

26. O/N 24/P21/Q3/c

3(c)	Two loop solution One mark per point: 1 Loop until given line number (-1) / <code><parameter></code> (-1) (lines have been read) 2 ... if end of file is reached then return FALSE 3 Loop for three lines // Read three lines // Repeat of 4 and 5 for three lines 4 Read a line and output it 5 ... if end of file is reached then return FALSE 6 After outputting the (required) lines return TRUE 7 Ordering of lines no longer needed Max 4 marks Alternative solution One loop solution mark scheme that reads to given line number + 2 One mark per point 1 Loop until given line number + 2 / <code><parameter></code> + 2 (lines have been read) 2 OR end of file is reached 3 Return FALSE if EOF reached 4 Read a line from the file in the loop 5 Continue to order the last three lines read in the loop // Steps 4 to 6 stay the same 6 (If FALSE has not been returned) output the required three lines (in the correct order) 7 ... and Return TRUE Max 4 marks	4
3(a)	One mark per emboldened part: 1 Open the file (Result.txt) in read mode 2 Loop until EOF(Result.txt) // EOF // end of file 3 Read a / the / next line (from the file) and store in <code>ThisLine</code> 4 Assign <code>LineY</code> to <code>LineX</code> 5 Assign <code>LineZ</code> to <code>LineY</code> 6 Assign <code>ThisLine</code> to <code>LineZ</code> 7 After the loop, close the file (Result.txt) 8 Output <code>LineX</code>, <code>LineY</code>, <code>LineZ</code>	4

27. O/N 24/P21/Q4/a

4(a)	One mark per point: Type: Run-time error Cause: A divide by zero operation is attempted // Attempt to convert a non-numeric string to a number	2
------	---	----------

28. O/N 24/P21/Q5/a,b

5(a)	<table><tr><th>Activity</th><th>Name of life cycle stage</th></tr><tr><td>The walkthrough method is used.</td><td>Testing</td></tr><tr><td>An algorithm is implemented in a programming language.</td><td>Coding</td></tr><tr><td>The client is interviewed about problems with the current system.</td><td>Analysis</td></tr><tr><td>The program is modified to run on new hardware.</td><td>Maintenance</td></tr><tr><td>Records and file structures are defined.</td><td>Design</td></tr></table> One mark per row	Activity	Name of life cycle stage	The walkthrough method is used.	Testing	An algorithm is implemented in a programming language.	Coding	The client is interviewed about problems with the current system.	Analysis	The program is modified to run on new hardware.	Maintenance	Records and file structures are defined.	Design	5															
Activity	Name of life cycle stage																												
The walkthrough method is used.	Testing																												
An algorithm is implemented in a programming language.	Coding																												
The client is interviewed about problems with the current system.	Analysis																												
The program is modified to run on new hardware.	Maintenance																												
Records and file structures are defined.	Design																												
5(b)(i)	One mark per row <table><tr><th>Type of test data</th><th>Test data value</th><th>Expected result</th></tr><tr><td>Abnormal</td><td>12 (< 23)</td><td>FALSE</td></tr><tr><td>Abnormal / Boundary / Extreme</td><td>23</td><td>FALSE</td></tr><tr><td>Boundary / Extreme</td><td>24</td><td>TRUE</td></tr><tr><td>Boundary</td><td>25</td><td>TRUE</td></tr><tr><td>Boundary</td><td>36</td><td>TRUE</td></tr><tr><td>Boundary /Extreme</td><td>37</td><td>TRUE</td></tr><tr><td>Abnormal / Boundary / Extreme</td><td>38</td><td>FALSE</td></tr><tr><td>Abnormal</td><td>99 (> 38)</td><td>FALSE</td></tr></table> Max 4 marks	Type of test data	Test data value	Expected result	Abnormal	12 (< 23)	FALSE	Abnormal / Boundary / Extreme	23	FALSE	Boundary / Extreme	24	TRUE	Boundary	25	TRUE	Boundary	36	TRUE	Boundary /Extreme	37	TRUE	Abnormal / Boundary / Extreme	38	FALSE	Abnormal	99 (> 38)	FALSE	4
Type of test data	Test data value	Expected result																											
Abnormal	12 (< 23)	FALSE																											
Abnormal / Boundary / Extreme	23	FALSE																											
Boundary / Extreme	24	TRUE																											
Boundary	25	TRUE																											
Boundary	36	TRUE																											
Boundary /Extreme	37	TRUE																											
Abnormal / Boundary / Extreme	38	FALSE																											
Abnormal	99 (> 38)	FALSE																											
5(b)(ii)	Integration (testing)	1																											

29. O/N 24/P21/Q7/a,b

7(a)	One mark per point: 1 All module identified in correct hierarchy 2 Parameters to both Sync and Activate and return values 3 Parameters to both Reset and Enable 4 Parameters to Initialise 5 Decision diamond	5
7(b)	Explanation: Means that Sync repeatedly calls Reset and (then) Enable One mark for each point: MP1: reference to iteration MP2: naming all three modules correctly including correct sequence of calls	2

30. O/N 24/P22/Q1/a

1(a)	(Corrective) Maintenance	1
------	--------------------------	---

31. O/N 24/P22/Q5/a,b

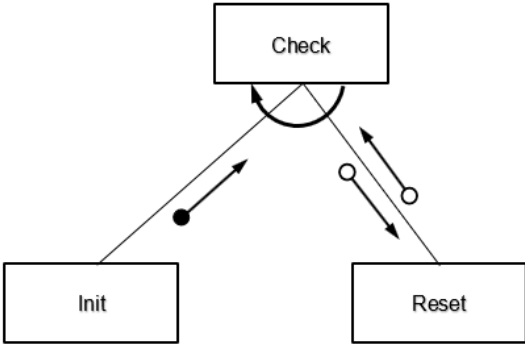
5(a)	MP1 <code>Count ← INT(100 / Number)</code> Number could be zero (giving a divide by zero) MP2 <code>Index ← Data[Number]</code> Potential error: Value <code>Number</code> could be outside the range of array indices MP3 <code>ReturnValue ← TO_UPPER(RIGHT(Label, Count))</code> Potential Error: Number to extract may be too big / negative / out of range for use in the <code>RIGHT</code> function // <code>Label</code> has insufficient characters MP4 <code>RETURN RetVal</code> Potential Error: There is no value to be returned // there is no variable named <code>RetVal</code> Mark as follows: 1 mark for each statement and description Max 3 marks	3
5(b)	MP1 Construct: A (pre/post) <u>conditional</u> loop MP2 Explanation: The terminating condition is never satisfied	2

32. O/N 24/P22/Q6/b

6(b)	Answers include: MP1 For each filling, create a list of <u>acceptable</u> / <u>incompatible</u> fillings/indexes MP2 When selecting the second filling, (as well as checking for an unused element) <u>check</u> that the filling / index is / is not on the list ALTERNATIVE: MP1 Create a list of 'good' combinations MP2 <u>Randomly select</u> from this list	2
------	---	----------

33. O/N 24/P22/Q7/c,d

7(c)	MP1 Data structures // data dictionary // identifier table(s) // validation rules MP2 Data-flow diagram // state-transition diagram MP3 User interface // Format for the email MP4 Testing method / Test plan / Test data / Trace tables MP5 Choice of email protocol to be used // Programming language to be used // Development environment MP6 Use of library routines // program to send the email Max 3 marks	3
------	---	----------

7(d)	 <pre> graph TD Init[Init] --> Check[Check] Check --> Reset[Reset] Reset --> Check Check --> Check </pre> MP1 Three boxes correctly labelled and correct hierarchy MP2 Parameter and return values MP3 Iteration arrow	3
------	---	----------

34. O/N 24/P23/Q5/a,b

5(a)	<table><tr><th>Activity</th><th>Name of life cycle stage</th></tr><tr><td>A compiler is be used.</td><td>Coding</td></tr><tr><td>A program that has been released for general use is modified</td><td>Maintenance</td></tr><tr><td>The dry run method is used.</td><td>Testing</td></tr><tr><td>The program structure is specified.</td><td>Design</td></tr></table> One mark per row	Activity	Name of life cycle stage	A compiler is be used.	Coding	A program that has been released for general use is modified	Maintenance	The dry run method is used.	Testing	The program structure is specified.	Design	4
Activity	Name of life cycle stage											
A compiler is be used.	Coding											
A program that has been released for general use is modified	Maintenance											
The dry run method is used.	Testing											
The program structure is specified.	Design											
5(b)	One mark for method – two marks for the description MP1 Stub testing MP2 The modules <u>Test A()</u> and <u>Test B()</u> are replaced by dummy modules MP3 ... which return a known result // An output statement is displayed when called (to check it works) // gives expected output	3										

35. O/N 24/P23/Q7/a

7(a)	One mark per answer: MP1 Analysis MP2 Named document, examples include: • A <u>requirement specification</u> • Definition of System Objectives • List of problems with existing system • Survey results • Feasibility study • Interview notes Max 2 marks	2
------	---	---

36. M/J 24/P21/Q5/a,b

5(a)	<table><tr><th>Count</th><th>C</th><th>L</th><th>Index</th><th>Result</th><th>Data[1]</th><th>Data[2]</th><th>Data[3]</th></tr><tr><td></td><td></td><td>1</td><td></td><td>"*****"</td><td>"aaaaaaa"</td><td>"bbbbbbb"</td><td>"ccccccc"</td></tr><tr><td>1</td><td>'X'</td><td>3</td><td>1</td><td>"AAAAA A"</td><td>"AAA"</td><td></td><td></td></tr><tr><td>3</td><td>'Y'</td><td>2</td><td>2</td><td>"bbbbb b"</td><td></td><td>"bb"</td><td></td></tr><tr><td>5</td><td>'W'</td><td>4</td><td>3</td><td></td><td></td><td></td><td>"bbbb"</td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table> One mark per zone.	Count	C	L	Index	Result	Data[1]	Data[2]	Data[3]			1		"*****"	"aaaaaaa"	"bbbbbbb"	"ccccccc"	1	'X'	3	1	"AAAAA A"	"AAA"			3	'Y'	2	2	"bbbbb b"		"bb"		5	'W'	4	3				"bbbb"																																																	6
Count	C	L	Index	Result	Data[1]	Data[2]	Data[3]																																																																																			
		1		"*****"	"aaaaaaa"	"bbbbbbb"	"ccccccc"																																																																																			
1	'X'	3	1	"AAAAA A"	"AAA"																																																																																					
3	'Y'	2	2	"bbbbb b"		"bb"																																																																																				
5	'W'	4	3				"bbbb"																																																																																			
5(b)(i)	<u>OTHERWISE : CALL Error(C)</u>	1																																																																																								
5(b)(ii)	After the 'z' clause in the CASE construct // before the ENDCASE	1																																																																																								

37. M/J 24/P21/Q6/b

6(b)	1 mark for statement of problem: Problem: - The function will return an incorrect value // the test will fail 1 mark for solution: Solution: - Round the calculated values (to a known number of decimal places) - Define a threshold below which any difference can be ignored	2
6(a)	<pre> FUNCTION IsRA(x1, y1, x2, y2, x3, y3 : INTEGER) RETURNS BOOLEAN DECLARE Len1, Len2, Len3 : INTEGER Len1 ← (x1 - x2) ^ 2 + (y1 - y2) ^ 2 Len2 ← (x1 - x3) ^ 2 + (y1 - y3) ^ 2 Len3 ← (x2 - x3) ^ 2 + (y2 - y3) ^ 2 IF (Len1 = Len2 + Len3) OR___ (Len2 = Len1 + Len3) OR___ (Len3 = Len1 + Len2) THEN RETURN TRUE ELSE RETURN FALSE ENDIF ENDFUNCTION </pre> Mark as follows: - Calculate the square of the length of at least one side - Calculation of all three lengths squared correctly One correct comparison of square of three lengths All three comparisons... - combined using logical operators / nested IF // completely correct selection - Return result correctly in both cases	6

38. M/J 24/P21/Q7/b

7(b)(i)	Means that Update calls (one of) either Sub-A, Sub-B or Sub-C One mark for each point: - reference to selection / decision / if - naming all four modules correctly	2
7(b)(ii)	<pre> PROCEDURE Sub-A (Name : STRING, BYREF P2 : BOOLEAN) FUNCTION Sub-B (P1 : REAL) RETURNS REAL </pre> One mark per underlined part in each case	4

39. M/J 24/P22/Q5/a(i),b

5(a)(i)	One mark per error: Syntax: 1. <code>NEXT Index</code> (should be <code>ENDWHILE</code>) 2. <code>'&'</code> used to concatenate an integer (in <code>OUTPUT</code> statement) Other: 3. <code>Accesses element outside range</code> // <code>Accesses element 0</code>	3
5(b)	Run-time	1

40. M/J 24/P22/Q7/b

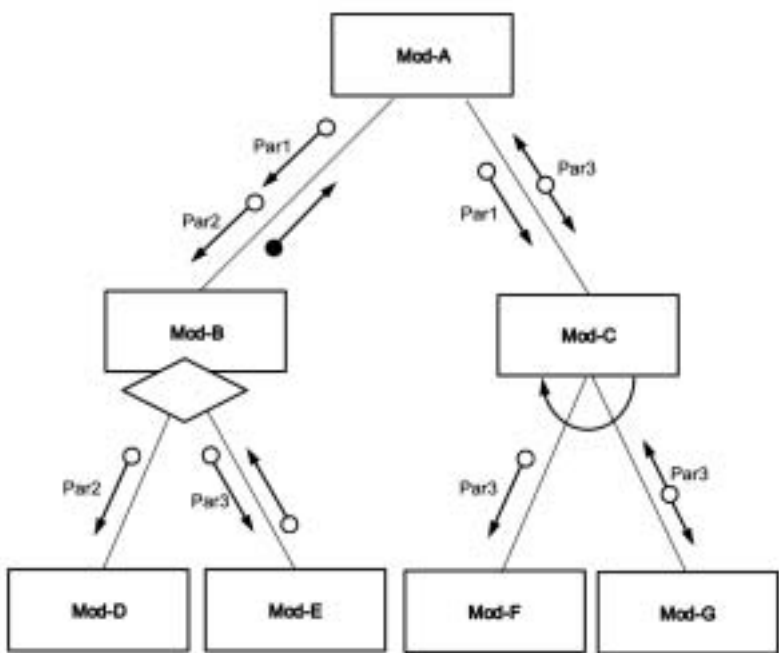
7(b)(i)	<table border="1"> <thead> <tr> <th>Input</th><th>Output</th><th>Next state</th></tr> </thead> <tbody> <tr> <td></td><td></td><td>S1</td></tr> <tr> <td>Input-A</td><td>none</td><td>S3</td></tr> <tr> <td>Input-A</td><td>Output-W</td><td>S3</td></tr> <tr> <td>Input-B</td><td>none</td><td>S2</td></tr> <tr> <td>Input-B</td><td>none</td><td>S5</td></tr> <tr> <td>Input-A</td><td>none</td><td>S2</td></tr> <tr> <td>Input-A</td><td>Output-X</td><td>S4</td></tr> </tbody> </table> One mark per row 3 to 7	Input	Output	Next state			S1	Input-A	none	S3	Input-A	Output-W	S3	Input-B	none	S2	Input-B	none	S5	Input-A	none	S2	Input-A	Output-X	S4	5
Input	Output	Next state																								
		S1																								
Input-A	none	S3																								
Input-A	Output-W	S3																								
Input-B	none	S2																								
Input-B	none	S5																								
Input-A	none	S2																								
Input-A	Output-X	S4																								
7(b)(ii)	Input-B, Input-A	1																								

41. M/J 24/P23/Q6/b(ii)

6(b)(ii)	The progress display will more accurately show the progress of the task	1
----------	---	----------

42. M/J 24/P23/Q7/a(i),b

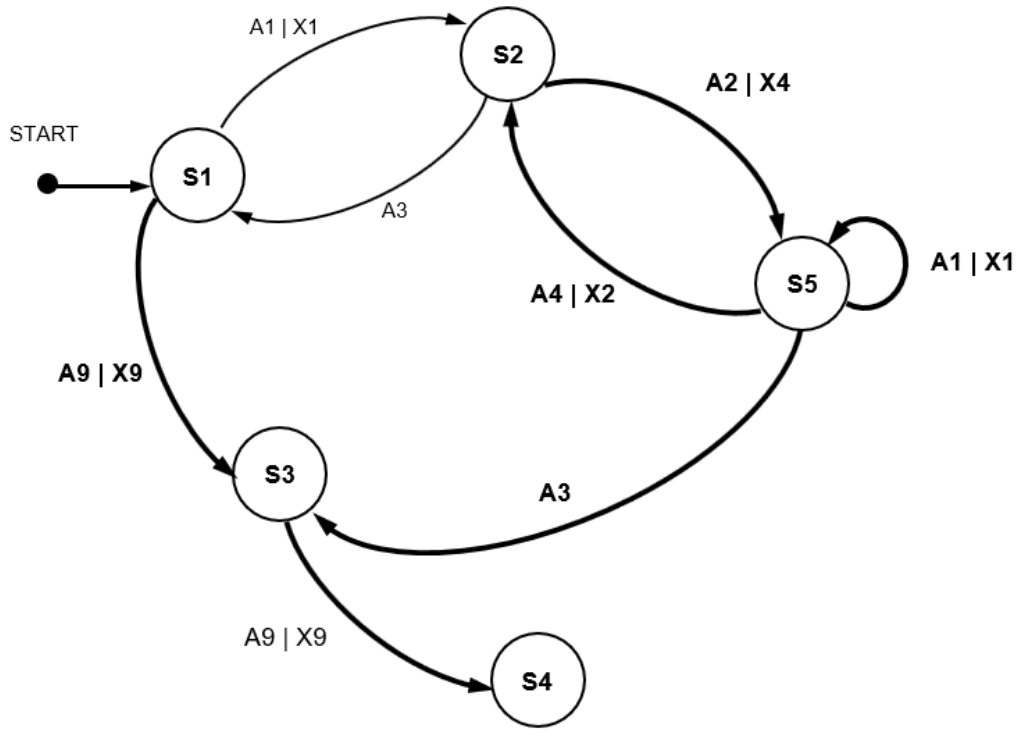
7(a)(i)	Mod-B() and Mod-E()	1
---------	---------------------	----------

7(b)	 One mark per bullet: 1 All modules correctly labelled and interconnected 2 Parameters between Mod-A and Mod-B and return value from Mod-B 3 Parameters between Mod-A and Mod-C 4 Diamond applied to Mod-B only 5 Iteration arrow applied to Mod-C only 6 All parameters at lower level and return value	6
------	--	---

43. O/N 23/P21/Q5/a,b

5(a)	MP1 Num > Min should be Num < Min MP2 Count & 1 should be Count + 1 MP3 NextInput ← "END" should be NextInput = "END"	3
5(b)(i)	MP1 If all the numeric input values are greater than 999 // If there are no numeric values in the sequence MP2 then the minimum will be given as <u>999</u> (and not one of the input values)	2
5(b)(ii)	Many possible correct answers, for example: MP1 Mixture non-numeric and numeric with 3 or 4 values - with all numerics greater than 999 Examples: 1325, DOG, 7868, 7615 // SNAKE, 3478, SPIDER MP2 Final value: END	2

44. O/N 23/P21/Q7

7	 MP1 S2 labelled MP2 S3, S4 and S5 added MP3 Line from S1 to S3 MP4 All three lines between S2 and S5 (including S5 to S5) MP5 Line from S5 to S3 AND from S3 to S4	5
---	--	---

45. O/N 23/P21/Q8/c

8(c)(i)	MP1 A message cannot contain a zero-length data field MP2 ... so a blank line cannot be sent // there is no way to send a blank line	2
8(c)(ii)	MP1 Append a (special) character to the start of the message text MP2 interpret the new field data as a blank line ALTERNATIVE MP1 Change the message protocol and use an additional field to act as an indicator MP2 Interpret the new field data	2

46. O/N 23/P22/Q4/b

4(b)	One mark per set, including stated purpose. Max 3 marks Example answers: 1 data set with (only) odd values, terminated with 99 2 data set with (only) even values, terminated with 99 3 data sets with same number of odd and even values, terminated with 99 4 data sets with all even / all odd with just one odd/even value, terminated with 99 5 data sets with no values just final 99 6 data sets without (terminating) 99 // missing or incorrectly placed 99	3
------	---	---

47. O/N 23/P22/Q5

5

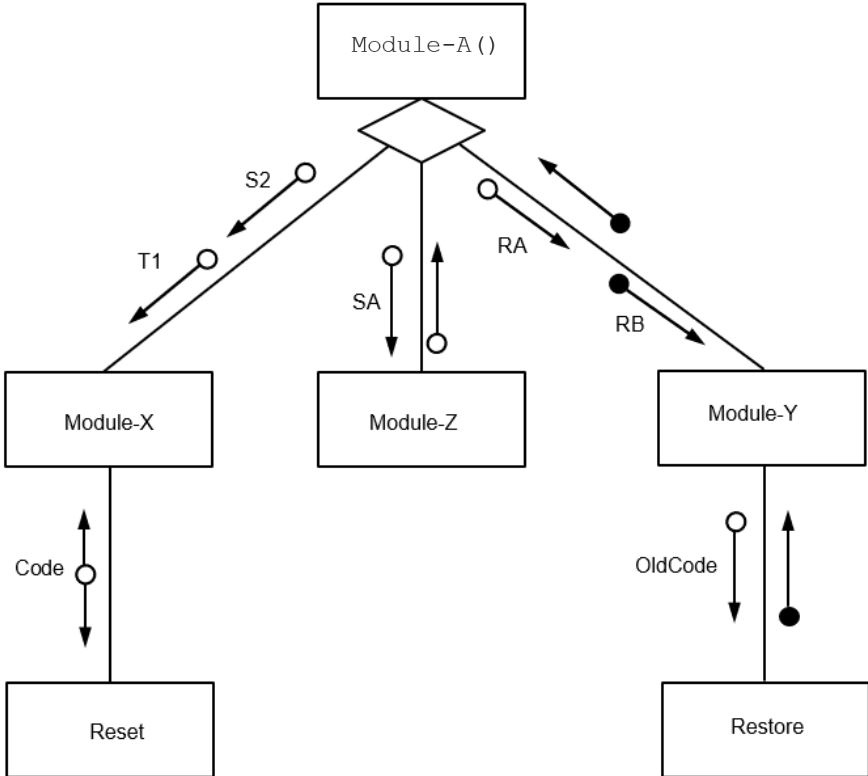
Index	Value	Count	Mix[1]	Mix[2]	Mix[3]	Mix[4]
3		0	1	3	4	2
	4				3	
4		1				
	2					1
2		2				
	3			2		
3		3				
	3				2	
3		4				
	2				1	
2		5				
						10

6

One mark for:

- Initialisation row
- Each iteration of Count 1 to 4 with correct Index, Count and array Mix as shown
- Final iteration, Count = 5, correct Index, Count and array Mix as shown plus Mix[4] assignment

48. O/N 23/P22/Q7/a,b

7(a)	 One mark per point: 1 Module names 2 Parameters with labels to Module-X and between Module-X and Reset 3 Parameter (with label) to Module-Z and return from Module-Z 4 Parameters with labels to Module-Y and Restore and return values from Module-Y	4
7(b)	Means that Module-A calls either one of Module-X, Module-Y or Module-Z (which one is called is decided at runtime). One mark for reference to selection One mark for naming all four modules correctly	2

49. O/N 23/P23/Q1/d

1(d)	MP1 Type: Adaptive MP2 Reason: The (user) requirement(s) changes // to accommodate legislative changes	2
------	--	---

50. O/N 23/P23/Q5

5

Index	Value	Total	Mix[1]	Mix[2]	Mix[3]	Mix[4]
2		0	4	2	3	5
	2	2				
				5		
3						
	3	5				
					8	
4						
	5	10				
						9
2						
	5	15				
				13		
2						
	13	28				
				21		
2						
			56			

6

MP1 Row 1 (initialisation)

Each iteration (1 – 5):

MP2 1 – Total 2

MP3 2 – Total 5

MP4 3 – Total 10

MP5 4 – Total 15

MP6 5 – Total 28 **and** final Mix[1] = 56

51. O/N 23/P23/Q7/a

7(a)	MP1 iteration / looping MP2 naming all four modules correctly in the correct sequence //_e.g. Module-A repeatedly calls Sub-Y1, then SubY2 then Sub-9	2
------	---	---

52. O/N 23/P23/Q8/b,d

8(b)	Max 3 marks Problem: • If the file being sent contains a line of the string "****" • then the file being written by <code>ReceiveFile()</code> will end at this point // subsequent file lines will be lost Solution: • Read the file (at the sending end) to find the number of lines it contains • Send an initial message which defines the number of lines in the file ALTERNATIVE SOLUTION: • (Transmitter program) chooses a different terminator string / character that doesn't occur in the file • Transmitter program sends the terminator string / character before first line of file / before the transfer begins	3
8(d)	Note: Max 3 marks (from either limitation or modification list) Limitation: 1 <code>GetData()</code> does not return a value until a message has been received 2 So once a message has been sent the user has to wait for a reply // chat is half-duplex Modification: 3 If no response allow the receiver to exit chat at any time ... 4 <code>GetData()</code> should immediately return a suitable message // set a time limit 5 ... which <code>Chat()</code> can detect and respond by allowing the conversation to continue	3

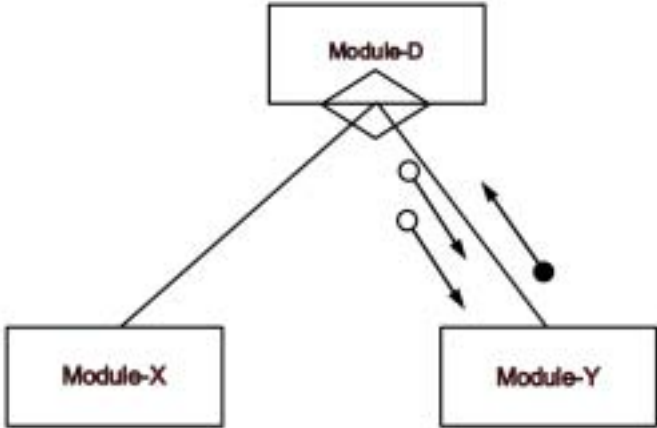
8(c)	<pre> PROCEDURE Chat(Destination : STRING, Port : INTEGER) DECLARE Data : STRING DECLARE Finished : BOOLEAN CONSTANT Terminator = "Bye" CONSTANT STX = CHR(2) CONSTANT ETX = CHR(3) Finished ← FALSE REPEAT Data ← GetData() OUTPUT Data IF Data = Terminator THEN Finished ← TRUE ENDIF IF NOT Finished THEN //about to reply INPUT Data Transmit(STX & Destination & MyID & Data & ETX, Port) IF Data = Terminator THEN Finished ← TRUE ENDIF ENDIF UNTIL Finished = TRUE ENDPROCEDURE </pre> Conditional loop MP1 Conditional loop MP2 Test for terminator in both cases MP3 Use <code>GetData()</code> to get the data from the message MP4 <code>OUTPUT</code> the data in a loop MP5 <code>INPUT</code> the data reply MP6 'Attempted' use of <code>Transmit</code> to send it in a loop MP7 Correct formation of parameters to <code>Transmit()</code>	7
------	---	---

53. M/J 23/P21/Q5/a,b

5(a)(i)	Reasons include: 1 No working software until late in the life cycle so slower to market than competitors // Does not allow the creation of early versions/prototypes (which can be updated later) 2 More difficult/slower to cope with changes to the requirements // website slower to be updated to reflect new requirements 3 Needs high involvement/feedback of the stake holders /customer / client One mark per point Max 2 marks	2
5(a)(ii)	Iterative / Rapid Application Development / RAD	1

5(b)	One mark for Stage Stage: Beta testing Max 2 marks for Description Description: 1 Testing carried out by a small group of (potential) users 2 Users will check that the website/software works as required / works in the real world // User will identify errors in the website/software 3 Users will feedback (problems) / suggestions for improvement 4 Problems / suggestions identified are addressed (before the program is sold)	3
------	---	----------

54. M/J 23/P21/Q7/b

7(b)(i)	One mark per point: • Module-A calls the other three modules • The process is repeated	2
7(b)(ii)	 One mark per bullet point: • All rectangles correctly labelled and interconnected • All and only parameters as shown • Selection diamond	3

55. M/J 23/P21/Q8/c

8(c)(i)	Integration testing	1
8(c)(ii)	Two marks for the description: • A dummy/simple module is written to replace the module that does not work properly • The dummy/simple module will return an expected value // will output a message to show it has been called	2

56. M/J 23/P22/Q5/a,b,d

5(a)	One mark per point: - parameter / Num has been passed by reference // should have been passed by value - so when the value / ThisNum is modified (in procedure DisplaySqrt()) - the new value will be used in the loop (lines 40–43) // Num will be changed to modified value	3
5(b)	The rules of the language have not been broken // there are no syntax errors	1
5(d)	Answers include: - Change the statement into a comment - Change the statement to a string representing a literal value and assign it to a variable / output it Note: max 1 mark	1

57. M/J 23/P22/Q7/b

7(b)	<pre> graph LR START((START)) --> S1((S1)) S1 -- "Input PIN" --> S2((S2)) S2 -- "Re-input PIN Display error" --> S2 S2 -- "Too many tries Block Account" --> S3((S3)) S2 -- "Valid PIN Enable payment" --> S4((S4)) S2 -- "Cancel Re-prompt" --> S1 </pre> One mark for each: - Line from S1 to S2 and label - S2 loop label - S3 circle and label on incoming - S4 circle and label on incoming	4
------	---	----------

58. M/J 23/P22/Q8/c

8(c)	One mark for method: Method: Stub testing Two marks for description: • The modules <code>SuppExists()</code> and <code>CheckSupplier()</code> are replaced by dummy modules • ...which return a known result / contain an output statement to show they have been called	3
------	--	----------

59. M/J 23/P23/Q1/b

1(b)	One mark per point: • The (second dimension/index of the) array is declared from 1 to 50 but the loop runs from 0 to 49 • Line number: 10 / 100 / 101 / 102	2
------	---	----------

60. M/J 23/P23/Q5/a,b

5(a)	Max 3 marks Additional Information: MP1 The (program/source) code/specification MP2 test plan // inputs/test data <u>and</u> expected outputs Explanation: MP3 The structure / design / algorithm of the program of the program needs to be known MP4 so that all paths through the program can be tested	3
5(b)	Perfective	1

61. M/J 23/P23/Q7/b

7(b)	Max 3 marks MP1 All modules correctly labelled and interconnected. MP2 Correct parameters and return type to Module-X and Reset MP3 Correct parameters and return type to Module-Y and Replace MP4 Correct parameters and return type to Overlay and Module-Y	3
------	---	----------

62. M/J 23/P23/Q8/c

8(c)(i)	Max 2 marks MP1 Must 'calculate' the count before any item + description output / after the file is read once MP2 Lines to be output have to be <u>stored</u> ... MP3 The file has to be read twice	2
8(c)(ii)	One mark per point: MP1 Loop through the file calculating the count MP2 Save 'selected' items in <u>an array</u> MP3 (After all lines have been read), output the header lines / count MP4 Loop through <u>the array</u> to output each array element	3

8(b)	<pre> PROCEDURE Report_1(Supp : STRING) DECLARE Count : INTEGER DECLARE ThisItemNum, ThisDesc, ThisLine, ThisCode : STRING Count ← 0 OPENFILE "Stock.txt" FOR READ OUTPUT "Report for Supplier:" & Supp OUTPUT "" //Blank line as per example OUTPUT "Item Description" OUTPUT "" //Blank line as per example WHILE NOT EOF("Stock.txt") READFILE("Stock.txt", ThisLine) ThisCode ← Mid(ThisLine, 5, 3) IF ThisCode = Supp THEN ThisItemNum ← LEFT(ThisLine, 4) ThisDesc ← RIGHT(ThisLine, LENGTH(ThisLine) - 7) OUTPUT ThisItem & " " & ThisDesc Count ← Count + 1 ENDIF ENDWHILE CLOSEFILE "Stock.txt" OUTPUT "" //Blank line as per example OUTPUT "Number of items listed: ", Count ENDPROCEDURE </pre> MP1 Output report header (blank lines optional) – Must contain the parameter code MP2 Conditional loop until EOF("Stock.txt") MP3 Read a line from Stock.txt AND extract SupplierCode in a loop MP4 Test if SupplierCode = Supp then increment count (must have been Initialised) MP5 Extract AND output item and description in a loop MP6 Output the final line with count	6
------	--	---

63. O/N 22/P21/Q6/b

6(b)(i)	One mark for each area as outlined:	5																																																																																										
<table><tr><th>Index</th><th>NextChar</th><th>Dots</th><th>Ats</th><th>Others</th><th>Valid</th></tr><tr><td></td><td></td><td>0</td><td>0</td><td>0</td><td>TRUE</td></tr><tr><td>1</td><td>'L'</td><td></td><td></td><td>1</td><td></td></tr><tr><td>2</td><td>'i'</td><td></td><td></td><td>2</td><td></td></tr><tr><td>3</td><td>'z'</td><td></td><td></td><td>3</td><td></td></tr><tr><td>4</td><td>'.'</td><td>1</td><td></td><td></td><td></td></tr><tr><td>5</td><td>'1'</td><td></td><td></td><td>4</td><td></td></tr><tr><td>6</td><td>'2'</td><td></td><td></td><td>5</td><td></td></tr><tr><td>7</td><td>'3'</td><td></td><td></td><td>6</td><td></td></tr><tr><td>8</td><td>'@'</td><td></td><td>1</td><td></td><td></td></tr><tr><td>9</td><td>'b'</td><td></td><td></td><td>7</td><td></td></tr><tr><td>10</td><td>'i'</td><td></td><td></td><td>8</td><td></td></tr><tr><td>11</td><td>'g'</td><td></td><td></td><td>9</td><td></td></tr><tr><td>12</td><td>'@'</td><td></td><td>2</td><td></td><td>FALSE</td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>			Index	NextChar	Dots	Ats	Others	Valid			0	0	0	TRUE	1	'L'			1		2	'i'			2		3	'z'			3		4	'.'	1				5	'1'			4		6	'2'			5		7	'3'			6		8	'@'		1			9	'b'			7		10	'i'			8		11	'g'			9		12	'@'		2		FALSE						
Index	NextChar	Dots	Ats	Others	Valid																																																																																							
		0	0	0	TRUE																																																																																							
1	'L'			1																																																																																								
2	'i'			2																																																																																								
3	'z'			3																																																																																								
4	'.'	1																																																																																										
5	'1'			4																																																																																								
6	'2'			5																																																																																								
7	'3'			6																																																																																								
8	'@'		1																																																																																									
9	'b'			7																																																																																								
10	'i'			8																																																																																								
11	'g'			9																																																																																								
12	'@'		2		FALSE																																																																																							
6(b)(ii)	FALSE	1																																																																																										

64. O/N 22/P21/Q7/b

7(b)	Example string: "23/0" (Any divide by zero example) Reason: The result is infinity / cannot be represented / is undefined // will cause the program to crash	2
------	---	----------

65. O/N 22/P21/Q8/d

8(d)	One mark for each (Max 2): Examples: 1 Incorrect block structure. Missing keyword denoting part of block (for example <code>ENDPROCEDURE</code> , <code>ENDFUNCTION</code> , <code>ENDTYPE</code>) 2 Data type errors, for example, assigning an integer value to a string 3 Identifier used before it is declared 4 Incorrect parameter use	2
------	---	----------

66. O/N 22/P22/Q1/b

1(b)	One mark for test stage, one mark for each description point (Max 3 for Description) Test stage: Beta testing Description: 1 Testing carried out by a small group of (potential) users 2 Users will check that the software works as required / works in the real world / does not contain errors 3 Users will feedback problems / suggestions for improvement 4 Problems / suggestions identified are addressed (before the program is sold)	4
1(a)(i)	One mark for each point (Max 2): • When a task which is repeated / reused / performed in several places • When a part of an algorithm performs a specific task • Reduces complexity of program / program is simplified // subroutine already available • Testing / debugging / maintenance is easier	2
1(a)(ii)	One mark for each part: Term: Parameter(s) Use: to pass values / arguments to the procedure	2

67. O/N 22/P22/Q2/b

2(b)	One mark per row (row 2 to 5): <table border="1"> <thead> <tr> <th>Input</th><th>Output</th><th>Next state</th></tr> </thead> <tbody> <tr> <td></td><td></td><td>S1</td></tr> <tr> <td>Input-A</td><td>Output-X</td><td>S2</td></tr> <tr> <td>Input-A</td><td>(none)</td><td>S2</td></tr> <tr> <td>Input-B</td><td>Output-W</td><td>S3</td></tr> <tr> <td>Input-A</td><td>Output-W</td><td>S4</td></tr> </tbody> </table>	Input	Output	Next state			S1	Input-A	Output-X	S2	Input-A	(none)	S2	Input-B	Output-W	S3	Input-A	Output-W	S4	4
Input	Output	Next state																		
		S1																		
Input-A	Output-X	S2																		
Input-A	(none)	S2																		
Input-B	Output-W	S3																		
Input-A	Output-W	S4																		

68. O/N 22/P22/Q6/b

6(b)	One mark per row. Examples of invalid strings: 1 Non-numeric but not "End" // contains space or other non-numeric characters 2 Real number 3 Integer value out of range (i.e. ≤ 0) 4 Empty string 5 Correct word but wrong case e.g. "end" rather than "End" <table><tr><th>Input</th><th>Reason for choice</th></tr><tr><td>"Aardvark"</td><td>Non-numeric (and not "End")</td></tr><tr><td>"27.3"</td><td>Numeric but not an integer</td></tr><tr><td>"-3" // "0"</td><td>A non-positive integer</td></tr><tr><td>""</td><td>An empty string</td></tr></table>	Input	Reason for choice	"Aardvark"	Non-numeric (and not "End")	"27.3"	Numeric but not an integer	"-3" // "0"	A non-positive integer	""	An empty string	4
Input	Reason for choice											
"Aardvark"	Non-numeric (and not "End")											
"27.3"	Numeric but not an integer											
"-3" // "0"	A non-positive integer											
""	An empty string											

69. O/N 22/P23/Q1/b,c,d

1(b)(i)	Easier to manage/plan/cost // Clear deliverables produced at (end of) each stage	1
1(b)(ii)	One mark per point (Max 1): • The problem definition • Requirements specification // Client requirements • Documentation related to current system (e.g. ER diagram of current system, DFD of current system, feasibility study)	1
1(c)(i)	One mark per point (Max 1): • Modules are developed in parallel / as prototypes • Minimal / no detailed planning is carried out // Allows for changes to requirements • Flexible development process • Small incremental releases are made, each adding functionality • Used for time critical development • Client involved during (all stages) of development	1

1(c)(ii)	Examples include: Benefits: (Max 2 marks) • Quicker development possible / Multiple areas can be worked on at same time • Prototype produced (at early stage in process) • Easier to change requirements / quicker delivery of usable modules • Early review possible / closer cooperation between client and developers Drawback: (Max 1 mark) • Difficult to estimate cost / time to complete project • Documentation often omitted • Lack of client availability throughout life cycle // too easy for client to keep changing their mind	3
1(d)	One mark for each point (Max 2) Examples include: 1 Change to website requirements 2 New technologies available to host website // changes made to library modules used 3 Change in relevant legislation	2

70. O/N 22/P23/Q5/d

5(d)	One mark per point (Max 3) 1 The program is checked by creating a trace table / going through the program a line at a time 2to record/check variable (values) as they change 3 Error may be indicated when variable given an unexpected value 4 Error may be indicated by an unexpected path through the program // Faults in the logic of the program can be detected	3
------	--	----------

71. O/N 22/P23/Q6/a

6(a)	One mark per row	5																														
	Examples:																															
	<table><tr><th>Test number</th><th>Component weight</th><th>Min</th><th>Max</th><th>Expected return value</th></tr><tr><td>1</td><td>300</td><td>290</td><td>315</td><td>'A'</td></tr><tr><td>2</td><td>> 317</td><td>290</td><td>315</td><td>'R'</td></tr><tr><td>3</td><td>317</td><td>290</td><td>315</td><td>'C'</td></tr><tr><td>4</td><td>288</td><td>290</td><td>315</td><td>'C'</td></tr><tr><td>5</td><td>< 288</td><td>290</td><td>315</td><td>'R'</td></tr></table>	Test number	Component weight	Min	Max	Expected return value	1	300	290	315	'A'	2	> 317	290	315	'R'	3	317	290	315	'C'	4	288	290	315	'C'	5	< 288	290	315	'R'	
Test number	Component weight	Min	Max	Expected return value																												
1	300	290	315	'A'																												
2	> 317	290	315	'R'																												
3	317	290	315	'C'																												
4	288	290	315	'C'																												
5	< 288	290	315	'R'																												

72. O/N 21/P21/Q1/a

1(a)(i)	One from: - The program obeys the rules / grammar of the programming language used - The program will run // it can be compiled / interpreted - Accept by example. e.g. 'no mis-spelt keywords' / 'all brackets match'	1
1(a)(ii)	One mark for type plus one for corresponding description Type of error: A logic error Description: - An error in the algorithm / design of the solution - the program does not behave as expected / give the expected output. - Accept by example e.g. wrong arithmetic operator used / wrong loop count OR Type of error: Run-time error Description: - The program performs an illegal instruction / invalid operation - Accept by example: divide by zero or endless loop or simply 'crashes' / freezes	2

73. O/N 21/P21/Q4/a,b

4(a)

Test	Test data value	Explanation	Expected Outcome
1	23	Normal Data	Data is accepted
2	0	Boundary Data	Data is accepted
3	40	Boundary Data	Data is accepted
4	>= 41	Abnormal Data	Data is rejected
5	<= -1	Abnormal Data	Data is rejected

One mark per row for rows 2 to 5.

4

4(b)	<pre> graph LR Start((Start)) --> Off((Heaters Off)) Off -- "Temp >= 10" --> Off Off -- "Temp < 10" --> On((Heaters On)) On -- "Temp <= 20" --> On On -- "Temp > 20" --> Off </pre> One mark for each label: • Temp < 10 (Heaters Off to Heaters On) • Temp > 20 (Heaters On to Heaters Off) • on BOTH loops (non-contradictory values)	3
------	--	---

74. O/N 21/P21/Q5/b

5(b)	Design	1
------	--------	---

75. O/N 21/P22/Q2/b

2(b)(i)	One mark for each: Life cycle method: Iterative // Rapid Application Development (RAD) Reason: Provides a working model / prototype at an early stage for the principal to approve / review	2
2(b)(ii)	Decisions will be made regarding: • Data structures • Algorithms / flowcharts / pseudocode • Program structure (modules) / use of library routines / module - team allocation • User interface // Web-page layout / content (for given scenario) • Testing method / plan • Choice of programming language / program environment Max 3	3

76. O/N 21/P22/Q6/c(i)

6(c)(i)	One mark for: • Parameter(s) need to be passed to the module to identify the type of search • Search algorithm is controlled by (global) variables / parameters Alternative: • The search algorithms from the original modules are included in the new module • The new module needs to return / store the four values (the results of the four searches)	2
---------	---	----------

77. O/N 21/P23/Q1/a

1(a)(i)	One from: • The program obeys the rules / grammar of the programming language used • The program will run // it can be compiled / interpreted • Accept by example. e.g. 'no mis-spelt keywords' / 'all brackets match'	1
1(a)(ii)	One mark for type plus one for corresponding description Type of error: A logic error Description: • An error in the algorithm / design of the solution • the program does not behave as expected / give the expected output. • Accept by example e.g. wrong arithmetic operator used / wrong loop count OR Type of error: Run-time error Description: • The program performs an illegal instruction / invalid operation • Accept by example: divide by zero or endless loop or simply 'crashes' / freezes	2

78. O/N 21/P23/Q4/a,b

4(a)	Test	Test data value	Explanation	Expected Outcome	4
	1	23	Normal Data	Data is accepted	
	2	0	Boundary Data	Data is accepted	
	3	40	Boundary Data	Data is accepted	
	4	>= 41	Abnormal Data	Data is rejected	
	5	<= -1	Abnormal Data	Data is rejected	
One mark per row for rows 2 to 5.					

4(b)	<pre> graph LR Start((Start)) --> Off((Heaters Off)) Off -- "Temp >= 10" --> Off Off -- "Temp < 10" --> On((Heaters On)) On -- "Temp <= 20" --> On On -- "Temp > 20" --> Off </pre> One mark for each label: • Temp < 10 (Heaters Off to Heaters On) • Temp > 20 (Heaters On to Heaters Off) • on BOTH loops (non-contradictory values)	3
------	--	----------

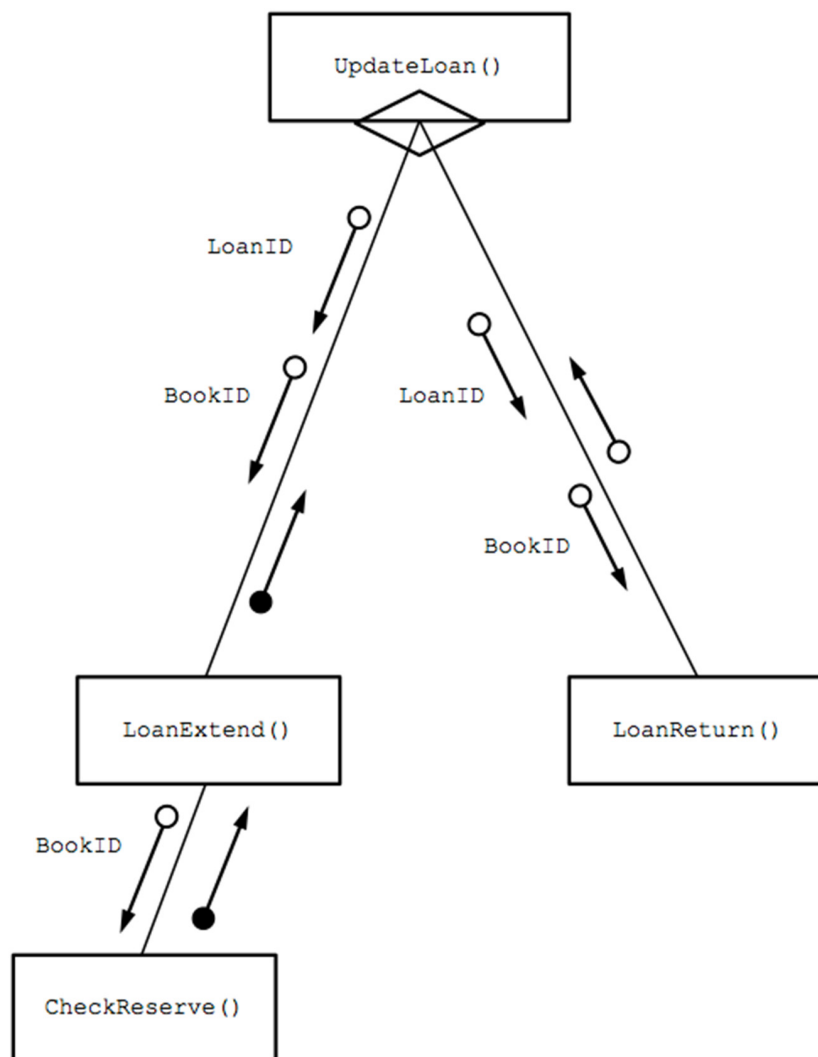
79. O/N 21/P23/Q5/b

5(b)	Design	1
------	--------	----------

80. M/J 21/P21/Q2/a

2(a)

5



One mark for:

- 1 All four boxes correctly labelled and positioned
- 2 Selection diamond (only on `UpdateLoan` and no iteration arrows)
- 3 Parameters to and from `LoanExtend()`
- 4 Parameters to and from `CheckReserve()`
- 5 Parameters to and from `LoanReturn()`

81. M/J 21/P21/Q4/a,c

4(a)

5

Name	Flag	Index	NewName	ThisChar
"VinVaVVcup"				
	TRUE	1	""	
				'v'
			"v"	
		2		'i'
	FALSE		"vI"	
		3		'n'
			"vIn"	
		4		'v'
	TRUE		"vInv"	
		5		'a'
	FALSE		"vInvA"	
		6		'v'
	TRUE		"vInvAv"	
		7		'v'
			"vInvAvv"	
		8		'c'
	FALSE		"vInvAvvC"	
		9		'u'
			"vInvAvvCu"	
		10		'p'
			"vInvAvvCup"	
	FALSE	11	"vInvAvvCup"	'p'

Mark as follows:

One mark for each of columns 2 to 5 (condone missing final 11)

One mark for final row all correct (including final 11)

4(c)(i)	A couple of solutions: <pre>24 ThisChar ← LCASE (MID (Name, Index, 1)</pre> ALTERNATIVE: <pre>31 NewName ← NewName & LCASE (ThisChar)</pre> Ignore line number	1
4(c)(ii)	One mark for each: Line number: 26 New position: Move to after line 27 / line 28	2

82. M/J 21/P22/Q2/a

2(a)(i)	The number of transitions that result in a different state	3	4
	The number of transitions with associated outputs	2	
	The label that should replace 'X'	Start	
	The final or halting state	S3	
	One mark per row		
2(a)(ii)	Number of outputs: 1 Current state: S2	2	

83. M/J 21/P22/Q7/a,b

7(a)	Test string should contain: • At least one leading space (before the first word) • at least one trailing space (after the last word) • at least one instance of multiple spaces between words • At least one upper case character Mark as follows: One mark for one correct Two marks for three correct Three marks for all correct	3
------	---	----------

7(b)	One mark for each: • Dry run / produce a trace table / walk through the code • Add output statements to allow the code to be tracked • Insert a Breakpoint into the program // use single-stepping to execute instructions // monitor variables using a watch window • Try different test values to see which ones fail Note: Max 2 marks	2
------	---	----------

84. M/J 21/P23/Q2/a

2(a)	<pre> graph TD UpdateLoan[UpdateLoan()] LoanExtend[LoanExtend()] CheckReserve[CheckReserve()] LoanReturn[LoanReturn()] UpdateLoan -- LoanID --> LoanExtend LoanExtend -- BookID --> CheckReserve CheckReserve --> LoanExtend UpdateLoan -- LoanID --> LoanReturn LoanReturn -- BookID --> UpdateLoan </pre> The diagram shows four activity boxes: <code>UpdateLoan()</code> at the top, <code>LoanExtend()</code> at the bottom left, <code>CheckReserve()</code> at the bottom left below <code>LoanExtend()</code>, and <code>LoanReturn()</code> at the bottom right. A selection diamond is located at the bottom of the <code>UpdateLoan()</code> box. Two paths emerge from the diamond: one to <code>LoanExtend()</code> labeled <code>LoanID</code>, and another to <code>LoanReturn()</code> labeled <code>LoanID</code>. From <code>LoanExtend()</code>, an arrow labeled <code>BookID</code> points to <code>CheckReserve()</code>, which then has an arrow pointing back to <code>LoanExtend()</code>. From <code>LoanReturn()</code>, an arrow labeled <code>BookID</code> points back to the selection diamond in <code>UpdateLoan()</code>. One mark for: 1 All four boxes correctly labelled and positioned 2 Selection diamond (only on <code>UpdateLoan()</code> and no iteration arrows) 3 Parameters to and from <code>LoanExtend()</code> 4 Parameters to and from <code>CheckReserve()</code> 5 Parameters to and from <code>LoanReturn()</code>	5
------	---	----------

85. M/J 21/P23/Q4/a,c

4(a)						5
	Name	Flag	Index	NewName	ThisChar	
	"VinVaVVcup"					
		TRUE	1	" "		
					'v'	
				"v"		
			2		'i'	
		FALSE		"vI"		
			3		'n'	
				"vIn"		
			4		'v'	
		TRUE		"vInv"		
			5		'a'	
		FALSE		"vInvA"		
			6		'v'	
		TRUE		"vInvAv"		
			7		'v'	
				"vInvAvv"		
			8		'c'	
		FALSE		"vInvAvvC"		
			9		'u'	
				"vInvAvvCu"		
			10		'p'	
				"vInvAvvCup"		
		FALSE	11	"vInvAvvCup"	'p'	

Mark as follows:

- One mark for each of columns 2 to 5 (condone missing final 11)
- One mark for final row all correct (including final 11)

4(c)(i)	A couple of solutions: <pre>24 ThisChar ← LCASE (MID (Name, Index, 1)</pre> ALTERNATIVE: <pre>31 NewName ← NewName & LCASE (ThisChar)</pre> Ignore line number	1
4(c)(ii)	One mark for each: Line number: 26 New position: Move to after line 27 / line 28	2